



POLITECNICO DI BARI

INGEGNERIA DELL'AUTOMAZIONE

Identificazione e Controllo Intelligente

APPUNTI DEL CORSO

A.A. 2017/2018

a cura di

Luca Mennoia

Indice

I	Identificazione lineare	1
1	Identificazione di sistemi lineari	2
1.1	Introduzione	2
1.2	Modelli matematici	4
1.3	Identificazione basata su modello	6
2	Famiglie di modelli	8
2.1	Modelli di Box-Jenkins	10
2.2	Modelli Output Error e Input Error	11
2.3	Modelli ARX	12
2.4	Modelli ARMAX	13
2.5	Modelli ARARX	14
2.6	Modelli AR, ARMA e ARAR	15
3	Predittori	17
3.1	Estrazione del predittore	17
3.2	Predittore ARX	19
3.3	Predittore ARMAX	20
3.4	Predittore ARARX	21
4	Validazione dei modelli	22
4.1	Analisi asintotica	22
4.2	Analisi di bianchezza	25
4.3	Quantificazione dell'errore di stima parametrica	26
4.4	Ordine ottimo di un modello	27
5	Ottimizzazione ai minimi quadrati	29
5.1	Algoritmo dei minimi quadrati	29
5.2	Applicazione a modelli ARX	31
5.3	Analisi probabilistica	32
5.4	Persistente eccitazione	33
5.5	Algoritmo dei minimi quadrati estesi	36
5.6	Identificabilità	38
5.7	Analisi spettrale	38
5.8	Prefiltraggio	40
5.9	Operazioni di pre-processing	41

II	Identificazione non lineare	44
6	Identificazione di sistemi non lineari	45
6.1	Da lineare a non lineare	45
6.2	Modelli di Hammerstein e Wiener	46
6.3	Scelta del segnale di ingresso	47
6.4	Algoritmi a discesa del gradiente	50
6.5	Modelli alle variazioni e modelli affini al controllo	51
7	Approssimatori e reti neurali artificiali	54
7.1	Approssimatori	54
7.2	Reti neurali artificiali	56
7.2.1	Introduzione	56
7.2.2	Topologie e strutture	57
7.2.3	Rappresentazione di una rete neurale artificiale	58
7.2.4	Retropropagazione	64
7.2.5	Reti neurali a base radiale	71
7.2.6	Considerazioni conclusive	74
	Appendice	76
A	Teoria della probabilità	77
A.1	Correlazione e covarianza	77
B	Algebra lineare	79
B.1	Test di Sylvester	79

Parte I

Identificazione lineare

Capitolo 1

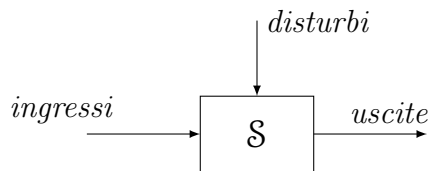
Identificazione di sistemi lineari

1.1 Introduzione

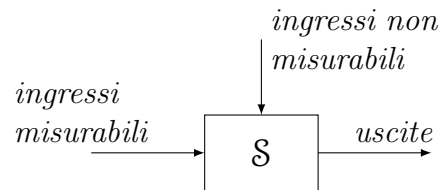
Identificare significa trovare un **modello matematico** basandosi su **dati sperimentali**.

Si riconsideri la nozione di *sistema*, ben nota nel contesto dell'automazione e in particolare in ambito controllistico: un sistema è essenzialmente un complesso di elementi interagenti che produce una o più **uscite**, influenzate da **ingressi** e **disturbi**.

Il modello che descrive questo comportamento è illustrato in fig. 1.1a.



(a) modello per il controllo



(b) modello per l'identificazione

Figura 1.1: Modelli di sistemi orientati ai fini del controllo e dell'identificazione

Nei controlli automatici, si trattano i disturbi come segnali la cui dinamica e le cui caratteristiche sono completamente note (e.g. rumore bianco).

Nella realtà, i disturbi non sono sempre misurabili: in molti casi non ne si conosce la dinamica, oppure si è in presenza di fattori che influenzano il sistema in maniera talmente complessa da non poter esser modellati adeguatamente.

Fenomeni di questo tipo sono spesso incontrollabili, incalcolabili, di natura casuale o addirittura sconosciuti e assumono un ruolo rilevante nell'ambito dell'identificazione, venendo chiamati **ingressi non misurabili**. Di conseguenza, gli ingressi controllati dall'utente prendono il nome di **ingressi misurabili** (fig. 1.1b).

Esempio 1. *Sistema di controllo di velocità cruise-control per veicolo*

Un sistema di *cruise-control* per veicolo può essere modellato individuando un ingresso θ , corrispondente all'angolo del pedale acceleratore e un'uscita v , che rappresenta la velocità del veicolo ed è la variabile che di fatto viene controllata.

Durante la marcia, il sistema è sollecitato da una serie di disturbi causati da fattori come la pendenza della strada ($I\%$), la velocità del vento, etc.

Sebbene queste siano grandezze misurabili, potrebbe risultare sconveniente o anche non necessario effettuarne una misura, ragion per la quale esse sono modellate come ingressi non misurabili.

Esempio 2. *Controllo di rotta per veicolo marittimo*

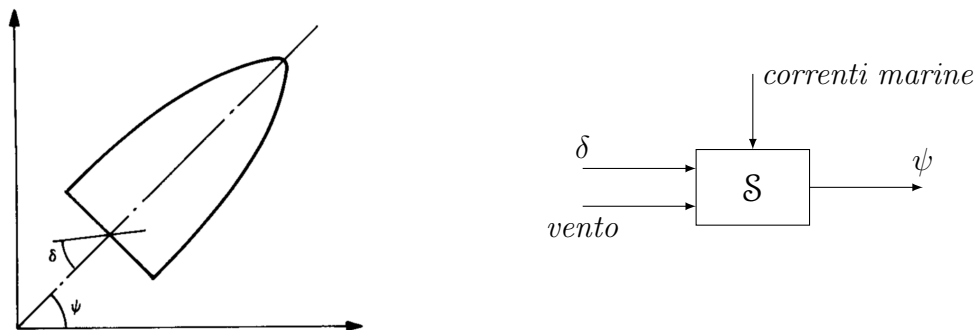


Figura 1.2: Controllo di rotta per un veicolo marittimo

Un sistema di controllo per la rotta di una barca un ingresso misurabile può essere individuato nell'angolo δ del timone, misurato rispetto alla direzione della rotta desiderata. L'angolo ψ , misurato sempre rispetto alla rotta desiderata può essere un'uscita del sistema. Le correnti marittime e il vento sono perturbazioni per il sistema e si possono considerare come ingressi non misurabili.

Esempio 3. *Controllo solare-termico*

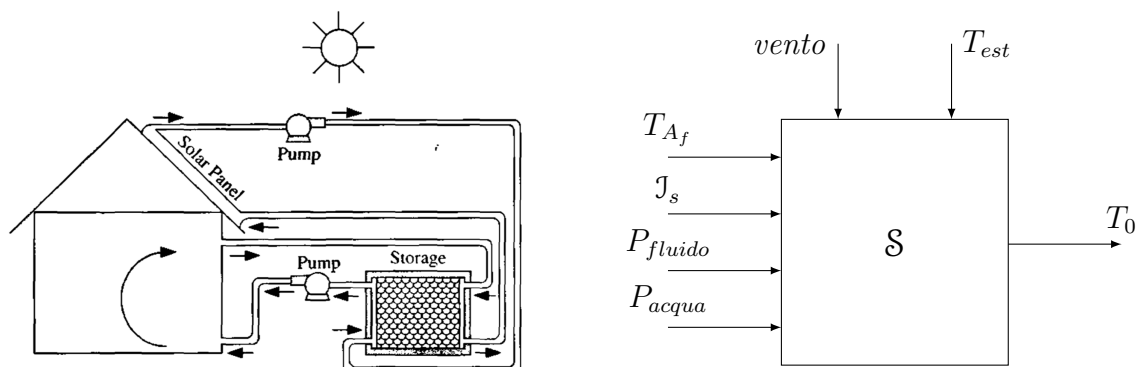


Figura 1.3: Controllo di un sistema solare-termico

Un sistema di controllo per l'accumulo di acqua calda basato sull'energia solare (*solare-termico*) è illustrato in figura 1.3: la temperatura dell'acqua è l'uscita del sistema in quanto

se ne vuole controllare il valore. I raggi solari riscaldano il pannello solare in cui è presente un fluido di scambio termico, spinto tramite una pompa in un sistema di scambio termico da cui esce acqua calda a temperatura T_0 ed entra acqua fredda. Gli ingressi misurabili che influenzano il comportamento del sistema sono l'irraggiamento solare J_s , la temperatura dell'acqua fredda T_{Af} , la portata per il fluido di scambio P_{fluido} e la portata in uscita dell'acqua calda P_{acqua} . Gli ingressi non misurabili possono essere il vento e la temperatura dell'ambiente esterno T_{est} .

Esempio 4. *Modello matematico della voce umana*

La voce umana è determinata essenzialmente da due variabili: la tensione delle corde vocali $\mathcal{T}$ e l'aria. Queste due variabili sono ingressi pressoché impossibili da misurare, pertanto la voce umana (variabile in uscita dal sistema) è modellata da un sistema privo di ingressi misurabili (fig. 1.4).

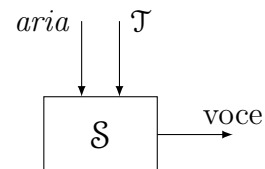


Figura 1.4: modello della voce umana

In quest'ultimo esempio si adopera un modello matematico privo di ingressi misurabili che prende il nome di **serie temporale**:

$$v(t) = f(v(t-1), v(t-2), \dots) \quad (1.1)$$

Le serie temporali sono state impiegate in borsa per predire il valore dei titoli azionari e, pur presentando un'alta incertezza, hanno portato ad enormi guadagni. Per questa ragione un buon modello non deve necessariamente presentare una bassa incertezza ma deve essere all'altezza di raggiungere gli scopi prefissi.

1.2 Modelli matematici

I **modelli** possono essere essenzialmente di **quattro tipi**: i **modelli mentali** sono realizzati da un operatore con finalità **autodescrittive circa il funzionamento** di un sistema. Con un **modello numerico o grafico** si descrive il comportamento del sistema attraverso una mappa statica (i.e. caratteristica di un diodo). Un **modello software** è realizzato mediante codice: grafi, alberi, pile, code, automi finiti deterministici, reti logiche sono esempi di modelli software. Infine un **modello matematico** descrive i legami tra le variabili: questi modelli possono essere statici o dinamici, lineari o non lineari, etc. e ve ne sono essenzialmente sei possibili utilizzi.

1. **Predizione**: i predittori sono dispositivi che predicono l'uscita basandosi sulla **storia precedente** del sistema. I predittori sono caratterizzati da un **numero di passi** (un passo, due passi, tre passi, etc.) che rappresenta il numero di campioni con cui viene effettuata la predizione dell'uscita del sistema. Le variabili in ingresso per un modello predittore sono dunque sia gli **ingressi** propriamente detti che le **uscite ritardate** di un certo numero finito di passi; ne consegue una stima dell'uscita $\hat{y}$.

2. **Simulazione**: in questo caso non si impiega il modello per effettuare calcoli e simulazioni al fine di ottenere un'uscita da comparare a quelli sperimentali che offrono i valori veri dell'uscita. Può accadere, nelle simulazioni, di avere un **gap molto ampio** fra l'uscita simulata $\tilde{y}$ e y . Si deve tener sempre presente che i valori in uscita da un modello di simulazione sono appunto simulati e non è detto che coincidano con i valori veri. Se si retroaziona l'uscita di un modello simulatore si otterrebbe lo schema di un predittore (o stimatore) ma il sistema rimarrebbe comunque un simulatore, dal momento che le uscite simulate possono differire largamente rispetto a quelle stimate, le quali dipendono dai valori veri delle uscite in istanti di tempo precedenti.
3. **Ottimizzazione**: un modello può essere anche usato ai fini dell'ottimizzazione. Tipicamente si **studia** l'ottimalità del modello mediante una **strategia di validazione** e lo si **ottimizza** impiegando una **strategia di miglioramento**, reiterando eventualmente il processo.
4. **Controllo**: un modello utilizzato ai fini del controllo è impiegato per effettuare il controllo in catena chiusa di un processo tramite un controllore.

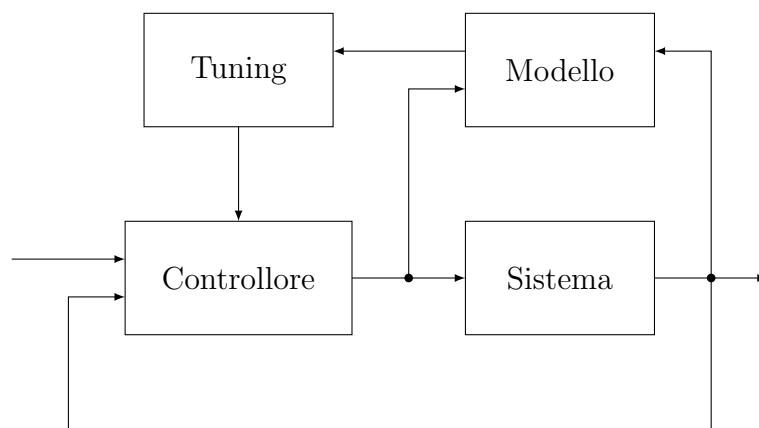


Figura 1.5: schema di un modello orientato ai fini del controllo adattativo

5. **Analisi**: un modello impiegato per l'analisi aiuta a **comprendere** quali sono le **domande alle quali il modello stesso può rispondere**: l'implementazione di un sistema dinamico pone delle domande; la formulazione di un modello offre delle potenziali risposte.
6. **Fault Diagnosis**: un modello di questo tipo è impiegato per la diagnosi dei guasti ed ingloba a sua volta diverse versioni del sistema, modellato sotto le ipotesi di specifici malfunzionamenti. Si ricavano diversi modelli del sistema in questione, **uno nominale e diversi modelli *fault***, in cui si modellano alcune condizioni di funzionamento difettoso del sistema. Calcolando gli **errori** fra l'uscita del sistema e le uscite di ciascun modello si riesce ad avere una misura numerica di quanto si è vicini a una **condizione di guasto**. Quanto più ci si allontana dal modello nominale, più si concretizza l'ipotesi che il sistema sia guasto.

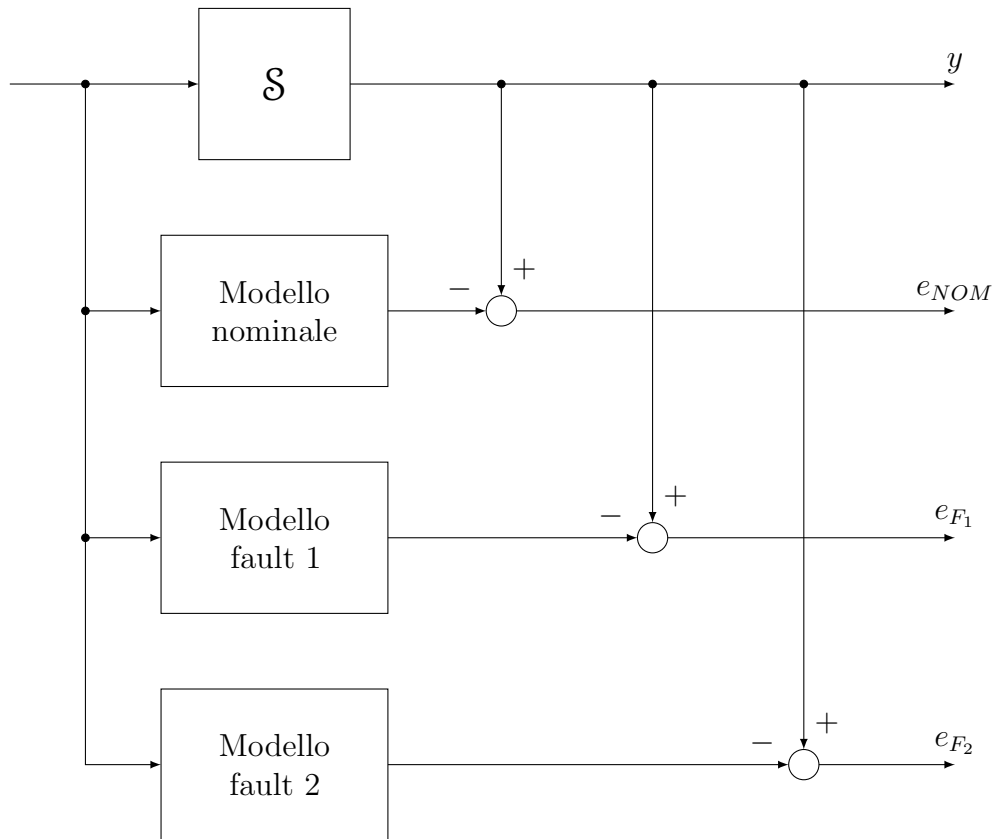


Figura 1.6: schema di un modello orientato ai fini di diagnosi dei guasti

1.3 Identificazione basata su modello

L'identificazione basata su modello fonda le sue basi su tre *requisiti chiave*.

- **Dati sperimentali:** i dati sperimentali devono essere ricchi di informazione sul processo da identificare e devono essere sufficientemente rappresentativi. La scelta e la rilevazione dei dati sperimentali è fatta mediante il progetto dell'esperimento (*experiment design*).
- **Famiglie di modelli:** la scelta del modello non è semplice in quanto serve esperienza pregressa e intuizione ingegneristica per capire quale famiglia di modelli scegliere. In genere si parte da modelli relativamente semplici per poi aumentarne il dettaglio in base al fitting dei risultati.
- **Criteri di scelta:** si definisce un meccanismo di generazione dei dati. Tipicamente si svolgono valutazioni sugli errori per avere una misura della bontà del modello identificante un dato processo. Naturalmente la procedura di validazione va fatta sui dati in uscita dal modello e non su quelli di training del modello stesso: se si istruisce un modello fornendo in ingresso dei dati specifici non ha senso valutare come il modello risponde agli stessi dati; bisogna interrogare il modello con ingressi diversi da quelli impiegati per il training e poi valutare gli scostamenti delle uscite ottenute dai valori veri.

Nella figura seguente è illustrato un diagramma di flusso che rappresenta le fasi di una corretta procedura di identificazione.

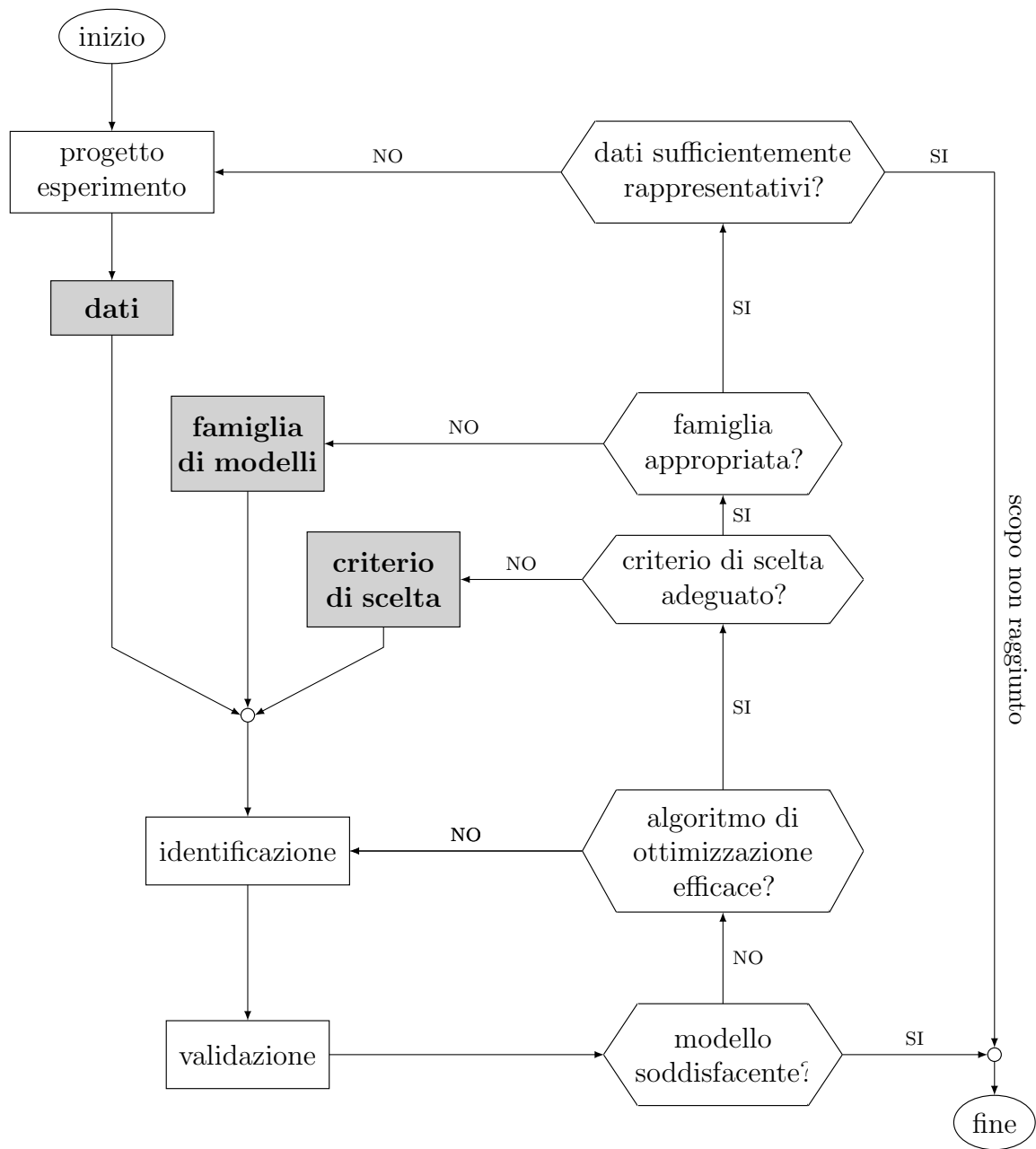


Figura 1.7: Struttura di un processo di identificazione

Capitolo 2

Famiglie di modelli

Le famiglie di modelli presentate di seguito si riferiscono a sistemi LTI (*Linear Time Invariant*), SISO (*Single Input Single Output*), tempo discreto.

Il processo di acquisizione di dati sperimentali non può essere effettuato in un dominio di tempo continuo: tale processo è intrinsecamente di natura discreta.

Vengono presi in considerazione solamente i modelli con funzioni di trasferimento strettamente proprie (ossia con un numero di poli maggiore del numero di zeri) in quanto esiste sempre un ritardo di almeno un passo fra l'ingresso e l'uscita.

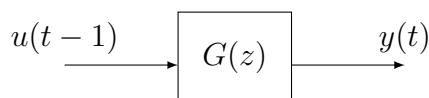


Figura 2.1: modello semplice di un sistema LTI, SISO, tempo discreto per identificazione

La compresenza di segnali nel dominio del tempo t e di blocchi di trasferimento rappresentati in funzione della frequenza complessa z negli stessi schemi costituisce un abuso di notazione che ha il fine di alleggerire, d'ora in avanti, la trattazione (per rappresentare in modo formalmente corretto lo schema in figura 2.1 si deve sostituire $u(t-1)$ con $\mathcal{Z}^{-1}\{u(t-1)\}$ e $y(t)$ con $\mathcal{Z}^{-1}\{y(t)\}$).

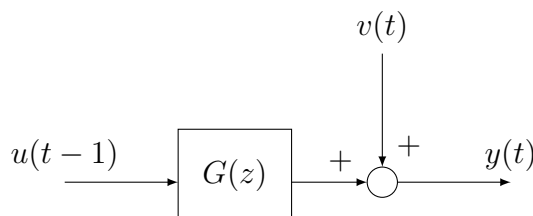


Figura 2.2: modello completo di un sistema LTI, SISO, tempo discreto per identificazione

L'ingresso misurabile $u(t)$ è detto **variabile esogena** (che proviene dall'esterno), mentre la **variabile rumore** $v(t)$ è un segnale canonico in grado di descrivere bene i disturbi sconosciuti e non misurabili. La variabile $v(t)$ è un segnale definito come l'uscita di una serie temporale.

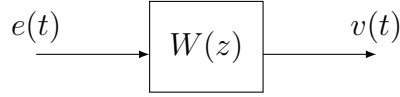


Figura 2.3: modello della variabile rumore

Il segnale $e(t)$ è un rumore bianco, che rappresenta la casualità per eccellenza. Il rumore bianco ha uno spettro uniforme a tutte le frequenze ed è un esempio di **segnale probabilistico i.i.d.** (identicamente distribuito, ossia con campioni completamente scorrelati e identicamente distribuiti).

Se la funzione di trasferimento $W(z)$ è asintoticamente stabile, lo spettro del segnale in uscita $\Phi_v(\omega)$ è pari al prodotto tra il modulo della funzione di risposta armonica discreta e lo spettro del segnale in ingresso $\Phi_e(\omega)$. In simboli:

$$\Phi_v(\omega) = |W(e^{j\omega})| \cdot \Phi_e(\omega) \quad (2.1)$$

La funzione di trasferimento $W(z)$ rappresenta un **filtro lineare** che, attraversato dal rumore bianco $e(t)$, produce il segnale $v(t)$, interpretabile come **rumore colorato** (poiché il filtro lineare seleziona una porzione dello spettro del segnale in ingresso e la produce in uscita).

Una funzione di trasferimento può essere espressa mediante una **rappresentazione parametrica** o mediante una **rappresentazione non parametrica**.

- **Rappresentazione parametrica**

$$G(z) = k \frac{z^m + b_1 z^{m-1} + \dots + b_m}{z^n + a_1 z^{n-1} + \dots + b_n} \quad (2.2)$$

Identificare un modello in forma parametrica consiste nell'identificazione dei coefficienti del numeratore e del denominatore della funzione di trasferimento $G(z)$.

- **Rappresentazione non parametrica**

$$y(t) = \sum_{i=1}^{\infty} g(t-i)u(i) \quad (2.3)$$

L'identificazione di un modello in forma non parametrica si ottiene tramite un algoritmo che implementi il metodo della lunga divisione.

Tutte le considerazioni (precedenti e seguenti) su modelli e su famiglie di modelli si possono estendere a sistemi MIMO passando dalle funzioni di trasferimento alle matrici di trasferimento.

2.1 Modelli di Box-Jenkins

La famiglia di modelli più generica, che include tutte le altre, è la **famiglia di Box-Jenkins**. Un modello di Box-Jenkins è schematizzato come segue:

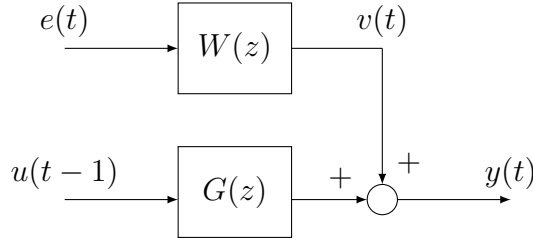


Figura 2.4: modello di Box-Jenkins

Il modello di Box-Jenkins rispetta le seguenti ipotesi:

1. $W(z)$ è asintoticamente stabile, quindi ha tutte le radici del polinomio a denominatore entro il cerchio di raggio unitario;
2. i polinomi a numeratore e a denominatore della $W(z)$ sono monici, coprimi e di uguale grado (la funzione è propria);
3. $G(z)$ è asintoticamente stabile, quindi ha tutte le radici del polinomio a denominatore entro il cerchio di raggio unitario;
4. $G(z)$ è strettamente propria, ovvero ha un numero di poli uguale al numero di zeri.

Le equazioni¹ del modello sono le seguenti:

$$\begin{aligned} y(t) &= G(z) \circ u(t-1) + W(z) \circ e(t) \\ v(t) &= W(z) \circ e(t) \end{aligned} \tag{2.4}$$

Per specificare un modello di Box-Jenkins bisogna dunque determinare le funzioni di trasferimento $G(z)$ e $W(z)$ insieme alle grandezze che caratterizzano il rumore bianco: tipicamente il **valore atteso** (che di norma è nullo) e la **varianza**, la quale fornisce un'indicazione sull'entità dei segnali non misurabili.

Il modello di Box-Jenkins è generico e porta ad algoritmi di identificazione piuttosto complessi, per questa ragione vi sono diverse famiglie di modelli più semplici, estratte da quella di Box-Jenkins, che meglio si prestano all'identificazione.

¹con l'operatore funzionale $\circ$ si indica che prima di valutare il prodotto tra una funzione in z e un segnale del tempo bisogna rappresentare quest'ultimo mediante la sua $\mathcal{Z}$ -trasformata.

2.2 Modelli Output Error e Input Error

Una famiglia di modelli molto comune, estratta da quella di Box-Jenkins, è chiamata **famiglia Output Error**, in cui si assume $W(z) = 1$.

$$\begin{aligned} y(t) &= G(z) \circ u(t-1) + e(t) \\ v(t) &= e(t) \end{aligned} \tag{2.5}$$

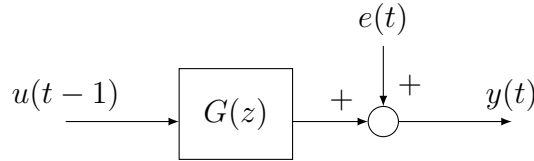


Figura 2.5: Modello Output Error

La **famiglia Input Error** è meno comune e prevede l'uscita della serie temporale sommata alla variabile esogena a monte del blocco di trasferimento $G(z)$ e $W(z) = 1$.

$$\begin{aligned} y(t) &= G(z) \circ [u(t-1) + e(t)] \\ v(t) &= e(t) \end{aligned} \tag{2.6}$$

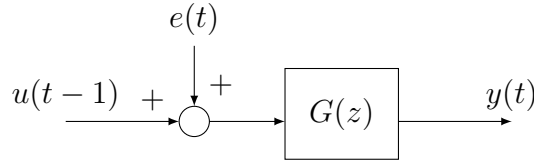


Figura 2.6: Modello Input Error

In questi modelli il filtro lineare $W(z)$ è assente pertanto l'uscita della serie temporale $v(t)$ coincide con il rumore bianco $e(t)$. I modelli Output Error prevedono un errore che si somma al segnale in uscita dal blocco $G(z)$ mentre nei modelli Input Error l'errore si somma al segnale in ingresso al blocco $G(z)$.

2.3 Modelli ARX

I **modelli ARX** (*AutoRegressive eXogenous Models*) sono modelli a due gradi di libertà² che prevedono un'uscita costituita dalla somma di tre componenti:

- un contributo **autoregressivo** (AR) sulla uscita³;
- il contributo puro della variabile errore;
- un contributo **esogeno** (X) in media mobile.

$$y(t) = \underbrace{a_1 \cdot y(t-1) + \dots + a_{n_a} \cdot y(t-n_a)}_{AR} + e(t) + \underbrace{b_1 \cdot u(t-1) + \dots + b_{n_b} \cdot u(t-n_b)}_X \quad (2.7)$$

Per la proprietà di traslazione nel tempo della $\mathcal{Z}$ -trasformata si ha:

$$y(t) \circ \underbrace{[1 - a_1 z^{-1} - a_2 z^{-2} - \dots - a_{n_a} z^{-n_a}]}_{A(z)} = u(t-1) \circ \underbrace{[b_1 + b_2 z^{-1} + \dots + b_{n_b} z^{-n_b+1}]}_{B(z)} + e(t) \quad (2.8)$$

con:

$$\begin{aligned} A(z) &= 1 - a_1 z^{-1} - a_2 z^{-2} - \dots - a_{n_a} z^{-n_a} \\ B(z) &= b_1 + b_2 z^{-1} + b_3 z^{-2} + \dots + b_{n_b} z^{-n_b+1} \end{aligned}$$

Si può riscrivere la 2.8 come segue:

$$\begin{aligned} A(z) \circ y(t) &= B(z) \circ u(t-1) + e(t) \\ \Downarrow \\ y(t) &= \frac{B(z)}{A(z)} \circ u(t-1) + \frac{1}{A(z)} \circ e(t) \end{aligned} \quad (2.9)$$

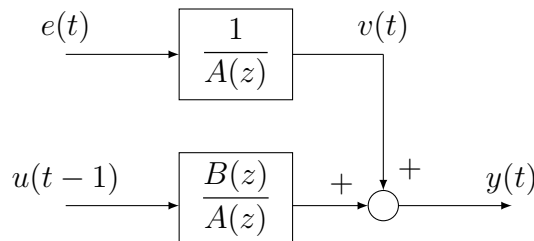


Figura 2.7: Modello ARX

²il numero di gradi di libertà di un modello indica la sua variabilità e coincide con il numero di polinomi da cui esso è caratterizzato.

³un segnale autoregressivo è un segnale i cui campioni sono calcolati come combinazione lineare dei campioni precedenti dello stesso segnale, in altre parole l'autoregressione su un dato campione è in sostanza una media pesata dei campioni precedenti.

dove $G(z)$ e $W(z)$ devono essere asintoticamente stabili, quindi gli zeri della funzione $A(z)$ devono trovarsi entro il cerchio di raggio unitario.

2.4 Modelli ARMAX

I **modelli ARMAX** (*AutoRegressive Moving Average eXogenous Models*) sono modelli a tre gradi di libertà, costituiti da un'uscita che è la somma di:

- un contributo **autoregressivo** (AR) sulla uscita;
- un contributo della variabile errore in **media mobile** (MA);
- un contributo **esogeno** (X) in media mobile.

$$\begin{aligned}
 y(t) = & \underbrace{a_1 \cdot y(t-1) + \dots + a_{n_a} \cdot y(t-n_a)}_{AR} + \\
 & e(t) + \underbrace{c_1 \cdot e(t-1) + \dots + c_{n_c} \cdot e(t-n_c)}_{MA} + \\
 & \underbrace{b_1 \cdot u(t-1) + \dots + b_{n_b} \cdot u(t-n_b)}_X
 \end{aligned} \tag{2.10}$$

Per la proprietà di traslazione nel tempo della $\mathcal{Z}$ -trasformata si ha:

$$\begin{aligned}
 y(t) \circ \underbrace{[1 - a_1 z^{-1} - a_2 z^{-2} - \dots - a_{n_a} z^{-n_a}]}_{A(z)} = & u(t-1) \circ \underbrace{[b_1 + b_2 z^{-1} + \dots + b_{n_b} z^{-n_b+1}]}_{B(z)} + \\
 & e(t) \circ \underbrace{[1 + c_1 z^{-1} + \dots + c_{n_c} z^{-n_c}]}_{C(z)}
 \end{aligned} \tag{2.11}$$

con:

$$\begin{aligned}
 A(z) &= 1 - a_1 z^{-1} - a_2 z^{-2} - \dots - a_{n_a} z^{-n_a} \\
 B(z) &= b_1 + b_2 z^{-1} + b_3 z^{-2} + \dots + b_{n_b} z^{-n_b+1} \\
 C(z) &= 1 + c_1 z^{-1} + c_2 z^{-2} + \dots + c_{n_c} z^{-n_c}
 \end{aligned}$$

Si può riscrivere la 2.11 come segue:

$$\begin{aligned}
 A(z) \circ y(t) &= B(z) \circ u(t-1) + C(z) \circ e(t) \\
 &\Downarrow \\
 y(t) &= \frac{B(z)}{A(z)} \circ u(t-1) + \frac{C(z)}{A(z)} \circ e(t)
 \end{aligned} \tag{2.12}$$

dove $G(z)$ e $W(z)$ devono essere asintoticamente stabili, quindi gli zeri della funzione $A(z)$ devono trovarsi entro il cerchio di raggio unitario. A differenza di quanto accade per la famiglia ARX, in questo caso il numeratore della $W(z)$ è arbitrario.

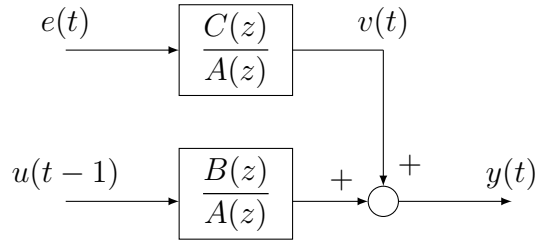


Figura 2.8: Modello ARMAX

2.5 Modelli ARARX

I **modelli ARARX** o **ARXAR** (*AutoRegressive AutoRegressive eXogenous Models*) presentano tre gradi di libertà e sono costituiti da un'uscita che è la somma di:

- un contributo **autoregressivo** (AR) sulla uscita;
- un contributo **autoregressivo** (AR) sulla variabile errore;
- il contributo puro della variabile errore;
- un contributo **esogeno** (X) in media mobile.

$$y(t) = \underbrace{a_1 \cdot y(t-1) + \dots + a_{n_a} \cdot y(t-n_a)}_{AR} + \underbrace{b_1 \cdot u(t-1) + \dots + b_{n_b} \cdot u(t-n_b)}_X + w(t) \quad (2.13)$$

dove:

$$w(t) = \underbrace{d_1 w(t-1) + d_2 w(t-2) + \dots + w(t-n_d)}_{AR} + e(t) \quad (2.14)$$

Per la proprietà di traslazione nel tempo della $\mathcal{Z}$ -trasformata si ha:

$$y(t) \circ \underbrace{[1 - a_1 z^{-1} - a_2 z^{-2} - \dots - a_{n_a} z^{-n_a}]}_{A(z)} = u(t-1) \circ \underbrace{[b_1 + b_2 z^{-1} + \dots + b_{n_b} z^{-n_b+1}]}_{B(z)} + e(t) \circ \underbrace{\frac{1}{1 - d_1 z^{-1} - \dots - d_{n_d} z^{-n_d}}}_{D(z)^{-1}} \quad (2.15)$$

con:

$$D(z) \circ w(t) = e(t) \quad \Leftrightarrow \quad w(t) = \frac{e(t)}{D(z)}$$

per cui:

$$\begin{aligned} A(z) &= 1 - a_1 z^{-1} - a_2 z^{-2} - \dots - a_{n_a} z^{-n_a} \\ B(z) &= b_1 + b_2 z^{-1} + b_3 z^{-2} + \dots + b_{n_b} z^{-n_b+1} \\ D(z) &= 1 - d_1 z^{-1} - d_2 z^{-2} - \dots - d_{n_d} z^{-n_d} \end{aligned}$$

Si può riscrivere la 2.15 come segue:

$$\begin{aligned} A(z) \circ y(t) &= B(z) \circ u(t-1) + w(t) = B(z) \circ u(t-1) + \frac{1}{D(z)} \circ e(t) \\ &\Downarrow \\ y(t) &= \frac{B(z)}{A(z)} \circ u(t-1) + \frac{1}{A(z) \cdot D(z)} \circ e(t) \end{aligned} \quad (2.16)$$

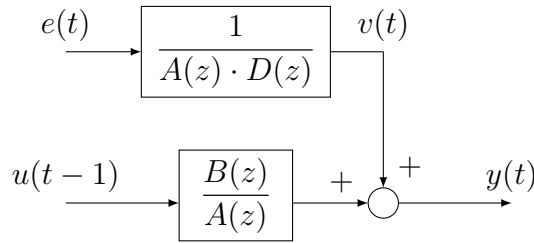


Figura 2.9: Modello ARARX

dove $G(z)$ e $W(z)$ devono essere asintoticamente stabili. Ciò implica che sia gli zeri della funzione $A(z)$ che quelli di $D(z)$ debbano trovarsi entro il cerchio di raggio unitario. Diversamente dalle famiglie ARX e ARMAX, nei modelli ARARX le funzioni $G(z)$ e $W(z)$ condividono un sottoinsieme di poli che coincide con l'insieme di tutte le radici dal polinomio $A(z)$; il blocco $W(z)$ presenta un numero maggiore di poli rispetto a $G(z)$ per via della presenza del polinomio $D(z)$.

2.6 Modelli AR, ARMA e ARAR

Le sottofamiglie AR (*AutoRegressive Models*), ARMA (*AutoRegressive Moving Average Models*) e ARAR (*AutoRegressive AutoRegressive Models*) sono **famiglie di serie temporali** in quanto in esse non è contemplata la presenza di ingressi. Esse sono ottenute rispettivamente dalle famiglie ARX, ARMAX e ARARX annullando il contributo esogeno.

I **modelli AR** sono rappresentati da equazioni del tipo:

$$\begin{aligned} A(z) \circ y(t) &= e(t) \\ &\Downarrow \\ y(t) &= \frac{1}{A(z)} \circ e(t) \end{aligned} \quad (2.17)$$

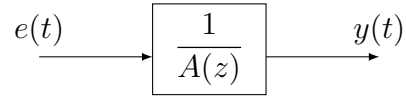


Figura 2.10: Modello AR

I **modelli ARMA** sono rappresentati da equazioni del tipo:

$$\begin{aligned}
 A(z) \circ y(t) &= C(z) \circ e(t) \\
 \Downarrow \\
 y(t) &= \frac{C(z)}{A(z)} \circ e(t)
 \end{aligned} \tag{2.18}$$

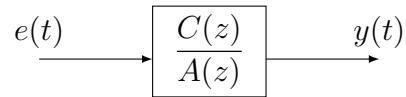


Figura 2.11: Modello ARMA

I **modelli ARAR** sono rappresentati da equazioni del tipo:

$$\begin{aligned}
 A(z) \cdot D(z) \circ y(t) &= e(t) \\
 \Downarrow \\
 y(t) &= \frac{1}{A(z) \cdot D(z)} \circ e(t)
 \end{aligned} \tag{2.19}$$

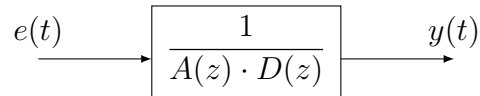


Figura 2.12: Modello ARAR

Capitolo 3

Predittori

La parte significativa di un processo di identificazione inizia sostanzialmente quando si raccolgono dei dati sulla base di osservazioni sperimentali. Una volta raccolti i dati si deve scegliere una famiglia di modelli ed estrarre un modello attraverso un criterio di scelta (come illustrato nell'algoritmo in figura 1.7).

Si dispone di un set di **osservazioni sperimentali** con N campioni:

$$\begin{aligned} y_{t_0}^{t_N} &= \{y(t_0), y(t_1), \dots, y(t_N)\} \\ u_{t_0}^{t_N} &= \{u(t_0), u(t_1), \dots, u(t_N)\} \end{aligned}$$

La **famiglia di modelli** è formalmente rappresentata da un insieme

$$\mathbb{M} = \{\mathcal{M}(\theta) \mid \theta \in \Theta\} \quad (3.1)$$

dove $\mathcal{M}$ indica il **modello**, funzione del suo **vettore di parametri** θ , individuati in uno **spazio vettoriale dei parametri** denotato con Θ .

Con la notazione $ARX(3, 3)$ si sta indicando un modello ARX con 3 coefficienti da identificare per il polinomio $A(z)$ e 3 per il polinomio $B(z)$. Si può evincere che i polinomi $A(z)$ e $B(z)$ sono di secondo grado. Lo spazio dei parametri è $\Theta = \mathbb{R}^6$.

Un modello $ARMAX(3, 3, 2)$ prevede 3 coefficienti per $A(z)$ (polinomio di secondo grado), 3 coefficienti per $B(z)$ (polinomio di secondo grado) e 2 coefficienti per $C(z)$ (polinomio di primo grado). Lo spazio dei parametri è $\Theta = \mathbb{R}^8$.

3.1 Estrazione del predittore

Un possibile approccio per soddisfare il criterio di scelta è quello dell'estrazione di un modello deterministico di un predittore da una famiglia di modelli probabilistici.

Un **predittore a un passo** è indicato con le seguenti forme equivalenti:

$$\hat{y}(t) = \hat{y}(t|t-1) = \hat{y}(t|t-1, \theta) = \hat{y}(y_{t_0}^{t-1}, u_{t_0}^{t-1}, \theta) \quad (3.2)$$

e presenta un **errore di predizione**:

$$\epsilon(t) = y(t) - \hat{y}(t) \quad (3.3)$$

l'approccio chiave all'identificazione è basato sulla minimizzazione dell'errore di predizione. Questo problema è affrontato facendo uso di un **indice di qualità del modello**:

$$\mathcal{J}(\theta) = \frac{1}{N} \sum_{t=1}^N [y(t) - \hat{y}(t)]^2 \quad (3.4)$$

Si evince facilmente dall'espressione della 3.4 che l'indice $\mathcal{J}$ è un indice integrale nel dominio discreto dei campioni, più specificamente $\mathcal{J}$ è una media aritmetica sui quadrati degli errori di predizione.

L'**estrazione del predittore** da una famiglia di modelli si fa partendo dal modello di Box-Jenkins:

$$y(t) = G(z) \circ u(t-1) + W(z) \circ e(t) \quad (3.5)$$

moltiplicando ambo i membri della 3.5 per $W(z)^{-1}$ si ottiene:

$$W(z)^{-1} \circ y(t) = y(t) + [W(z)^{-1} \cdot G(z)] \circ u(t-1) + e(t) \quad (3.6)$$

sommando membro a membro le equazioni 3.5 e 3.6 si ottiene:

$$y(t) + W(z)^{-1} \circ y(t) = y(t) + [W(z)^{-1} \cdot G(z)] \circ u(t-1) + e(t) \quad (3.7)$$

ossia:

$$y(t) = [1 - W(z)^{-1}] \circ y(t) + [W(z)^{-1} \cdot G(z)] \circ u(t-1) + e(t) \quad (3.8)$$

L'equazione 3.8 rappresenta un predittore che sembrerebbe stimare i campioni dell'uscita istantaneamente, senza ritardo finito. Ciò è ovviamente impossibile perché violerebbe il principio di causalità, infatti la suddetta problematica è solo apparente e ciò viene dimostrato di seguito.

$W(z)$ è per ipotesi un rapporto di polinomi monici e di ugual grado, pertanto può essere scritto nella forma:

$$W(z) = \frac{z^l + \alpha_1 z^{l-1} + \alpha_2 z^{l-2} + \dots}{z^l + \beta_1 z^{l-1} + \beta_2 z^{l-2} + \dots} \quad (3.9)$$

e $W(z)^{-1}$ differisce da $W(z)$ solo per il nome dati ai coefficienti. Dividendo il numeratore e il denominatore con l'algoritmo della lunga divisione si ottiene un polinomio nella forma:

$$W(z)^{-1} = 1 + \gamma_1 z^{-1} + \gamma_2 z^{-2} + \dots \quad (3.10)$$

se si sostituisce l'equazione 3.10 nella 3.8 si ha:

$$y(t) = \left[1 - \underbrace{(1 + \gamma_1 z^{-1} + \gamma_2 z^{-2} + \dots)}_{W(z)^{-1}} \right] \circ y(t) + [W(z)^{-1} \cdot G(z)] \circ u(t-1) + e(t) \quad (3.11)$$

ovvero:

$$y(t) = (-\gamma_1 z^{-1} + \gamma_2 z^{-2} + \dots) \circ y(t) + [W(z)^{-1} \cdot G(z)] \circ u(t-1) + e(t) \quad (3.12)$$

la scomparsa del termine noto nel polinomio a fattore di $y(t)$ indica che tutti i campioni dell'uscita rappresentati al secondo membro della 3.8 sono, di fatto, ritardati di un passo e che l'equazione stessa rappresenta effettivamente un predittore.

Infine, per finalizzare l'estrazione del predittore occorre “condensare” la funzione $e(t)$ in un unico valore: quello maggiormente rappresentativo è il valore atteso (o media), che nel caso di rumore bianco è nullo. L'estrazione del predittore è completata. L'equazione è la seguente:

$$\hat{y}(t) = [1 - W(z)^{-1}] \circ y(t) + [W(z)^{-1} \cdot G(z)] \circ u(t-1) \quad (3.13)$$

L'equazione 3.13 consente di estrarre un generico predittore, imponendo alle funzioni $G(z)$ e $W(z)$ le strutture corrispondenti alla famiglia di modelli che si vuole impiegare.

3.2 Predittore ARX

Un **predittore ARX** possiede due gradi di libertà in quanto è completamente caratterizzabile con due polinomi $A(z)$ e $B(z)$:

Le funzioni di trasferimento per la parte esogena e per quella relativa al rumore sono:

$$G(z) = \frac{B(z)}{A(z)} \quad , \quad W(z) = \frac{1}{A(z)} \quad (3.14)$$

dove:

$$\begin{aligned} A(z) &= 1 - a_1 z^{-1} - a_2 z^{-2} - \dots - a_{n_a} z^{-n_a} \\ B(z) &= b_1 + b_2 z^{-1} + b_3 z^{-2} + \dots + b_{n_b} z^{-n_b+1} \end{aligned}$$

inserendo queste informazioni nella 3.13 si ottiene:

$$\hat{y}(t) = [1 - A(z)] \circ y(t) + \left[A(z) \cdot \frac{B(z)}{A(z)} \right] \circ u(t-1)$$

ovvero:

$$\hat{y}(t) = [1 - A(z)] \circ y(t) + B(z) \circ u(t-1) \quad (3.15)$$

la cui equazione alle differenze è:

$$\hat{y}(t) = a_1 y(t-1) + a_2 y(t-2) + \dots + a_{n_a} y(t-n_a) + b_1 u(t-1) + b_2 u(t-2) + \dots + b_{n_b} u(t-n_b) \quad (3.16)$$

Il numero di parametri del predittore è $n = n_a + n_b$.

Un predittore ARX ha la peculiarità di essere sempre asintoticamente stabile ($\hat{y}$ non è mai infinito), in quanto la predizione non dipende da precedenti predizioni; questo risultato implica che l'algoritmo di ricerca può cercare in tutto lo spazio dei parametri, senza vincoli.

3.3 Predittore ARMAX

Un **predittore ARMAX** possiede tre gradi di libertà, essendo caratterizzato dai polinomi $A(z)$, $B(z)$ e $C(z)$.

Le funzioni di trasferimento del predittore sono:

$$G(z) = \frac{B(z)}{A(z)} \quad , \quad W(z) = \frac{C(z)}{A(z)} \quad (3.17)$$

dove:

$$\begin{aligned} A(z) &= 1 - a_1 z^{-1} - a_2 z^{-2} - \dots - a_{n_a} z^{-n_a} \\ B(z) &= b_1 + b_2 z^{-1} + b_3 z^{-2} + \dots + b_{n_b} z^{-n_b+1} \\ C(z) &= 1 + c_1 z^{-1} + c_2 z^{-2} + \dots + c_{n_c} z^{-n_c} \end{aligned}$$

inserendo queste informazioni nella 3.13 si ottiene:

$$\hat{y}(t) = \left[1 - \frac{A(z)}{C(z)} \right] \circ y(t) + \left[\frac{A(z)}{C(z)} \cdot \frac{B(z)}{A(z)} \right] \circ u(t-1)$$

ovvero:

$$\hat{y}(t) = \left[\frac{C(z) - A(z)}{C(z)} \right] \circ y(t) + \frac{B(z)}{C(z)} \circ u(t-1) \quad (3.18)$$

L'equazione alle differenze di un predittore ARMAX non è lineare rispetto ai suoi parametri (ovvero i coefficienti dell'equazione alle differenze potrebbero non essere tutti di grado unitario).

Se ad esempio:

$$A(z) = 1 - a_1 z^{-1} \quad , \quad B(z) = b_1 \quad , \quad C(z) = 1 + c_1 z^{-1}$$

si ha:

$$\frac{C(z) - A(z)}{C(z)} = \frac{(c_1 + a_1)z^{-1}}{1 + c_1 z^{-1}} \quad , \quad \frac{B(z)}{C(z)} = \frac{1}{1 + c_1 z^{-1}}$$

Il polinomio $\frac{(c_1 + a_1)z^{-1}}{1 + c_1 z^{-1}}$ può essere riscritto inglobando il termine z^{-1} (del numeratore) nel segnale $y(t)$ che diventa $y(t-1)$.

Eseguendo la divisione fra i polinomi si può dimostrare la non linearità dei coefficienti:

$$\begin{array}{r|l} c_1 + a_1 & 1 + c_1 z^{-1} \\ -c_1 - a_1 - c_1(c_1 + a_1)z^{-1} & c_1 + a_1 \\ \hline & - (c_1^2 + a_1 c_1)z^{-1} \end{array}$$

Nell'identificazione di un predittore ARMAX si deve imporre un vincolo di asintotica stabilità sul polinomio $C(z)$.

Si noti inoltre che il predittore ARX è un caso particolare di un predittore ARMAX: ponendo infatti $C(z) = 1$ in un predittore ARMAX, si ottiene un predittore ARX.

3.4 Predittore ARARX

Un **predittore ARARX** o **ARXAR** possiede anch'esso tre gradi di libertà ed è caratterizzato dai polinomi $A(z)$, $B(z)$ e $D(z)$.

Le funzioni di trasferimento del predittore sono:

$$G(z) = \frac{B(z)}{A(z)} \quad , \quad W(z) = \frac{1}{A(z) \cdot D(z)} \quad (3.19)$$

dove:

$$\begin{aligned} A(z) &= 1 - a_1 z^{-1} - a_2 z^{-2} - \dots - a_{n_a} z^{-n_a} \\ B(z) &= b_1 + b_2 z^{-1} + b_3 z^{-2} + \dots + b_{n_b} z^{-n_b+1} \\ D(z) &= 1 - d_1 z^{-1} - d_2 z^{-2} - \dots - d_{n_d} z^{-n_d} \end{aligned}$$

inserendo queste informazioni nella 3.13 si ottiene:

$$\hat{y}(t) = [1 - A(z) \cdot D(z)] \circ y(t) + \left[A(z) \cdot D(z) \cdot \frac{B(z)}{A(z)} \right] \circ u(t-1)$$

ovvero:

$$\hat{y}(t) = [1 - A(z) \cdot D(z)] \circ y(t) + [B(z) \cdot D(z)] \circ u(t-1)$$

La ricerca dei polinomi $A(z)$, $B(z)$ e $D(z)$ può essere effettuata con un algoritmo iterativo costituito dai seguenti passi:

1. si fissa il polinomio $D(z)$ a caso o secondo un criterio conveniente;
2. noto $D(z)$, si cercano i polinomi $A(z)$ e $B(z)$;
3. noti $A(z)$ e $B(z)$, si cerca un polinomio $D(z)$ migliore;
4. si reitera il procedimento ripartendo dal passo 2.

Ogni predittore ARARX è sempre asintoticamente stabile.

Capitolo 4

Validazione dei modelli

4.1 Analisi asintotica

Come già detto, nel contesto dell'identificazione, la ricerca di un predittore è fondata su un principio cardine: la **minimizzazione dell'errore di predizione**.

Lo scenario tipico prevede:

1. un numero N di **osservazioni sperimentali** a disposizione

$$\begin{aligned}y_{t_0}^{t_N} &= \{y(t_0), y(t_1), \dots, y(t_N)\} \\ u_{t_0}^{t_N} &= \{u(t_0), u(t_1), \dots, u(t_N)\}\end{aligned}$$

2. una **famiglia di modelli**

$$\mathbb{M} = \{\mathcal{M}(\theta) \mid \theta \in \mathbb{R}^n\}$$

scelta per l'identificazione, con n che indica il numero complessivo di parametri del modello.

Ci si chiede qual è il legame che sussiste tra N ed n , ovvero quanti debbano essere i campioni sperimentali che servono per identificare n parametri.

Certamente si può intuire che il numero di campioni N deve sempre superare il numero di parametri n del modello, considerando che nel contesto dell'identificazione vi è sempre la modellazione dei fenomeni stocastici (oltre a quelli deterministici) a gravare sul numero totale dei parametri.

Le osservazioni sperimentali dell'uscita di un sistema affetto da incertezza sono di fatto osservazioni di un processo stocastico, caratterizzato da una completa non ripetibilità dell'esperimento di acquisizione dei dati (i dati osservati in un esperimento non sono mai gli stessi osservati in un successivo esperimento, pur essendo questo stato condotto come il precedente). La non ripetibilità nell'acquisizione dei dati sperimentali si ripercuote sull'identificazione del predittore, il quale di conseguenza non sarà caratterizzato sempre dagli stessi parametri.

Sommando tutti i quadrati degli errori di predizione si ottiene un **indice quadratico** che misura lo scostamento quadratico della predizione dall'osservazione sperimentale:

$$\mathcal{J}_N(\theta) = \frac{1}{N} [y(t_i) - \hat{y}(t_i)]^2 = \frac{1}{N} \sum_{i=1}^N [y(t) - \hat{y}(t)]^2 \quad (4.1)$$

Questo scostamento quadratico rappresenta l'indice da minimizzare ed è tanto più accurato quanto più grande è il numero di dati sperimentali.

Il **miglior modello** è quello che meglio minimizza l'indice.

In una situazione ideale, ovvero nella situazione in cui tutti i fenomeni stocastici sono completamente caratterizzati, si può definire un indice:

$$\bar{\mathcal{J}}(\theta) = \mathbb{E} [\epsilon^2(t)] \quad (4.2)$$

che rappresenta appunto la descrizione completa teorica di un dato fenomeno stocastico.

L'indice $\mathcal{J}_N(\theta)$ è ottenuto con N dati ed è pertanto una **stima campionaria**: al crescere di N migliora la caratterizzazione del fenomeno stocastico, pertanto se:

$$N \rightarrow \infty \quad \text{allora} \quad \mathcal{J}_N(\theta) \rightarrow \bar{\mathcal{J}}(\theta) \quad (4.3)$$

L'**insieme dei punti di minimo** di $\bar{\mathcal{J}}(\theta)$, nello spazio dei parametri, è:

$$\Delta = \{\bar{\theta} \in \Theta \mid \bar{\mathcal{J}}(\bar{\theta}) \leq \bar{\mathcal{J}}(\theta)\} \quad (4.4)$$

con tale insieme si indicano tutti i vettori dei parametri (ciascuno dei quali identifica un modello) i quali minimizzano l'indice $\bar{\mathcal{J}}$.

Il generico **punto di minimo** di $\mathcal{J}_N$ è indicato con la seguente notazione:

$$\hat{\theta}_N = \underset{\theta \in \Theta}{\operatorname{argmin}} \mathcal{J}_N(\theta) \quad (4.5)$$

il punto $\hat{\theta}_N$ è il vettore dei parametri che minimizza l'indice $\mathcal{J}_N$ ossia il minimo¹ della stima campionaria di $\bar{\mathcal{J}}$ che si vuole comparare a $\bar{\mathcal{J}}$.

A questo punto, se vale l'implicazione:

$$N \rightarrow \infty \quad \not\Rightarrow \quad \hat{\theta}_N \rightarrow \Delta \quad (4.6)$$

si può concludere che un eventuale aumento della quantità N di dati sperimentali non produrrebbe alcun del modello.

In caso contrario, sia $\mathcal{S}$ il **meccanismo di generazione dei dati (MGD)** ovvero il modello da identificare². Se è vera l'implicazione:

$$N \rightarrow \infty \quad \Rightarrow \quad \hat{\theta}_N \rightarrow \Delta \quad (4.7)$$

cioè se a un aumento della quantità dati si tende a un modello migliore, allora si distinguono quattro possibilità a seconda della cardinalità:

¹possono anche esistere infiniti punti di minimo che portano ad ottenere lo stesso indice $\mathcal{J}_N$.

²può capitare che il meccanismo di generazione dei dati non sia incluso nella famiglia scelta ($\mathcal{S} \notin \mathbb{M}$).

1. se $|\Delta| = 1$ (vi è un solo punto di minimo) e $\mathcal{S} \in \mathbb{M}$ (il modello identificato appartiene nella famiglia di modelli scelta) allora $\hat{\theta}_N$ converge verso $\mathcal{S}$, nell'insieme $\mathbb{M}$ (figura 4.1a);
2. se $|\Delta| = 1$ e $\mathcal{S} \notin \mathbb{M}$ allora $\hat{\theta}_N$ converge verso un modello $\hat{\mathcal{S}}$ che è la migliore stima del MGD all'interno della famiglia di modelli scelta e giacerebbe ad esempio lungo la frontiera di $\mathbb{M}$, avvicinandosi a $\mathcal{S}$ (figura 4.1c).
3. se $|\Delta| > 1$ e $\mathcal{S} \in \mathbb{M}$ può accadere che $\hat{\theta}_N$ converga verso un elemento di Δ ma può anche accadere che esso oscilli fra i vari punti di Δ senza convergere (figura 4.1b).
4. se $|\Delta| > 1$ e $\mathcal{S} \notin \mathbb{M}$ può accadere che $\hat{\theta}_N$ converga verso un elemento di Δ (contenente la miglior stima del MGD all'interno della famiglia di modelli scelta) oppure che oscilli fra i punti di Δ senza convergere (figura 4.1d).

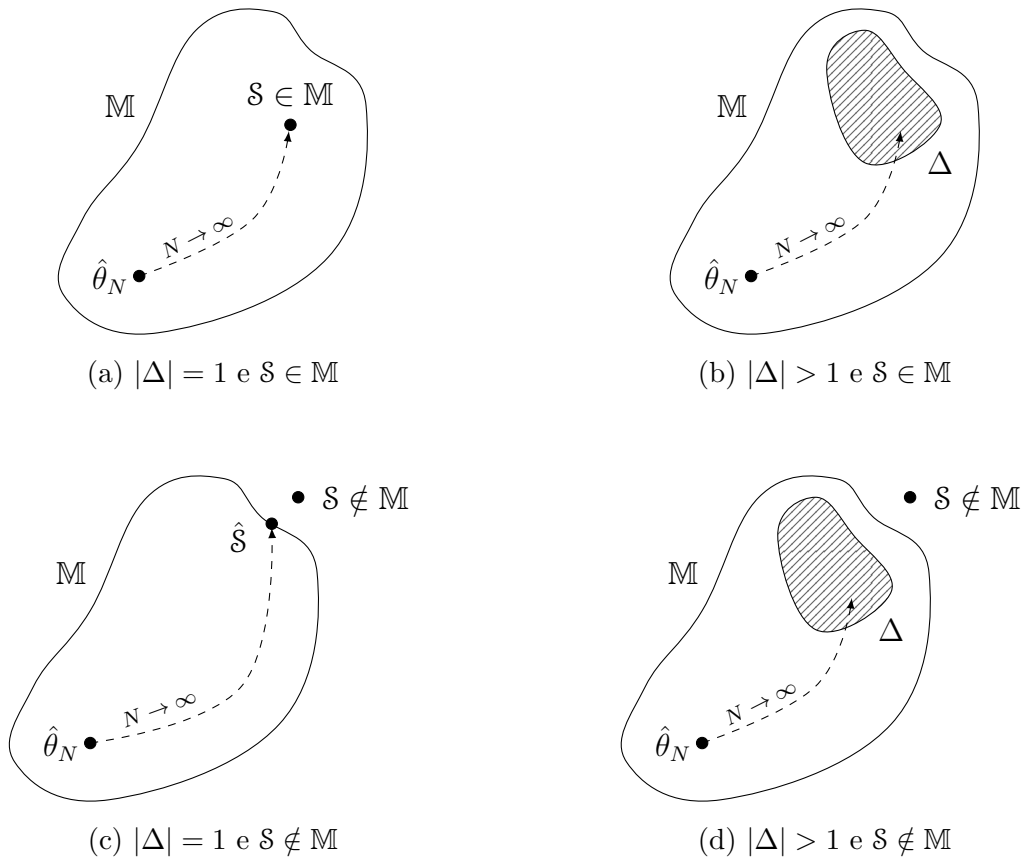


Figura 4.1: Analisi asintotica del modello da identificare

Nella pratica sperimentale, ci si ritrova per lo più nei casi 4.1c e 4.1d perché in tali circostanze il vero MGD non è assimilabile a una famiglia di modelli.

Nel caso illustrato in figura 4.1a si può dimostrare³ che al tendere a infinito del di dati N il modello identificato converge verso il **modello vero** ossia verso il modello associato al

³recuperare dimostrazione mancante (lezione 11/10/2017)

caso ideale in cui i fenomeni stocastici sono completamente caratterizzati. Il **vettore dei parametri veri** del sistema, è indicato con θ^0 .

4.2 Analisi di bianchezza

Giudicare la qualità di un modello dal *fit* tra l'andamento dei dati e la sua risposta all'impulso corrisponde a valutare l'ampiezza dell'indice quadratico $\bar{J}$: tanto più piccolo è questo indice, quanto più accurato è il modello.

L'**insieme dei dati di training**, è impiegato per ottenere il modello, e deve essere diverso dall'**insieme dei dati di validazione**, su cui va effettivamente valutato $\bar{J}$.

L'indice $\bar{J}$ è tuttavia insufficiente per valutare la qualità di un modello: se infatti l'errore di predizione non ha una natura puramente casuale, ciò significa che il modello non è riuscito a spiegare completamente la dinamica deducibile dai dati sperimentali.

Una buona identificazione non solo produce un valore di $\bar{J}$ basso ma deve produrre un errore di predizione quanto più casuale.

In una situazione di identificazione ideale, tutto ciò che c'è da dedurre dai dati sperimentali è stato spiegato dal modello identificato, perciò il **residuo della predizione** (l'errore di predizione) dev'essere il più possibile un segnale casuale, dal momento che esso non deve contenere alcun **offset**⁴ (cioè non devono essere presenti errori sistematici).

Il rumore bianco è il segnale casuale per eccellenza pertanto, per valutare quanto casuali siano gli errori di predizione residui, si effettua l'**analisi di bianchezza**, ovvero un test di assimilabilità degli errori di predizione residui a un rumore bianco. La comparazione è effettuata basandosi sulla **funzione di autocorrelazione**: il rumore bianco ha una funzione di autocorrelazione che è un impulso di Dirac (ciascun campione è correlato solamente a sé stesso).

Se si calcola una stima campionaria della funzione di autocorrelazione⁵ dei residui della predizione, questa dev'essere quanto più possibile simile a un impulso di Dirac (rumore bianco); non sarà mai perfettamente coincidente con un impulso di Dirac perché non si può disporre di infiniti campioni e quindi di una funzione di autocorrelazione completamente nota dal punto di vista teorico. Un'altra valutazione della bianchezza dell'errore di predizione potrebbe essere l'assimilabilità del suo spettro ad uno **spettro costante** (caratteristico del rumore bianco).

Un'altro tipo di analisi di bianchezza è quella chiamata **Model Error Identification**: nel caso ideale in cui il modello ha spiegato bene cosa c'è nei dati, non vi è alcuna dinamica fra l'ingresso e l'errore di predizione.

Se invece si identifica un sistema dinamico presente fra ingressi ed errori, ciò significa che esiste una dinamica non spiegata dal modello, una dinamica rimasta presente negli errori di predizione.

Questa analisi fornisce anche un'indicazione utile sull'errore commesso nella scelta della famiglia di modelli.

⁴l'operazione di rimozione degli offset prende il nome di *detrend*.

⁵la funzione di autocorrelazione teorica è calcolabile con infiniti campioni.

4.3 Quantificazione dell'errore di stima parametrica

Si è detto in precedenza che nel caso ideale in cui $|\Delta| = 1$ e $\mathcal{S} \in \mathbb{M}$, all'aumentare del numero di dati sperimentali N , il modello stimato associato al vettore $\hat{\theta}_N$ tende al modello vero rappresentato dal vettore θ^0 (caso in figura 4.1a). È evidente che essendo N , nella pratica, un numero finito, non può esservi una corrispondenza esatta tra il modello identificato e quello vero.

È utile dunque ottenere una **quantificazione dell'errore di stima parametrica**, ovvero misurare quanto $\hat{\theta}_N$ si avvicini a θ^0 , al variare di N .

Si consideri l'errore di predizione associato al modello, $\epsilon(t, \theta)$ e il suo vettore gradiente rispetto ai suoi n parametri, inclusi nel vettore θ :

$$\psi(t, \theta) = -\frac{\partial}{\partial \theta} [\epsilon(t, \theta)]^T \quad (4.8)$$

questo gradiente⁶ ha dimensione $(n \times 1)$ e misura la variazione dell'errore di predizione al variare dei parametri del modello.

Si definisce la **matrice dei gradienti**:

$$\overline{R} = \mathbb{E} [\psi(t, \theta^0) \cdot \psi^T(t, \theta^0)] \quad (4.9)$$

quadrata, di dimensione $(n \times n)$.

Si può dimostrare che l'errore di stima normalizzato rispetto a N tende asintoticamente a una distribuzione normale o gaussiana:

$$\sqrt{N} \cdot (\hat{\theta}_N - \theta^0) \sim \mathcal{N} [0, \overline{P}] \quad (4.10)$$

dove con $\mathcal{N} [0, \overline{P}]$ si intende una distribuzione gaussiana a media nulla e varianza $\overline{P}$.

La 4.10 è anche esprimibile nella forma equivalente:

$$(\hat{\theta}_N - \theta^0) \sim \mathcal{N} \left[0, \frac{\overline{P}}{N} \right] \quad (4.11)$$

Detto in parole, l'errore di stima parametrica tende ad assumere una distribuzione gaussiana con valor medio nullo e dipendenza inversamente proporzionale al numero dei dati sperimentali N e direttamente proporzionale alla varianza dell'incertezza λ .

La matrice:

$$\overline{P} = \lambda^2 \cdot \overline{R}^{-1} \quad (4.12)$$

è anch'essa quadrata, di dimensione $(n \times n)$ e dipende dalla inversa della matrice dei gradienti, inoltre λ è la varianza del predittore ideale, pari a:

$$\lambda = \epsilon(t, \theta^0) = e(t) \quad (4.13)$$

Si evince che la scelta di aumentare il numero N dei dati sperimentali da parte del progettista può essere utile a migliorare la qualità del modello solamente nel caso ideale in cui $\mathcal{S} \in \mathbb{M}$.

⁶l'operazione di trasposizione del vettore è legata al fatto che il gradiente è un vettore riga

4.4 Ordine ottimo di un modello

Quando si fa identificazione si può incorrere in un fenomeno chiamato **overtraining**. La sua insorgenza si ha quando si sovrapparametrizza un modello o, equivalentemente, se ne aumenta indefinitamente l'ordine.

Contrariamente a quanto ci si aspetta, un aumento del numero di parametri del modello può comportare il peggioramento nella qualità dell'identificazione! Ciò accade perché il modello identificato inizia a descrivere non più solamente la dinamica del MGD ma anche quella del rumore e dell'incertezza da cui esso è affetto. Può anche accadere che all'aumentare dell'ordine del modello oltre una certa soglia non si riscontrano miglioramenti ma neppure peggioramenti.

Risulta dunque opportuno studiare la complessità ottimale del modello al fine di determinarne l'**ordine ottimo**, evitando così di incorrere in sovrapparametrizzazioni o in un modello descritto da una quantità ridondante di parametri.

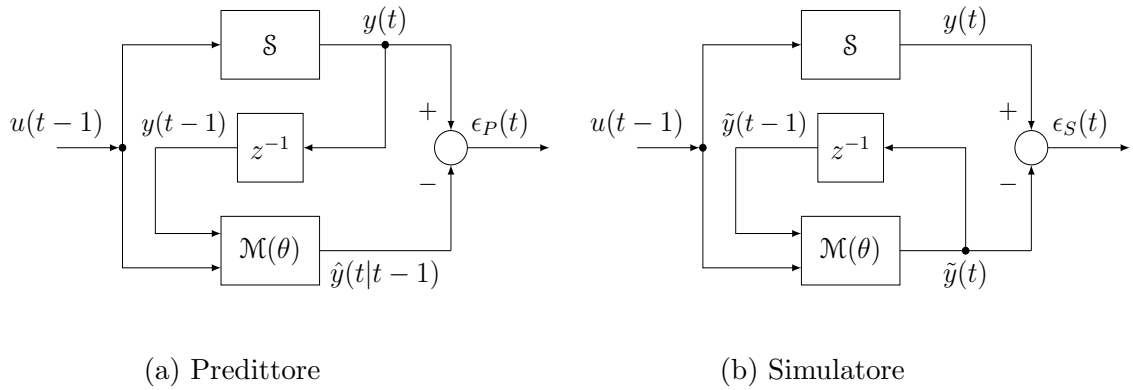


Figura 4.2: Schemi a blocchi dei sistemi per la stima dell'errore

Si può considerare quindi un predittore (figura 4.2a) o, in alternativa, un simulatore (figura 4.2b): l'uso di un simulatore dà risultati in genere migliori ma costruirne uno è difficile, dal momento che le uscite del sistema simulato (non essendo direttamente misurate) risentono di errori che tendono a divergere e a causare instabilità (risultati sempre più errati).

L'errore di predizione è dato da:

$$\epsilon_P(t) = y(t) - \hat{y}(t|t-1) \quad (4.14)$$

L'errore di simulazione è dato da:

$$\epsilon_S(t) = y(t) - \tilde{y}(t) \quad (4.15)$$

La ricerca dell'ordine ottimo è attuata studiando come varia l'indice $\mathcal{J}$ al variare dell'ordine del modello da identificare, ossia del suo numero di parametri.

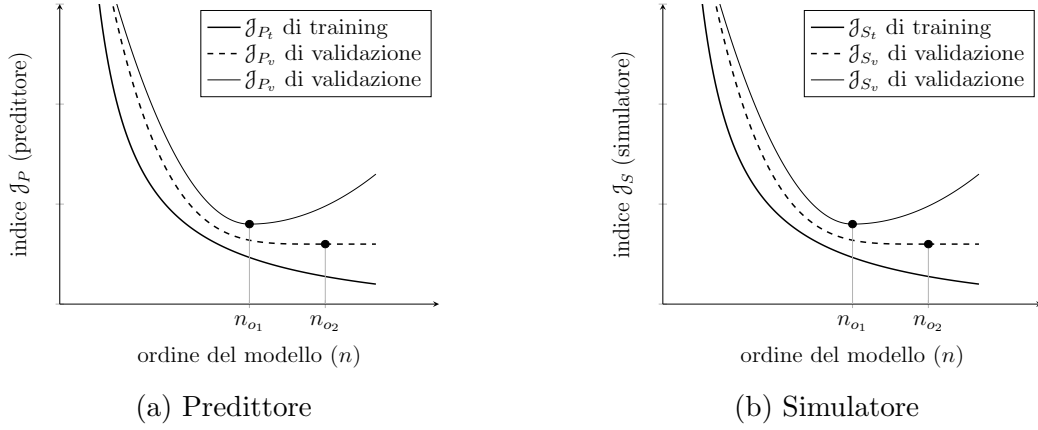


Figura 4.3: Andamento degli indici $\mathcal{J}$ al variare dell'ordine del modello

Dopo il training del modello, si procede alla validazione: al crescere dell'ordine n del modello l'indice $\mathcal{J}$ diminuisce. Si incorre in overtraining quando $\mathcal{J}$ raggiunge un punto di minimo e poi inizia a crescere (ovvero il modello inizia a peggiorare). L'ordine ottimo è dunque l'ordine per cui l'indice $\mathcal{J}$ presenta un minimo. Può anche capitare che l'andamento dell'indice $\mathcal{J}$ diventi costante oltre un certo ordine: in tal caso si hanno infiniti minimi, quindi conviene sempre considerare il numero più ridotto possibile di parametri da introdurre che portino al modello ottimo.

Non essendo tuttavia sempre possibile disporre di un numero sufficiente di dati per il training, l'ordine ottimo deve essere stimato con approcci differenti: si ricorre all'utilizzo di alcuni indici di ottimalità.

Il loro scopo è considerare che se da un lato l'aumento di n riduce l'indice di aderenza $\mathcal{J}$ ai dati di training, dall'altro aumenta il rischio di overtraining.

Gli indici di ottimalità riportati di seguito dipendono tutti da n ma variano in maniera non monotona decrescente (presentano punti stazionari). Il valore di n per cui si ha un minimo per ciascuno di questi indici è ritenuto l'ordine ottimale. Tutti gli indici tendono a sovrastimare l'ordine corretto del modello.

1. FPE: Final Prediction Error

$$FPE = \frac{N + n}{N - n} \cdot \mathcal{J}(\hat{\theta}_N) \quad (4.16)$$

2. AIC: Akaike Information Criterion

$$AIC = \frac{2n}{N} + \log \left(\mathcal{J}(\hat{\theta}_N) \right) \quad (4.17)$$

3. MDL: Minimum Description Length

$$MDL = \log(N) + \frac{n}{N} + \log \left(\mathcal{J}(\hat{\theta}_N) \right) \quad (4.18)$$

Questo indice si basa sulla descrizione del modello stimato in termini di bit.

Il principio sul quale si fonda la sua utilità è che la complessità corretta è quella per cui la descrizione completa dei dati mediante il modello richiede il numero minimo di bit rispetto a tutti gli altri modelli.

Capitolo 5

Ottimizzazione ai minimi quadrati

L'ottimizzazione della funzione obiettivo consiste nella ricerca dei minimi dell'indice quadratico $\mathcal{J}_N(\theta)$. Questa operazione è svolta mediante specifici **algoritmi di ottimizzazione**.

5.1 Algoritmo dei minimi quadrati

I modelli ARX sono molto diffusi perché sono lineari nei parametri e, in quanto tali, sono identificabili avvalendosi in tecniche di ottimizzazione non iterative, la cui soluzione è ottenibile in un solo passo.

Un metodo molto comune di minimizzazione della funzione obiettivo, usato in tutti i casi in cui i parametri del modello abbiano una dipendenza lineare dall'uscita (come appunto i modelli ARX), è l'**algoritmo dei minimi quadrati**.

Si suppone di avere un'uscita $y(t)$ sconosciuta, con N osservazioni sperimentali note di y . Si definisce il **vettore delle uscite**:

$$Y = \begin{bmatrix} y(t_1) \\ y(t_2) \\ \vdots \\ y(t_N) \end{bmatrix} \quad (5.1)$$

e si cerca una funzione che approssimi la funzione sconosciuta $y(t)$:

$$\hat{y}(t) = \theta_1 x_1(t) + \theta_2 x_2(t) + \cdots + \theta_n x_n(t) = \theta^T \cdot \varphi(t) \quad (5.2)$$

dove i vettori:

$$\varphi(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{bmatrix}, \quad \theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{bmatrix} \quad (5.3)$$

sono rispettivamente il **vettore dei regressori** e il **vettore dei parametri**. I regressori sono funzioni note.

Il problema ai minimi quadrati è definito a partire dall'errore, definito in ciascun istante di tempo t_i in cui è stata raccolta sperimentalmente la corrispondente uscita:

$$\epsilon(t_i) = y(t_i) - \hat{y}(t_i) \quad \text{con} \quad i = 1, \dots, N \quad (5.4)$$

si esegue una misura integrale di tutti gli errori calcolati:

$$\mathcal{J}(\theta) = \frac{1}{N} \sum_{i=1}^N \epsilon^2(t_i) \quad (5.5)$$

La soluzione del problema ai minimi quadrati è per definizione:

$$\theta_{MQ} \equiv \underset{\theta \in \Theta}{\operatorname{argmin}} \mathcal{J}(\theta) = \hat{\theta}_N \quad (5.6)$$

Si definisce la **matrice dei regressori**:

$$\phi = \begin{bmatrix} \varphi^T(t_1) \\ \varphi^T(t_2) \\ \vdots \\ \varphi^T(t_N) \end{bmatrix} \quad (5.7)$$

le cui righe sono i vettori dei regressori calcolati in tutti gli N istanti di tempo corrispondenti alle osservazioni sperimentali.

Esprimendo l'errore in forma matriciale, si può scrivere:

$$Y - \hat{Y} = Y - \phi\theta \quad (5.8)$$

di conseguenza:

$$\begin{aligned} \mathcal{J}(\theta) &= \frac{1}{N} \sum_{i=1}^N \epsilon^2(t_i) = \frac{1}{N} (Y - \phi\theta)^T \cdot (Y - \phi\theta) = \\ &= \frac{1}{N} [(Y^T - \theta^T \phi^T)(Y - \phi\theta)] = \\ &= \frac{1}{N} [Y^T Y + \theta^T \phi^T \phi \theta - 2\theta^T \phi^T Y] \end{aligned} \quad (5.9)$$

Per minimizzare l'indice $\mathcal{J}$ bisogna imporre l'annullamento del suo gradiente:

$$\frac{\partial}{\partial \theta} \mathcal{J}(\theta) = 2\phi^T \phi \theta - 2\phi^T Y = 0 \quad (5.10)$$

da cui:

$$\phi^T \phi \theta = \phi^T Y \quad (5.11)$$

e quindi si ottiene la **soluzione** al problema:

$$\theta_{MQ} = (\phi^T \phi)^{-1} \phi^T Y \quad (5.12)$$

Se la matrice $\phi^T \phi$ non è invertibile allora non esiste la soluzione al problema dei minimi quadrati. La soluzione del problema ai minimi quadrati è di tipo algebrico pertanto si ottiene in un passo.

5.2 Applicazione a modelli ARX

Di seguito si mostra un'applicazione dell'algoritmo dei minimi quadrati all'identificazione di un modello ARX con $n = n_a + n_b$ parametri, a partire da N osservazioni sperimentali.

La soluzione al problema dei minimi quadrati è:

$$\theta_{MQ} = (\phi^T \phi)^{-1} \phi^T Y = \left[\sum_{i=1}^N \varphi(t_i) \cdot \varphi^T(t_i) \right]^{-1} \cdot \left[\sum_{i=1}^N y(t_i) \cdot \varphi(t_i) \right] \quad (5.13)$$

Il **vettore delle uscite**, di dimensioni $(N \times 1)$ è:

$$Y = \begin{bmatrix} y(t_1) \\ y(t_2) \\ \vdots \\ y(t_N) \end{bmatrix} \quad (5.14)$$

Si assegnano ai regressori, le osservazioni sperimentali dell'uscita e dell'ingresso negli istanti precedenti. Il **vettore dei regressori**, di dimensioni $(n \times 1)$, e il **vettore dei parametri**, di dimensioni $(N \times 1)$, sono strutturati come segue:

$$\varphi(t_i) = \begin{bmatrix} y(t_i - 1) \\ \vdots \\ y(t_i - n_a) \\ u(t_i - 1) \\ \vdots \\ u(t_i - n_b) \end{bmatrix}, \quad \theta = \begin{bmatrix} a_1 \\ \vdots \\ a_{n_a} \\ b_1 \\ \vdots \\ b_{n_b} \end{bmatrix} \quad (5.15)$$

Allora si ha:

$$\hat{Y}(t_i) = \theta^T \cdot \varphi(t_i) \quad (5.16)$$

la cui forma estesa è:

$$\hat{y}(t_i) = a_1 y(t_i - 1) + a_2 y(t_i - 2) + \dots + a_{n_a} y(t_i - n_a) + b_1 u(t_i - 1) + b_2 u(t_i - 2) + \dots + b_{n_b} u(t_i - n_b) \quad (5.17)$$

La **matrice dei regressori** ϕ avrà dimensioni $(N \times n)$ e sarà strutturata nel modo seguente:

$$\phi = \begin{bmatrix} y(t_1) & 0 & 0 & 0 & \dots & u(t_1) & 0 & 0 & 0 & \dots \\ y(t_2) & y(t_1) & 0 & 0 & \dots & u(t_2) & u(t_1) & 0 & 0 & \dots \\ y(t_3) & y(t_2) & y(t_1) & 0 & \dots & u(t_3) & u(t_2) & u(t_1) & 0 & \dots \\ \vdots & \vdots & \vdots & \vdots & \dots & \vdots & \vdots & \vdots & \vdots & \dots \end{bmatrix} \quad (5.18)$$

Il vettore Y è noto, quindi una volta calcolata la matrice ϕ si può trovare la soluzione:

$$\theta_{MQ} = (\phi^T \phi)^{-1} \phi^T Y \quad (5.19)$$

5.3 Analisi probabilistica

La linearità nei parametri dei modelli ARX è una caratteristica favorevole per svolgere l'analisi asintotica e studiare il comportamento del modello al crescere del numero di campioni N .

Nell'ipotesi di conoscere il MGD, se questo è un ARX allora ha come vettore dei parametri il vettore dei parametri vero θ^0 :

$$\mathcal{S} = \mathcal{M}(\theta^0) \quad (5.20)$$

L'equazione del MGD è:

$$y(t) = \varphi^T(t) \cdot \theta^0 + e(t) \quad (5.21)$$

dove $e(t)$ è l'incertezza e $\varphi(t)$ il vettore dei regressori:

$$\varphi(t) = [y(t-1) \quad \dots \quad y(t-n_a) \quad u(t-1) \quad \dots \quad u(t-n_b)]^T \quad (5.22)$$

l'equazione del predittore è:

$$\hat{y}(t) = \varphi^T(t) \cdot \theta \quad (5.23)$$

e l'errore di predizione vale:

$$\epsilon(t) = \varphi^T(t) (\theta^0 - \theta) + e(t) \quad (5.24)$$

Il termine $(\theta^0 - \theta)$ è l'**errore di stima parametrica**.

L'indice da minimizzare è dato da:

$$\begin{aligned} \bar{J}(\theta) &= \mathbb{E} [\epsilon^2(t)] = \mathbb{E} \left[(\theta^0 - \theta)^T \varphi(t) \varphi^T(t) (\theta^0 - \theta) + e^2(t) + 2\varphi^T(t) (\theta^0 - \theta) e(t) \right] = \\ &= (\theta^0 - \theta)^T \mathbb{E} [\varphi(t) \varphi^T(t)] (\theta^0 - \theta) + \mathbb{E} [e^2(t)] + \mathbb{E} [2\varphi^T(t) (\theta^0 - \theta) e(t)] \end{aligned}$$

Poiché $e(t)$ (rumore bianco) è correlato solamente a sé stesso si può scrivere il valore atteso del prodotto come il prodotto dei valori attesi:

$$\bar{J}(\theta) = (\theta^0 - \theta)^T \mathbb{E} [\varphi(t) \varphi^T(t)] (\theta^0 - \theta) + \mathbb{E} [e^2(t)] + \mathbb{E} [2\varphi^T(t)] (\theta^0 - \theta) \mathbb{E} [e(t)] \quad (5.25)$$

essendo $\mathbb{E} [e^2(t)] = \lambda^2$ e $\mathbb{E} [e(t)] = 0$ allora:

$$\bar{J}(\theta) = (\theta^0 - \theta)^T \mathbb{E} [\varphi(t) \varphi^T(t)] (\theta^0 - \theta) + \lambda^2 \quad (5.26)$$

il cui primo addendo è una forma quadratica. La matrice:

$$\mathbb{E} [\varphi(t) \varphi^T(t)]$$

è fortemente legata alla **matrice dei minimi quadrati** (nella equazione 5.13):

$$\mathcal{S}(N) = \sum_{t=1}^N \varphi(t) \varphi^T(t) \quad (5.27)$$

la quale è per definizione, **semidefinita positiva**.

Una matrice semidefinita positiva è definita positiva se il suo determinante è non nullo¹. Nel problema ai minimi quadrati bisogna pertanto imporre:

$$|\mathcal{S}(N)| \neq 0 \quad (5.28)$$

che equivale a dire che la forma quadratica $\mathcal{S}$ sia definita positiva. Una forma quadratica definita positiva si annulla solo nell'origine; in tali condizioni vi è un solo punto di minimo ($|\Delta| = 1$) ed è possibile stimare il vettore vero dei parametri θ^0 .

Tutto ciò è valido sotto l'ipotesi irrealistica per cui il MGD corrisponda a un modello ARX, caratterizzato da una $G(z) = \frac{B(z)}{A(z)}$ e da una $W(z) = \frac{1}{A(z)}$.

Nella realtà non è mai possibile avere un'incertezza caratterizzata dallo stesso denominatore della funzione $G(z)$.

Questo risultato, seppur teorico, conserva un significato utile ai casi pratici: il fatto che la matrice $\mathcal{S}(N)$ sia o meno definita positiva influenza la cardinalità dell'insieme dei punti di minimo, che auspicabilmente dev'essere pari a 1. In tal caso il problema dei minimi quadrati ammetterebbe una soluzione, di conseguenza diventerebbe possibile stimare il vettore vero dei parametri.

Una buona identificazione è pertanto basata sulla possibilità di manipolare $\mathcal{S}(N)$ in modo tale da renderla “il più possibile” non singolare.

5.4 Persistente eccitazione

Per studiare la singolarità della matrice $\mathcal{S}(N)$ si prende come esempio un modello $ARX(1, 1)$.

L'equazione alle differenze di un modello $ARX(1, 1)$ è:

$$y(t) = a_1 y(t-1) + b_1 u(t-1)$$

il vettore dei regressori:

$$\varphi(t) = [y(t-1) \quad u(t-1)]^T$$

di conseguenza:

$$\varphi(t)\varphi(t)^T = \begin{bmatrix} y(t-1) \\ u(t-1) \end{bmatrix} [y(t-1) \quad u(t-1)] = \begin{bmatrix} y^2(t-1) & y(t-1)u(t-1) \\ u(t-1)y(t-1) & u^2(t-1) \end{bmatrix}$$

La matrice dei minimi quadrati è:

$$\mathcal{S}(N) = \sum_{t=1}^N \varphi(t)\varphi(t)^T = \begin{bmatrix} \sum_{t=1}^N y^2(t-1) & \sum_{t=1}^N y(t-1)u(t-1) \\ \sum_{t=1}^N u(t-1)y(t-1) & \sum_{t=1}^N u^2(t-1) \end{bmatrix} \quad (5.29)$$

¹risultato che discende dal test di Sylvester (richiami presenti in appendice B.1).

La matrice $S(N)$ tende a divergere al crescere di N , perciò si normalizza questa matrice rispetto a N , ottenendo la **matrice dei minimi quadrati normalizzata**:

$$R(N) = \frac{S(N)}{N} = \begin{bmatrix} \frac{1}{N} \sum_{t=1}^N y^2(t-1) & \frac{1}{N} \sum_{t=1}^N y(t-1)u(t-1) \\ \frac{1}{N} \sum_{t=1}^N u(t-1)y(t-1) & \frac{1}{N} \sum_{t=1}^N u^2(t-1) \end{bmatrix} \quad (5.30)$$

Poiché i segnali $u(\cdot)$ e $y(\cdot)$ sono stazionari, i termini della matrice $R(N)$ sono le stime campionarie delle rispettive funzioni di correlazione² per $\tau = 0$ (varianza e covarianza), indicate per ragioni di chiarezza di notazione con la lettera greca γ :

$$R(N) = \begin{bmatrix} \gamma_{yy}(0) & \gamma_{yu}(0) \\ \gamma_{uy}(0) & \gamma_{uu}(0) \end{bmatrix} \quad (5.31)$$

Nel caso di un $ARX(2,2)$ si avrebbe:

$$\begin{aligned} \varphi(t) &= [y(t-1) \quad y(t-2) \quad u(t-1) \quad u(t-2)]^T \\ \varphi(t)\varphi^T(t) &= \begin{bmatrix} y(t-1) \\ y(t-2) \\ u(t-1) \\ u(t-2) \end{bmatrix} [y(t-1) \quad y(t-2) \quad u(t-1) \quad u(t-2)] = \\ &= \begin{bmatrix} y^2(t-1) & y(t-1)y(t-2) & y(t-1)u(t-1) & y(t-1)u(t-2) \\ y(t-2)y(t-1) & y^2(t-2) & y(t-2)u(t-1) & y(t-2)u(t-2) \\ u(t-1)y(t-1) & u(t-2)y(t-1) & u^2(t-1) & u(t-1)u(t-2) \\ u(t-2)y(t-1) & u(t-2)y(t-2) & u(t-2)u(t-1) & u^2(t-2) \end{bmatrix} \end{aligned}$$

La matrice R è:

$$R(N) = \frac{1}{N} \begin{bmatrix} \sum_{t=1}^N y^2(t-1) & \sum y(t-1)y(t-2) & \sum y(t-1)u(t-1) & \sum y(t-1)u(t-2) \\ \sum y(t-2)y(t-1) & \sum y^2(t-2) & \sum y(t-2)u(t-1) & \sum y(t-2)u(t-2) \\ \sum u(t-1)y(t-1) & \sum u(t-1)y(t-2) & \sum u^2(t-1) & \sum u(t-1)u(t-2) \\ \sum u(t-2)y(t-1) & \sum u(t-2)y(t-2) & \sum u(t-2)u(t-1) & \sum u^2(t-2) \end{bmatrix}$$

ovvero:

$$R(N) = \begin{bmatrix} \gamma_{yy}(0) & \gamma_{yy}(1) & \gamma_{yu}(0) & \gamma_{yu}(1) \\ \gamma_{yy}(1) & \gamma_{yy}(0) & \gamma_{yu}(1) & \gamma_{yu}(0) \\ \gamma_{uy}(0) & \gamma_{uy}(1) & \gamma_{uu}(0) & \gamma_{uu}(1) \\ \gamma_{uy}(1) & \gamma_{uy}(0) & \gamma_{uu}(1) & \gamma_{uu}(0) \end{bmatrix} \quad (5.32)$$

²richiami presenti in appendice A.1

Ciascun blocco è simmetrico e presenta sulla diagonale principale la varianza (blocchi yy e uu) e la covarianza³ (blocchi yu e uy). I termini fuori diagonale sono le funzioni di autocorrelazione e di cross-correlazione in un passo.

Il caso si può ora estendere a un modello $ARX(n_a, n_b)$, per cui si avrebbe:

$$R(N) = \begin{bmatrix} R_{yy} & R_{yu} \\ R_{uy} & R_{uu} \end{bmatrix} \quad (5.33)$$

dove:

- R_{yy} è il blocco relativo all'autocorrelazione del segnale di uscita, di dimensioni $n_a \times n_a$ (non manipolabile);
- $R_{yu} = R_{uy}$ sono i blocchi di correlazione incrociata fra l'uscita e l'ingresso, rispettivamente di dimensioni $n_b \times n_a$ e $n_a \times n_b$ (non manipolabili in quanto comunque dipendenti dall'uscita);
- R_{uu} è il blocco relativo all'autocorrelazione del segnale di ingresso, di dimensioni $n_b \times n_b$ (completamente manipolabile).

La matrice $R(N)$ è semidefinita positiva per definizione ma è importante che essa diventi definita positiva. Ciò è vero se $|R(N)| \neq 0$.

Un risultato utile nel perseguire tale scopo è il **lemma di Schur**.

Si consideri una matrice partizionata nel seguente modo:

$$U = \begin{bmatrix} F & K \\ K^T & G \end{bmatrix}$$

condizione necessaria e sufficiente affinché U sia definita positiva è:

$$\begin{cases} G > 0 & \text{(definita positiva)} \\ F - K^T G^{-1} K > 0 & \text{(definita positiva)} \end{cases} \quad (5.34)$$

La struttura della matrice $R(N)$ coincide con quella della matrice U : il blocco $G = R_{uu}$ è completamente manipolabile in quanto dipende unicamente dall'ingresso, mentre i blocchi $K = R_{yu}$ e $F = R_{yy}$ dipendono dall'uscita, pertanto nulla può garantire che la seconda disuguaglianza del sistema nella 5.34 sia valida.

Nella pratica si cerca, in accordo con il lemma di Schur, di rispettare almeno la prima disuguaglianza del sistema

Un problema ai minimi quadrati ben congegnato ha un blocco R_{uu} (di dimensioni $n_b \times n_b$) definito positivo, strutturato come di seguito:

$$R_{uu} = \begin{bmatrix} \gamma_{uu}(0) & \gamma_{uu}(1) & \gamma_{uu}(2) & \dots & \gamma_{uu}(n_b - 1) \\ \gamma_{uu}(1) & \gamma_{uu}(0) & \gamma_{uu}(1) & \dots & \gamma_{uu}(n_b - 2) \\ \dots & \dots & \dots & \dots & \dots \\ \gamma_{uu}(n_b - 1) & \gamma_{uu}(n_b - 2) & \gamma_{uu}(n_b - 3) & \dots & \gamma_{uu}(0) \end{bmatrix} \quad (5.35)$$

³le covarianze yu e uy sono uguali, pertanto i blocchi R_{yu} e R_{uy} sono uguali.

La particolare struttura di questa matrice la rende una **matrice di Toeplitz**, cioè una matrice avente elementi coincidenti lungo le varie diagonali.

Sulla diagonale principale si trova la varianza del segnale $u(\cdot)$, sulle diagonali al di sopra e al di sotto della diagonale principale si trovano le autocorrelazioni in un passo, e così via fino alle funzioni di autocorrelazione in $n_b - 1$ passi.

Un segnale $u(\cdot)$ è detto **persistentemente eccitante di ordine n** se la sua matrice R_{uu} è invertibile (ovvero è non singolare).

Per fare una buona identificazione dunque bisogna scegliere il segnale di ingresso opportunamente: ad esempio, per identificare un ARX avente una parte esogena di ordine n_b è necessario un segnale di ingresso $u(\cdot)$ persistentemente eccitante almeno di ordine n_b . Per il lemma di Schur un segnale persistentemente eccitante di ordine n_b lo è anche per qualsiasi altro ordine minore a n_b .

Di seguito vi sono esempi di segnali di questo tipo:

- il **rumore bianco**, è il segnale persistentemente eccitante per eccellenza, di ordine qualsiasi;
- il **rumore colorato**, è persistentemente eccitante di ordine qualsiasi (dal momento che esso è un rumore bianco filtrato linearmente);
- i **segnali PRBS** (*Pseudo Random Binary Signals*), sono segnali pseudo-casuali periodici e sono persistentemente eccitanti di ordine l , dove l è il numero di registri a scorrimento impiegati per generarlo;
- i segnali ottenuti come **somme di sinusoidi equispaziate in frequenza** sono persistentemente eccitanti di ordine m , dove m è il numero di sinusoidi (il rumore bianco è un segnale di questo tipo, ottenuto sommando infinite sinusoidi).

5.5 Algoritmo dei minimi quadrati estesi

L'algoritmo dei minimi quadrati richiede la linearità nei parametri per identificare i modelli, per questa ragione è impiegato con famiglie ARX.

Il metodo dei **minimi quadrati estesi** è un algoritmo che ha lo scopo di identificare modelli ARMAX, che ricordiamo essere non lineari nei parametri.

Il generico predittore ha equazione:

$$\hat{y}(t) = (1 - W^{-1}) \circ y(t) + W^{-1}G \circ u(t-1) \quad (5.36)$$

da cui:

$$y(t) - \hat{y}(t) = W^{-1} \circ y(t) - W^{-1}G \circ u(t-1) \quad (5.37)$$

l'errore di predizione ha espressione:

$$\epsilon(t) = \frac{A(z)}{C(z)} \circ y(t) - \frac{B(z)}{C(z)} \circ u(t-1) \quad (5.38)$$

da cui:

$$C(z) \circ \epsilon(t) = A(z) \circ y(t) - B(z) \circ u(t-1) \quad (5.39)$$

poiché:

$$\begin{aligned}C(z) &= 1 + c_1 z^{-1} + \dots + c_{n_c} z^{-n_c} \\A(z) &= 1 - a_1 z^{-1} - \dots - a_{n_a} z^{-n_a} \\B(z) &= b_1 + b_2 z^{-1} + \dots\end{aligned}$$

si può scrivere l'equazione 5.39 come:

$$\begin{aligned}[1 + c_1 z^{-1} + \dots + c_{n_c} z^{-n_c}] \epsilon(t) &= [1 - a_1 z^{-1} - \dots - a_{n_a} z^{-n_a}] y(t) - \\&\quad - [b_1 + b_2 z^{-1} + \dots] u(t-1)\end{aligned}$$

da cui:

$$\begin{aligned}\epsilon(t) + c_1 \epsilon(t-1) + \dots + c_{n_c} \epsilon(t-n_c) &= y(t) - a_1 y(t-1) - \dots - a_{n_a} y(t-n_a) - \\&\quad - b_1 u(t-1) - \dots - b_{n_b} u(t-n_b)\end{aligned}$$

poiché l'errore di predizione $\epsilon(t) = y(t) - \hat{y}(t)$, allora:

$$y(t) - \hat{y}(t) = y(t) - a_1 y(t-1) + \dots - b_1 u(t-1) - \dots - c_1 \epsilon(t-1) - \dots \quad (5.40)$$

quindi si ha il predittore:

$$\begin{aligned}\hat{y}(t) &= a_1 y(t-1) + \dots + a_{n_a} y(t-n_a) + b_1 u(t-1) + \dots + b_{n_b} u(t-n_b) + \\&\quad + c_1 \epsilon(t-1) + \dots + c_{n_c} \epsilon(t-n_c)\end{aligned}$$

Se si utilizzano dunque come regressori non solo gli ingressi e le uscite ma anche gli errori di predizione passati si può scrivere l'equazione che caratterizza un predittore ARMAX. Il predittore ottenuto risulta apparentemente (il motivo della parola “apparentemente” è chiarito di seguito) lineare nei parametri pertanto si può *estendere* l'algoritmo dei minimi quadrati per risolvere il problema, che in questo ambito, prende appunto il nome di metodo dei minimi quadrati estesi.

Questa tecnica si basa sull'applicazione iterativa del metodo dei minimi quadrati. Poiché si dimostra che un predittore ARMAX è lineare in un set più ampio di regressori - e questa è la ragione per cui il predittore ARMAX risulta lineare nei parametri soltanto in apparenza - per applicare la tecnica ai minimi quadrati ai modelli ARMAX è necessario stimare gli errori di predizione: una prima stima di questi errori si può avere utilizzando un modello ARX (quindi assumendo che i parametri c_i siano tutti nulli), dopodiché si potranno stimare i parametri c_i sulla base degli errori di predizione ottenuti e, iterando il processo, si potranno raffinare i valori dei parametri. Quando gli errori di predizione ottenuti in una data iterazione sono adeguatamente indistinguibili da quelli ottenuti nell'iterazione precedente si può concludere che anche i parametri del modello non possono più migliorare significativamente.

Il vantaggio di questo metodo è dato dalla possibilità di stimare un ARMAX, il quale è un modello non lineare nei parametri, impiegando una tecnica di identificazione lineare **in maniera iterativa** (metodo più semplice dei classici algoritmi di ottimizzazione non lineare).

5.6 Identificabilità

L'**identificabilità** studia le possibilità di trovare effettivamente un modello; in altre parole, l'identificabilità riguarda lo studio della cardinalità dell'insieme dei minimi Δ . Essa si suddivide in due tipologie:

- l'**identificabilità strutturale** prevede lo studio della cardinalità di Δ in relazione alla famiglia di modelli scelta. Naturalmente, una scarsa identificabilità strutturale deriva da errori sulla scelta della famiglia dei modelli;
- l'**identificabilità sperimentale** è legata a ingressi scelti in modo inopportuno. Gli ingressi vanno infatti scelti in modo tale da essere persistentemente eccitanti dell'ordine giusto.

5.7 Analisi spettrale

Fino ad ora si sono esaminati i risultati dell'identificazione nel dominio del tempo discreto. Relativamente all'algoritmo dei minimi quadrati è opportuno effettuare un'analisi nel dominio della frequenza al fine di evidenziare particolari peculiarità. L'analisi spettrale si fonda su due risultati fondamentali, sotto le ipotesi di **stazionarietà** e **stabilità**:

- Il **teorema del filtraggio** dei segnali afferma che, dato il filtro asintoticamente stabile in figura 5.1, se il segnale $u(t)$ è un segnale stazionario (ad esempio un rumore bianco) e $\Phi_{uu}(\omega)$ il suo spettro, lo spettro dell'uscita è:

$$\Phi_{yy}(\omega) = |H(e^{j\omega})|^2 \cdot \Phi_{uu}(\omega) \quad (5.41)$$

dal punto di vista frequenziale, il filtro ridistribuisce la potenza del segnale nel dominio della frequenza, alterando la forma dello spettro.

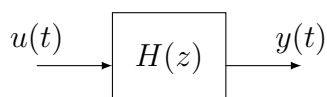


Figura 5.1: Filtraggio lineare

- Il **teorema di Wiener** afferma che la trasformata di Fourier della funzione di autocorrelazione γ di un segnale stazionario $u(\cdot)$ coincide con il suo spettro Φ :

$$\mathcal{F}\{\gamma_{uu}(\tau)\} = \Phi_{uu}(\omega) \iff \mathcal{F}^{-1}\{\Phi_{uu}(\omega)\} = \gamma_{uu}(\tau) \quad (5.42)$$

lo spettro esamina la distribuzione del segnale nel dominio della frequenza e la funzione di autocorrelazione indica quanto sono correlati i campioni nel dominio del tempo.

Vale la seguente catena di uguaglianze:

$$\bar{\mathcal{J}}(\theta) = \mathbb{E}[\epsilon^2(t)] = \text{var}[\epsilon(t)] = \gamma_{\epsilon\epsilon}(0) = \mathcal{F}\{\Phi_{\epsilon\epsilon}(\omega)\}|_{\tau=0} = \frac{1}{2\pi} \int_{-\frac{\pi}{\tau}}^{\frac{\pi}{\tau}} \Phi_{\epsilon\epsilon}(\omega) e^{2\pi j t} d\omega \quad (5.43)$$

la quale, valutata per $t = 0$ conduce a:

$$\bar{J}(\theta) = \frac{1}{2\pi} \int_{-\frac{\pi}{\tau}}^{\frac{\pi}{\tau}} \Phi_{\epsilon\epsilon}(\omega) d\omega \quad (5.44)$$

Si suppone di avere un MGD (Box-Jenkins) seguente:

$$S \quad : \quad y(t) = G_0(z) \circ u(t-1) + W_0(z) \circ e(t) \quad (5.45)$$

Il modello del predittore è:

$$M \quad : \quad \hat{y}(t) = (1 - W^{-1}(z)) \circ y(t) + W^{-1}(z)G(z) \circ u(t-1) \quad (5.46)$$

L'errore di predizione è:

$$\begin{aligned} \epsilon(t) &= y(t) - \hat{y}(t) = W^{-1}y(t) - W^{-1}Gu(t-1) = \\ &= W^{-1}[y(t) - Gu(t-1)] = \\ &= W^{-1}[G_0u(t-1) + W_0e(t) - Gu(t-1)] = \\ &= W^{-1}[(G_0 - G)u(t-1) + W_0e(t)] \end{aligned}$$

definendo $\Delta_G = G_0 - G$ si può scrivere:

$$\epsilon(t) = W^{-1}[\Delta_G \circ u(t-1) + W_0 \circ e(t)] \quad (5.47)$$

La 5.47 mette in evidenza l'errore di stima sulla funzione $G(z)$: la funzione Δ_G esprime la differenza tra la funzione $G_0(z)$ del MGD e quella del modello identificato $G(z)$. Poiché G_0 e G , in generale, hanno ordini diversi, la loro differenza è impraticabile; Δ_G è però ottenibile nel dominio della frequenza:

$$\bar{J}(\theta) = \frac{1}{2\pi} \int_{-\frac{\pi}{\tau}}^{\frac{\pi}{\tau}} |W(e^{j\omega})|^2 \cdot [|\Delta_G|^2 \cdot \Phi_{uu}(\omega) + |W_0(e^{j\omega})|^2 \cdot \lambda_e^2] d\omega \quad (5.48)$$

dove λ_e è lo spettro del rumore bianco.

Si suppone che la potenza dell'incertezza $|W_0(e^{j\omega})| \cdot \lambda_e$ è trascurabile: in tali circostanze si ha:

$$\bar{J}(\theta) = \frac{1}{2\pi} \int_{-\frac{\pi}{\tau}}^{\frac{\pi}{\tau}} \frac{\Phi_{uu}(\omega)}{|W(e^{j\omega})|^2} \cdot |\Delta_G(e^{j\omega})|^2 d\omega \quad (5.49)$$

in conclusione, nel valutare come si distribuisce la Δ_G nel dominio della frequenza, l'indice $\bar{J}(\theta)$ dipende da una **funzione di peso**:

$$g(\omega, \theta) = \frac{\Phi_{uu}(\omega)}{|W(e^{j\omega})|^2} \quad (5.50)$$

Il comportamento dell'errore di stima sulla funzione G è pesato per una funzione che dipende dall'ingresso e dalla funzione W^{-1} che è stata stimata: questa funzione ridistribuisce gli errori in frequenza.

Il seguente esempio chiarisce meglio il ruolo della funzione g . Si supponga di avere:

$$\bar{J}(\theta) = 1000 \cdot \theta_1 + 0.1 \cdot \theta_2$$

- se $\theta_1 = 1$ e $\theta_2 = 1$ allora $\bar{\mathcal{J}} = 1000.1$;
- se $\theta_1 = 0.5$ e $\theta_2 = 1$ allora $\bar{\mathcal{J}} = 500.1$;
- se $\theta_1 = 1$ e $\theta_2 = 0.5$ allora $\bar{\mathcal{J}} = 1000.05$;

Maggiore è il peso di una componente, tanto più significativa sarà la riduzione di $\bar{\mathcal{J}}$ da parte dell'algoritmo di ottimizzazione; nella ripartizione in frequenza, saranno minimizzate maggiormente le frequenze alla quale il peso della funzione g è maggiore.

In questo modo si può prioritizzare l'algoritmo di minimizzazione nei confronti delle componenti di θ più grandi, rispetto a quelle più piccole. Per realizzare ciò si deve poter ottenere una W di tipo **passa basso** (con più poli che zeri).

La funzione g dà dunque più peso alle alte frequenze, minimizzandole maggiormente rispetto alle basse frequenze, pertanto **gli errori alle basse frequenze sono maggiormente presenti**, dal momento che hanno una tendenza molto minore ad essere attenuati.

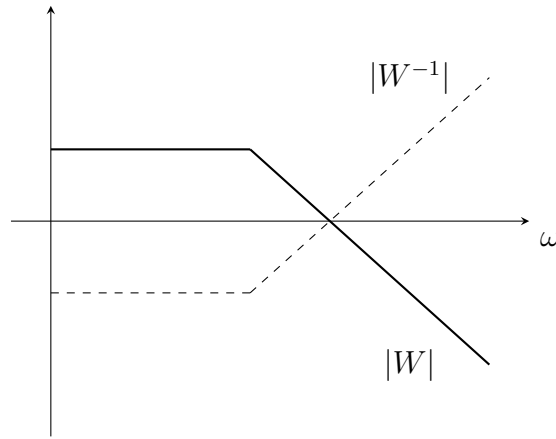


Figura 5.2: Diagrammi indicativi di Bode delle ampiezze di W e W^{-1} .

La dipendenza di $\bar{\mathcal{J}}$ dalla funzione G dipende essenzialmente da W^{-1} in quanto il rumore bianco in ingresso ha spettro costante. Senza alcun tipo di accorgimento sulla W , la tecnica dei minimi quadrati identificherebbe meglio il comportamento alle alte frequenze rispetto a quello alle basse frequenze, per questa ragione la funzione g non deve privilegiare le alte frequenze ed essere il più possibile “piatta”.

5.8 Prefiltraggio

Se si identifica, in un primo momento, la W si può fornire in ingresso un rumore colorato con un filtro lineare la cui funzione di trasferimento è simile alla W , ottenendo così una cancellazione fra il numeratore e il denominatore della g . A questo punto si dovrebbe fare una seconda identificazione per ottenere una nuova W (e quindi un nuovo rumore colorato in ingresso).

Per evitare il reiterarsi del processo di identificazione si ricorre alla **tecnica del pre-filtraggio**, tramite la quale si filtra anche l'uscita con lo stesso filtro usato per filtrare l'ingresso.

Si supponga dunque di utilizzare un filtro $L(z)$ asintoticamente stabile, il cui modulo è $|L(e^{j\omega})|^2$, con cui si filtra un ingresso $u(t)$ in modo tale che:

$$u_L(t) = L(z) \cdot u(t) \quad (5.51)$$

Filtrando anche l'uscita con lo stesso filtro si ottiene:

$$y_L(t) = L(z) \cdot y(t) \quad (5.52)$$

L'errore di predizione è:

$$\epsilon(t) = W^{-1} [y(t) - G(z) \cdot u(t-1)] \quad (5.53)$$

prefiltrando, si ottiene un errore di predizione pari a:

$$\epsilon_L(t) = W^{-1} [y_L(t) - G(z) \cdot u_L(t-1)] = W^{-1} \cdot L [y(t) - G \cdot u(t-1)] \quad (5.54)$$

quindi, applicare un filtro sia all'ingresso che all'uscita corrisponde (per l'ipotesi di linearità) ad applicare il filtro all'errore di predizione:

$$\epsilon_L(t) = L(z) \cdot \epsilon(t) \quad (5.55)$$

in questo modo si può evitare di reiterare il processo di identificazione:

$$g(\theta, \omega) = \frac{|L(e^{j\omega})|^2}{|W(e^{j\omega})|^2} \cdot \Phi_{uu}(\omega) \quad (5.56)$$

La tecnica del prefiltraggio prevede di filtrare prima di effettuare l'ottimizzazione. L'operazione di prefiltraggio può portare dei benefici se gli errori in frequenza non sono bilanciati; se gli errori sono invece già di per sé ben distribuiti in frequenza, essa non comporta alcun miglioramento.

La tecnica del prefiltraggio è conveniente perché “costa poco” ma non sempre porta a dei benefici perché in questo caso l'incertezza viene trascurata.

5.9 Operazioni di pre-processing

Per migliorare i risultati dell'identificazione vi sono altre operazioni preventive che si possono effettuare sui dati:

1. **Downsampling:** se, al momento della raccolta dei dati, non si è potuto scegliere l'intervallo di campionamento T e questo risulta troppo piccolo, il processo di identificazione può diventare più difficile. Al diminuire di T rispetto alle dinamiche del sistema, i poli del sistema tendono a concentrarsi attorno al punto⁴ $z = -1$. In tali circostanze, l'algoritmo di ottimizzazione deve scovare differenze molto lievi: il valore del segnale assunto durante T diventa trascurabile poiché il campione dell'uscita $y(T+1)$ è quasi uguale a $y(T)$. Se non vi sono variazioni significative di segnale allora l'uscita tende a essere uguale in entrambi gli istanti. Un predittore a un passo, con troppi campioni, non riuscirà dunque a spiegare la dinamica del sistema. Conviene dunque operare un sottocampionamento o *downsampling*, introducendo una frequenza di campionamento fittizia adeguatamente minore di quella originaria.

⁴per via della corrispondenza tra piano s e piano z , $z = e^{sT}$

2. **Filtraggio Antialiasing:** l'*aliasing* (o equivocazione) di un segnale è un effetto indesiderato che si verifica quando non si sceglie opportunamente la frequenza di campionamento, che dev'essere almeno pari al doppio della massima frequenza del segnale da campionare (in accordo con il teorema di Shannon). Se c'è aliasing, le code spettrali del segnale si sovrappongono causando equivocazione, per cui il segnale⁵ non è ricostruibile correttamente. I filtri antialiasing garantiscono la ricostruibilità tagliando lo spettro del segnale in accordo con la condizione imposta dal teorema di campionamento. L'aliasing è un fenomeno che può essere implicato anche dalla presenza di rumore sul segnale, per questo le operazioni di filtraggio antialiasing vanno effettuate prima del campionamento.
3. **Peak Shaving:** a volte il segnale presenta dei picchi (o *spike*) indesiderati, dovuti ad anomalie occasionali: se ci sono dei punti in cui l'errore di predizione diventa molto elevato (il cui quadrato è ancora più elevato) l'algoritmo di ottimizzazione si sforzerà di riprodurre anche questi fenomeni indesiderati. Per risolvere questo problema si può usare un filtro mediano o comunque una o più operazioni di media sui dati per livellare i picchi portandoli a valori più vicini alla media campionaria relativa ai dati in ingresso.
4. **Signal Slicing:** questa operazione è conveniente quando ci si accorge che nei dati c'è un'evidente variazione di qualche offset ovvero sono presenti delle discontinuità significative tra i valori assunti dai dati lungo tutti gli istanti di tempo (figura 5.3). In genere questi comportamenti sono imputabili a cambi delle condizioni operative (per esempio l'applicazione di un carico). In tali circostanze conviene:
 - individuare i punti di lavoro, in corrispondenza dei quali si tagliano i diversi sottoinsiemi dei dati (*slicing*);
 - rimuovere gli offset in ingresso e in uscita (*detrend*);
 - rimettere insieme le porzioni di dati precedentemente separate (*merging*);
 per poi procedere con l'identificazione, ricordando infine di ripristinare gli offset precedentemente rimossi.

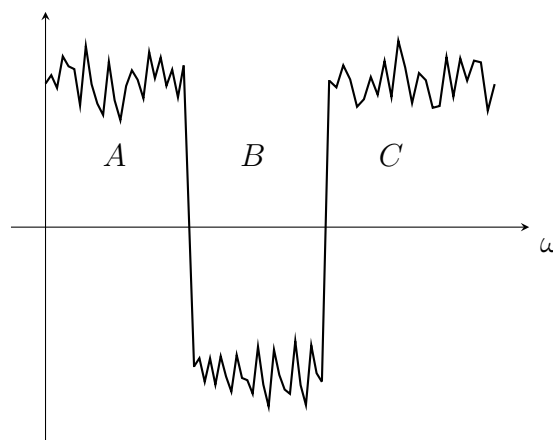


Figura 5.3: sottoinsiemi di dati *A*, *B* e *C* estrapolabili mediante slicing

⁵inteso come serie di sinusoidi e quindi caratterizzato da più frequenze

5. **Signal Scaling:** nei sistemi MIMO l'indice $\mathcal{J}$ è pari alla somma delle somme degli errori di predizione (gli errori su tutte le n uscite sono dati da somme che a loro volta sono delle somme).

$$\mathcal{J} = \frac{1}{n \cdot N} \left(\sum_{j=1}^n \sum_{i=1}^N [\epsilon_j(\tau_i)]^2 \right) \quad (5.57)$$

Poiché le grandezze in uscita possono essere eterogenee, quando queste non sono dimensionalmente confrontabili oppure presentano unità di misura riferite a differenti ordini di grandezza allora bisogna normalizzare tutti i segnali (operazione di *scaling*). In questo modo gli indici integrali degli errori sono confrontabili e i valori minimizzati sono consistenti.

Parte II

Identificazione non lineare

Capitolo 6

Identificazione di sistemi non lineari

6.1 Da lineare a non lineare

I procedimenti di **identificazione non lineare** sono notevolmente più complessi di quelli nel caso lineare. Per semplicità di trattazione si fanno, le seguenti ipotesi semplificative:

1. si assume che l'ordine della parte autoregressiva (AR) sia uguale all'ordine della parte esogena (X). Tutte le uscite e gli ingressi hanno lo **stesso ordine dinamico** m ;
2. si assume che il **sistema è strettamente proprio**. L'uscita $y(k)$ non è immediatamente influenzata dall'ingresso $u(k)$, ma ritardata di un passo (l'uscita $y(k)$ è influenzata dall'ingresso all'istante precedente $u(k-1)$);
3. non vi sono ulteriori ritardi fra l'ingresso $u(k)$ e l'uscita $y(k)$ ($n_k = 1$);
4. non si presta attenzione al modello dell'incertezza per via della complessità di quest'ultima.

Dal momento che non vi è incertezza, tutte le differenze fra i modelli trattati (ARX, ARMAX, Output Error, etc.) vengono meno: tutto confluisce in un unico modello che prende il nome di **NARX** (*Nonlinear AutoRegressive eXogenous Model*) anche chiamato (talvolta impropriamente) **NARMAX**.

Un modello NARX è rappresentato dalla seguente equazione:

$$\hat{y}(t) = f(y(t-1), y(t-2), \dots, y(t-m), u(t-1), u(t-2), \dots, u(t-m)) \quad (6.1)$$

Più in generale si può determinare una rappresentazione in variabili di stato dello stesso modello:

$$\begin{cases} x(k+1) = g(x(k), u(k)) \\ y(k) = h(x(k)) \end{cases} \quad (6.2)$$

I modelli non lineari sono anche detti **modelli a dinamica esterna** in quanto la dinamica non è una caratteristica delle funzioni non lineari che caratterizzano questi modelli, le quali

sono intrinsecamente statiche (prive di memoria), ma è imposta esternamente passando al sistema non lineare la storia passata del sistema.

Nel caso in cui la dinamica esterna sia ottenuta tramite le uscite del sistema si ha un predittore (fig. 6.1a). Se invece si ponesse il sistema in retroazione, passando al modello la stima $\hat{y}(k)$ dell'uscita si avrebbe un simulatore (fig. 6.1b).

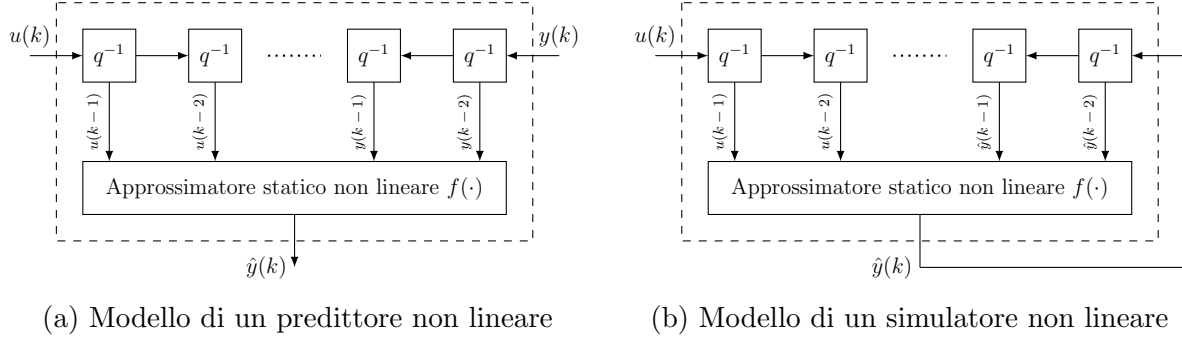


Figura 6.1: Modelli di sistemi a dinamica esterna

Dai modelli lineari più comuni (ARX, ARMAX e Output Error) si ottengono i corrispondenti **modelli non lineari**, avendo definito, come al solito:

$$\epsilon(k) = y(k) - \hat{y}(k)$$

e strutturando i regressori come segue:

$$NARX : \quad \varphi(k) = [u(k-1) \dots u(k-m) \quad y(k-1) \dots y(k-m)]^T$$

$$NARMAX : \quad \varphi(k) = [u(k-1) \dots u(k-m) \quad y(k-1) \dots y(k-m) \quad \epsilon(k-1) \dots \epsilon(k-m)]^T$$

$$NOE : \quad \varphi(k) = [u(k-1) \dots u(k-m) \quad \hat{y}(k-1) \dots \hat{y}(k-m)]^T$$

Le precedenti equazioni descrivono la struttura dei regressori rispettivamente per modelli *Non-Linear ARX*, *Non-Linear ARMAX* e *Non-Linear Output Error*.

6.2 Modelli di Hammerstein e Wiener

Una classe di modelli molto importanti nell'identificazione di sistemi non lineari sono i **modelli di Hammerstein e di Wiener**.

Se un dato sistema dinamico lineare $\mathcal{S}$ presenta delle **nonlinearità statiche** (come ad esempio una saturazione), queste possono essere modellate con dei blocchi puramente algebrici: se queste nonlinearità sono poste a monte di $\mathcal{S}$ si ha un modello di Hammerstein (figura 6.2); se invece si trovano a valle di $\mathcal{S}$ si ottiene un modello di Wiener (figura 6.3). Nel caso in cui le nonlinearità si trovano sia a monte che a valle di $\mathcal{S}$ si ha un modello di Hammerstein-Wiener (figura 6.4).

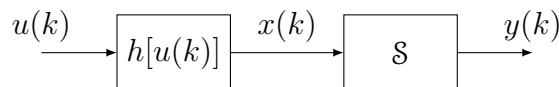


Figura 6.2: Modello di Hammerstein

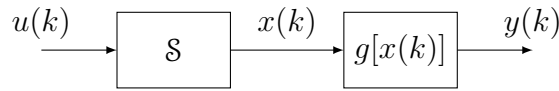


Figura 6.3: Modello di Wiener

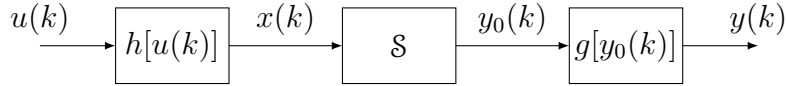


Figura 6.4: Modello di Hammerstein-Wiener

Molti fenomeni nonlineari sono modellabili attraverso blocchi statici, tuttavia questa strategia non è sempre valida perché non tutte le nonlinearità sono statiche: l'isteresi, per esempio, è una nonlinearità dinamica e la sua descrizione richiede la conoscenza della storia passata dell'ingresso e dell'uscita.

6.3 Scelta del segnale di ingresso

Nei modelli a dinamica esterna (figura 6.1a), il progettista può scegliere solamente il segnale di ingresso $u(\cdot)$.

Gli ingressi dell'approssimatore statico $f(\cdot)$ non sono mai del tutto indipendenti dal momento che i campioni successivi dei segnali sono fra loro legati.

Gli ingressi dell'approssimatore $f(\cdot)$ sono costituiti sostanzialmente da tanti segnali che condividono una stessa forma d'onda e differiscono solamente per un ritardo nel tempo. Se il **tempo di campionamento** T_s è molto piccolo, i campioni successivi $u(k-1)$ e $u(k)$ di un dato ingresso $u(\cdot)$ tenderanno ad essere molto simili fra loro e ciò introdurrebbe delle ridondanze nell'algoritmo di identificazione, compromettendone l'efficacia. Al ridursi di T_s , tutti i poli del sistema tendono a concentrarsi nell'intorno di $z = 1$: in tali circostanze sarà necessario un ordine elevato per rappresentare la dinamica del sistema, il che non si traduce unicamente in un aumento dei coefficienti ma anche di ingressi, dal momento che condizioni di sovracampionamento (T_s troppo piccolo) il numero di variabili può crescere a dismisura, rendendo potenzialmente ingestibile l'identificazione; se invece si ha sottocampionamento (T_s troppo grande) si rischia di non identificare bene e in modo sufficientemente completo la dinamica del sistema. È dunque di importanza fondamentale scegliere con attenzione il passo di campionamento.

Per avere dei dati sufficientemente informativi sul comportamento di questo sistema bisogna applicare un ingresso la cui **variabilità** sia compatibile con il numero di **punti di equilibrio** del sistema.

L'esempio seguente chiarirà meglio questo importante aspetto. Si considera un sistema dinamico non-lineare (figura 6.5) costituito da un blocco lineare $G(z)$, il cui ingresso è sottoposto a saturazione: la saturazione è un tipo di non-linearità statica e può essere modellata tramite un blocco puramente algebrico. Si impiega dunque un modello di Hammerstein, la cui non-linearità statica è posta a monte della dinamica lineare.

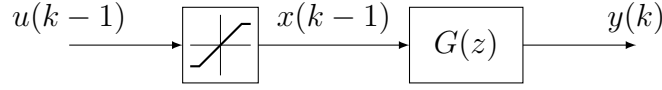


Figura 6.5: Sistema con nonlineari  statica sull'ingresso (modello di Hammerstein)

Il blocco statico di saturazione   modellato con una funzione arcotangente:

$$f(u) = 0.06 \cdot \arctg[u(k-1)] \quad (6.3)$$

e il blocco dinamico lineare ha funzione di trasferimento:

$$G(z) = \frac{1}{z + 0.94} \quad (6.4)$$

La legge (non-lineare) del MGD  :

$$y(k) = 0.06 \cdot \arctg[u(k-1)] + 0.94 \cdot y(k-1) \quad (6.5)$$

La saturazione introduce infiniti punti di equilibrio (a derivata nulla), localizzati oltre i limiti di saturazione¹.

  importante che il segnale di ingresso $u(k)$ esplori in maniera sufficientemente capillare il dominio della non-linearit .

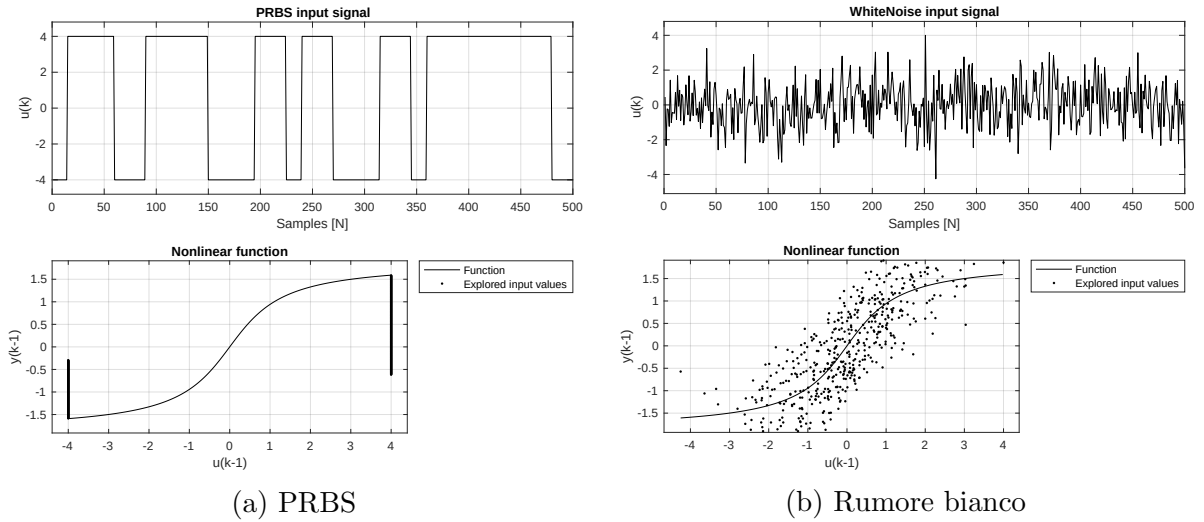


Figura 6.6: Ingressi per identificazione non-lineare (PRBS, Rumore bianco)

Un segnale di ingresso PRBS (*Pseudo-Random Binary Signal*) con ampiezza picco-picco $[-4, 4]$ (figura 6.6a) pu  assumere due soli valori distinti ed   inadatto per il sistema in esame, che ha un numero pressoch  infinito di punti di equilibrio.

¹la funzione arcotangente approssima il blocco di saturazione, perci  la sua derivata non   perfettamente costante; essa tende asintoticamente a zero man mano che ci si allontana dai limiti di saturazione.

Per un'esplorazione adeguata del dominio della non-linearità è necessario un **ingresso con ampiezza a più alta variabilità**, in grado di assumere più valori distinti.

Si può pensare allora di scegliere un segnale di ingresso che sia un rumore bianco (figura 6.6b) con lo stesso intervallo di ampiezza $[-4, 4]$: in questa situazione si esplora molto bene il dominio del blocco non-lineare ma l'ampiezza del segnale varia talmente rapidamente da impedire il raggiungimento dell'equilibrio (e quindi una sua esplorazione). Il problema del rumore bianco è dunque la sua eccessiva rapidità di variazione in frequenza, che comporta l'impossibilità di esplorare i punti di equilibrio.

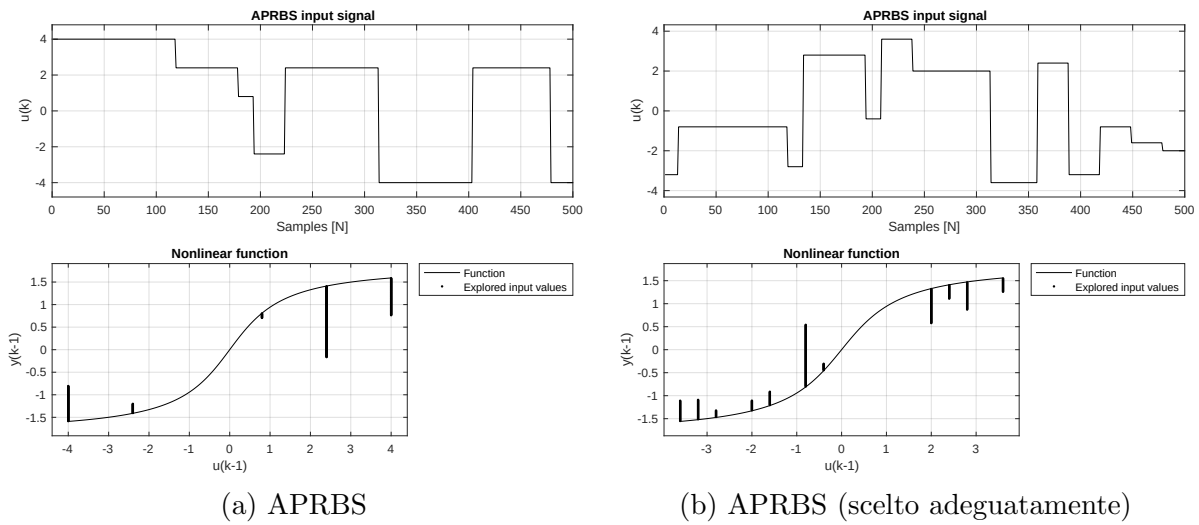


Figura 6.7: Ingressi per identificazione non-lineare (APRBS)

Bisogna allora produrre un ingresso, che abbia sufficiente variabilità in ampiezza e che, allo stesso tempo, non vari troppo rapidamente. I segnali **APRBS** (*Amplitude-Modulated PRBS*, in figura 6.7) rappresentano un'estensione dei segnali PRBS e sono in grado di soddisfare questo requisito.

Un segnale APRBS è un segnale PRBS modulato in ampiezza (figura 6.7a), in grado di assumere vari livelli di ampiezza, che rimangono costanti per un certo numero minimo di campioni T_h (*Hold Time*), chiamato **intervallo di minima permanenza**.

In questo modo si realizza un giusto compromesso fra l'esplorazione dei transitori (resa possibile dall'elevata variabilità dell'ampiezza) e l'esplorazione del regime (ossia dei punti di equilibrio) del sistema non lineare.

La figura 6.7a mostra un APRBS con cinque livelli di ampiezza: in questa situazione si esplora sufficientemente a lungo cinque diversi valori dello spazio dell'ingresso.

Riducendo il valore di T_h e modulando con una maggiore risoluzione (figura 6.7b), si riesce, a parità di numero di campioni, ad esplorare meglio il dominio della non-linearità, senza compromettere il raggiungimento dell'equilibrio.

6.4 Algoritmi a discesa del gradiente

Si è visto, nel caso di identificazione lineare, che la tecnica ai minimi quadrati è una tecnica di inversione matriciale.

Per quanto riguarda l'identificazione non lineare, si deve rinunciare alla risoluzione del problema in un solo passo algoritmico: le tecniche di ottimizzazione non lineare trovano il minimo dell'indice $\mathcal{J}$ (che è in questo caso una funzione non lineare di θ) in maniera necessariamente iterativa².

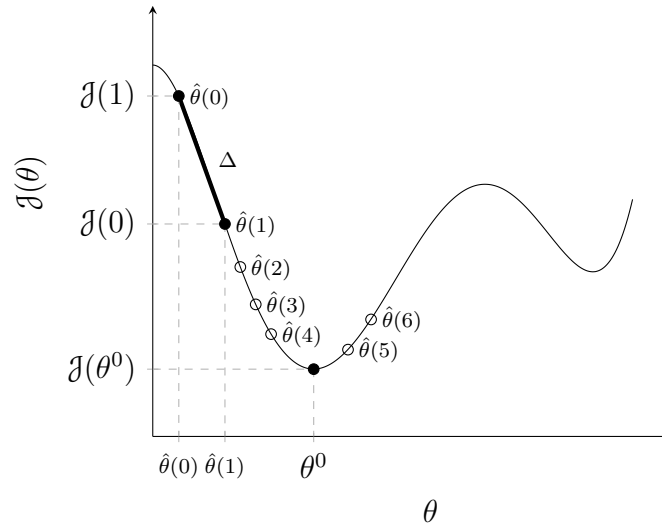


Figura 6.8: Rappresentazione schematica di una tecnica a discesa del gradiente

Fra gli algoritmi di ottimizzazione non lineare rientrano gli **algoritmi a discesa del gradiente**: queste tecniche ricercano i minimi di una funzione non lineare in uno spazio generalmente multidimensionale (che per ragioni di facilità di rappresentazione è rappresentato in una sola dimensione in figura 6.8).

Le tecniche a discesa del gradiente ricercano il vettore dei parametri θ tale per cui si abbia un valore minimo dell'indice quadratico $\mathcal{J}$.

Calcolando il valore di $\mathcal{J}$ per due valori di $\hat{\theta}$ sufficientemente vicini si può stimare la derivata (o il gradiente, nel caso multidimensionale):

$$\Delta = \frac{\mathcal{J}(1) - \mathcal{J}(0)}{\hat{\theta}(1) - \hat{\theta}(0)} \quad (6.6)$$

il cui segno (negativo, nel caso in figura 6.8) fornirà informazioni utili ad effettuare la discesa verso il punto di minimo.

Noto il valore di Δ , il punto successivo $\hat{\theta}(2)$ è determinato nel modo seguente:

$$\hat{\theta}(2) = \hat{\theta}(1) - \alpha \Delta \quad (6.7)$$

²anche il metodo dei minimi quadrati estesi richiede l'iterazione dei minimi quadrati per stimare un ARMAX, che è appunto non lineare nei parametri.

dove α è un fattore di convergenza dell'algoritmo che prende il nome di *learning rate*.

Nel caso in cui $\Delta < 0$ (gradiente negativo) il nuovo valore di θ sarà ricercato più a destra del valore precedente (spostamento positivo) secondo un passo la cui ampiezza dipende dall'ampiezza del gradiente, modulata dal learning rate α .

Un cambio del segno di Δ è interpretabile come il superamento di un punto di minimo: ciò consente di portare l'algoritmo alla convergenza.

Il valore del learning rate ha effetto sul comportamento e sulla velocità di convergenza dell'algoritmo: un valore di α troppo piccolo potrebbe comportare un aumento esagerato del numero di iterazioni necessarie e quindi del tempo impiegato dall'algoritmo a convergere. Se, viceversa, α fosse troppo grande, l'algoritmo avrebbe un comportamento oscillatorio e potrebbe anche non raggiungere mai la convergenza, scavalcando continuamente il punto di minimo e oscillando attorno ad esso.

Negli algoritmi evoluti si utilizzano learning rate dinamici, che sono funzione della storia passata, aumentano quando si è lontani dal punto di ottimo e diminuiscono in un intorno ragionevole del punto di ottimo.

Nel caso multidimensionale ci vogliono più learning rate: uno per ciascuna dimensione dello spazio dei parametri, dal momento che le velocità di convergenza richieste non sono in generale uguali per tutti i parametri.

I metodi più potenti a discesa del gradiente impiegano le cosiddette **tecniche di secondo ordine**, dove i learning rate sono sostituiti con la matrice Hessiana. La matrice Hessiana contiene informazioni sulla conformazione della funzione obiettivo e può fornire una stima automatica delle velocità di convergenza richieste per ciascun asse dello spazio dei parametri. Il suo calcolo è un'operazione potenzialmente onerosa dal punto di vista computazionale, tuttavia risulta, nella pratica, molto efficiente e viene quasi sempre usata.

La tecnica di discesa del gradiente presenta uno svantaggio di cui tener sempre conto: quando la funzione obiettivo da minimizzare è multimodale, la funzione presenta più ottimi locali (questo accade quasi sempre) ed è facile che l'algoritmo si concentri su uno di questi, che non necessariamente è l'ottimo globale. L'esplorazione dello spazio dei parametri, al fine di individuare eventuali estremi assoluti, è un problema che può diventare molto complicato al crescere delle dimensioni e le tecniche a discesa del gradiente non sono in grado di esplorare globalmente la funzione obiettivo. Questo inconveniente è conosciuto come **problema degli ottimi locali** ed è appunto una limitazione tipica dei metodi basati sul vettore gradiente.

6.5 Modelli alle variazioni e modelli affini al controllo

L'approccio alle dinamiche esterne è basato sulla mappatura dei precedenti ingressi e delle precedenti uscite rispetto all'uscita reale.

Un generico modello non lineare è espresso nella forma:

$$y(k) = f[u(k-1), u(k-2), \dots, u(k-m), y(k-1), y(k-2), \dots, y(k-m)] \quad (6.8)$$

ovvero l'uscita è una funzione non lineare dei valori precedenti degli ingressi e delle uscite.

Il modello rappresentato nella 6.8 può anche essere rappresentato nella forma seguente:

$$y(k) = f[u(k-1), \Delta u(k-1), \dots, \Delta^{m-1}u(k-1), y(k-1), \Delta y(k-1), \dots, \Delta^{m-1}y(k-1)] \quad (6.9)$$

che prende il nome di **modello in forma di variazioni**.

In un modello in forma di variazioni, il simbolo Δ^m indica l'operatore differenza di ordine m :

$$\begin{aligned} \Delta &= 1 - q^{-1} \\ \Delta^2 &= (1 - q^{-1})^2 \\ &\vdots \\ \Delta^m &= (1 - q^{-1})^m \end{aligned}$$

Un sistema del secondo ordine in forma di variazioni, avrebbe la seguente forma:

$$y(k) = f[u(k-1), \Delta u(k-1), y(k-1), \Delta y(k-1)] \quad (6.10)$$

dove le quantità:

$$\begin{aligned} \Delta u(k-1) &= u(k-1) - u(k-2) \\ \Delta y(k-1) &= y(k-1) - y(k-2) \end{aligned}$$

rappresentano le derivate (nel discreto) dei segnali.

Il modello messo in forma di variazioni consente di eludere a priori la problematica dell'esplorazione del transitorio e del regime, dal momento che il modello presenta esplicitamente le derivate dei segnali. I risultati sono dunque molto più facilmente interpretabili.

Questa tecnica è ad esempio adoperata nella pratica dei controlli automatici per prevenire l'insorgenza del fenomeno di wind-up.

Nel contesto dell'identificazione, è però presente il rumore, i cui effetti crescono all'aumentare dell'ordine delle differenze, per questa ragione, i modelli in forma di variazioni non sono mai utilizzati per l'identificazione di sistemi.

Una classe di modelli che ben si presta al controllo è quella dei **modelli affini nell'azione del controllo**.

Un modello affine nell'azione del controllo viene solitamente utilizzato per la progettazione dei controllori ed è lineare rispetto all'azione di controllo $u(k-1)$. La sua espressione generica è la seguente:

$$y(k) = b_1 u(k-1) + f[u(k-2), \dots, u(k-m), y(k-1), \dots, y(k-m)] \quad (6.11)$$

Con questo genere di modello è possibile realizzare dei controllori la cui azione di controllo è data da:

$$u(k-1) = \frac{1}{b_1} [r(k) - f[u(k-2), \dots, u(k-m), y(k-1), \dots, y(k-m)]] \quad (6.12)$$

dove $r(k)$ indica l'andamento dell'uscita desiderata (segnale di riferimento), nell'istante di tempo successivo. Sostituendo la 6.12 nella 6.11,

$$y(k) = r(k) \quad (6.13)$$

si ottiene un'uscita coincidente con il segnale di riferimento.

Questa tecnica prende il nome di **feedback linearization** (argomento approfondito più in avanti) e consente di ottenere una cancellazione della dinamica non lineare del plant tramite la retroazione. Questa tecnica è impiegabile se l'azione di controllo è svincolabile dalla dinamica non lineare del sistema.

Capitolo 7

Approssimatori e reti neurali artificiali

7.1 Approssimatori

La funzione non-lineare è descritta da un comportamento sperimentale. È importante scegliere una famiglia di modelli adeguata, che possa ben approssimare la funzione $f(\cdot)$. Cercare un approssimatore corrisponde a scegliere la tipologia di regressori¹ da fornire in ingresso al sistema e poi trovare il vettore dei parametri seguendo lo stesso approccio impiegato nell'identificazione di sistemi lineari, orientato alla minimizzazione dell'indice quadratico.

Il modello è descritto dalla funzione:

$$y(k) = f[x] \quad (7.1)$$

dove x è il vettore dei regressori (che rappresenta tutte le informazioni sulla storia passata) e f è non-lineare.

Si vuole cercare un approssimatore $g(\cdot)$ che ben approssimi $f(\cdot)$ in tutto l'intervallo di ingressi di interesse:

$$\hat{y} = g[x] = g(x, w) \quad (7.2)$$

dove il vettore w indica il vettore dei parametri dell'approssimatore.

L'errore di predizione è definito, come al solito, nel modo seguente:

$$\epsilon = y - \hat{y} \quad (7.3)$$

L'obiettivo è dunque trovare il vettore dei parametri in modo tale che ϵ sia minimo.

Gli approssimatori possono essere classificati per **tipologia**:

¹D'ora in avanti, per semplicità di notazione, si sceglie di indicare con x il vettore dei regressori, evitando di specificare la dipendenza dall'istante temporale k .

- **Approssimatori puramente lineari**, del tipo:

$$\hat{y} = ax \quad (w \equiv a) \quad (7.4)$$

- **Approssimatori lineari nei parametri (LIP)**, (*Linear In the Parameters*), definiti come combinazione lineare (media pesata) di funzioni elementari note con dei parametri legati all'uscita da una relazione lineare:

$$\hat{y} = \phi_1(x)\theta_1 + \dots + \phi_n(x)\theta_n = \phi^T \theta \quad (w \equiv \theta) \quad (7.5)$$

nell'approssimazione lineare i regressori sono valori passati dell'uscita e dell'ingresso, in questo caso i regressori diventano gli ingressi per funzioni elementari non-lineari.

- **Approssimatori puramente non-lineari**, del tipo:

$$\hat{y} = g(x, w) \quad (7.6)$$

o per famiglia:

- **Funzioni lineari o affini**: si utilizzano per rappresentare l'uscita con l'equazione di una retta $\hat{y} = mx + q$. Il termine m è una pendenza e rappresenta l'attributo *lineare* dell'approssimatore, mentre il termine q è un offset e indica la caratteristica *affine* dell'approssimatore.
- **Polinomi**: l'uscita dell'approssimatore è espressa come combinazione lineare di potenze dell'ingresso. L'espansione in serie di Taylor è un metodo con cui cercare approssimatori polinomiali. Il vantaggio degli approssimatori polinomiali è la caratteristica di linearità nei parametri, che permette di usare semplici algoritmi ai minimi quadrati.
Sfruttare i polinomi per approssimare modelli non-lineari è tuttavia un'operazione da trattare piuttosto delicatamente, dal momento che presenta uno svantaggio piuttosto sveniente: se il vettore x ha più componenti, un aumento, anche lieve, dell'ordine dell'approssimatore causa un aumento esponenziale del numero dei coefficienti da cercare, questa problematica è detta *Curse of Dimensionality*: per questa ragione, i polinomi sono impiegati per approssimare localmente funzioni non-lineari piuttosto regolari nella regione di interesse.
- **Splines**: sono impiegate per evitare la *Curse of Dimensionality*, a parità di qualità di approssimazione. Le splines sono porzioni di polinomi opportunamente raccordati in corrispondenza di specifici nodi. L'idea di base è di suddividere la regione in cui vogliamo approssimare $f(\cdot)$ in sottoregioni e utilizzare un polinomio diverso e di ordine basso in ciascuna sottoregione.
Una spline di ordine k con N sottoregioni approssima la funzione $f(\cdot)$ con N polinomi di ordine $k - 1$, quindi ha $N \cdot k$ parametri. Le splines sono più flessibili dei polinomi e, anche se non garantiscono l'eliminazione del problema della dimensionalità, consentono di attenuarlo.
- **Reti neurali**: è la categoria di approssimanti più moderna e potente, ispirata alle reti di cellule neurali presenti nel cervello degli organismi biologici.

7.2 Reti neurali artificiali

7.2.1 Introduzione

Una **rete neuronale artificiale** (*ANN: Artificial Neural Network*) è un approssimatore il cui funzionamento è ispirato a quello delle reti neurali biologiche, proprie degli organismi viventi.

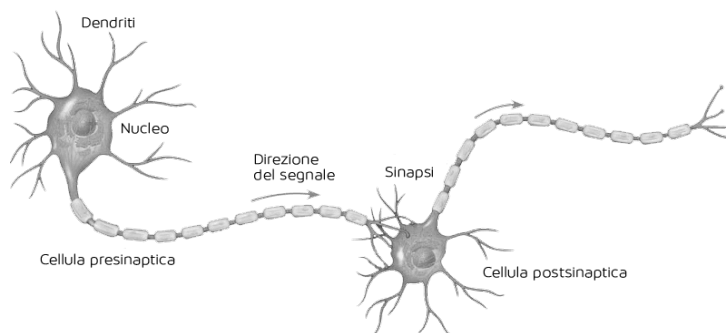


Figura 7.1: Connessione sinaptica fra neuroni biologici

Un neurone biologico, rappresentato in figura 7.1, è una cellula biologica che si comporta come un meccanismo di input-output elementare: l'informazione proveniente dai **dendriti** (gli ingressi del neurone) si propaga sotto forma di impulso elettrico, è elaborata nel **nucleo**, e giunge all'**assone** (l'uscita del neurone). La connessione fra due neuroni è detta **sinapsi** e avviene tramite un processo elettro-chimico. Queste cellule scambiano informazioni con altre cellule dello stesso tipo, formando una rete fortemente interconnessa: un normale cervello umano possiede 10^{10} neuroni e ciascun neurone ha un numero di connessioni sinaptiche compreso fra 10^4 e 10^5 . Il tempo di risposta di un neurone biologico è di circa 10^{-3} secondi e un processo come il riconoscimento di una sagoma avviene in circa 10^{-1} secondi. Si può facilmente dedurre, da come è strutturata una rete neuronale, che l'elaborazione dell'informazione avviene in maniera intrinsecamente parallela.

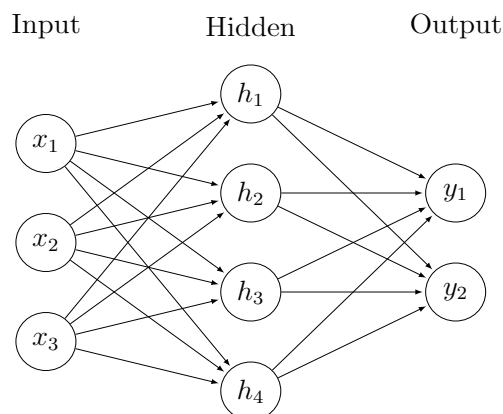


Figura 7.2: Rete neuronale artificiale

Una rete neuronale artificiale è rappresentabile mediante un grafo orientato (figura 7.2) in cui ciascun nodo rappresenta un neurone, il quale è una **funzione matematica elementare**. A ciascun arco del grafo corrisponde una sinapsi, rappresentata da un parametro numerico che prende il nome di **peso sinaptico**.

7.2.2 Topologie e strutture

Il **neurone artificiale**, anche detto **perceptrone** (*Perceptron*), è un sistema MISO (*Multiple-Input-Single-Output*): gli ingressi del neurone $\{x_1, x_2, \dots, x_n\}$ sono ad esso collegati tramite dei pesi sinaptici $\{w_1, w_2, \dots, w_n\}$. Il nucleo $\mathcal{N}$ del neurone ospita una funzione matematica elementare, la quale elabora gli n ingressi pesati fornendo l'uscita y , calcolata come somma pesata degli ingressi.

L'uscita di ciascun neurone presenta, oltre alla somma pesata, un peso costante w_0 (ossia un offset) che prende il nome di **bias**.

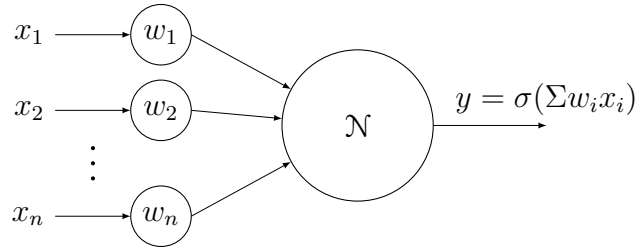


Figura 7.3: Neurone artificiale

Una ANN è dunque un sistema costituito dalla interconnessione di più neuroni artificiali secondo una ben specifica topologia. La topologia più comune è la cosiddetta **MLP** (*Multi-Layer Perceptron*), costituita da **strati** o **layer**. Uno strato è definito come l'insieme di neuroni che condividono gli stessi ingressi (figura 7.4). Le uscite di uno strato costituiscono gli ingressi dello strato successivo.

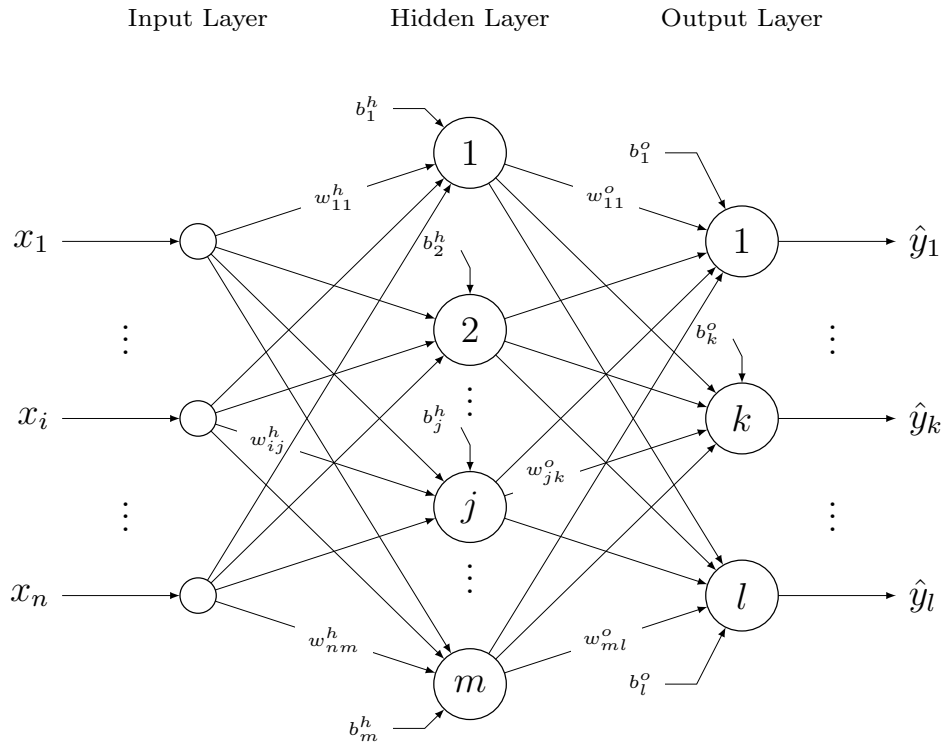


Figura 7.4: Rete neurale con architettura MLP

In questa architettura vi è uno **strato di ingresso** (*Input Layer*) attraverso il quale i dati in ingresso sono immessi nella rete, uno **strato di uscita** (*Output Layer*), che deve contenere tanti neuroni quante sono le uscite del sistema e da uno o più **strati nascosti** (*Hidden Layer*), i quali possono contenere un numero qualsiasi di neuroni.

Ad esempio, la rete in figura 7.4 ha n ingressi, contiene un solo strato nascosto composto da m neuroni e presenta l uscite.

La topologia MLP è la più classica ed è un'architettura di tipo **feed-forward**, cioè le informazioni transitano “in avanti”, lungo la rete: le uscite di uno strato sono ingressi unicamente per lo strato successivo.

Vi è anche un'altra topologia rilevante che prevede di retroazionare le uscite dei neuroni reimmettendole in ingresso agli stessi: le reti neurali di questo tipo sono dette **reti neurali ricorrenti**.

Una rete di questo tipo è la **rete di Hopfield**, mostrata in figura 7.5

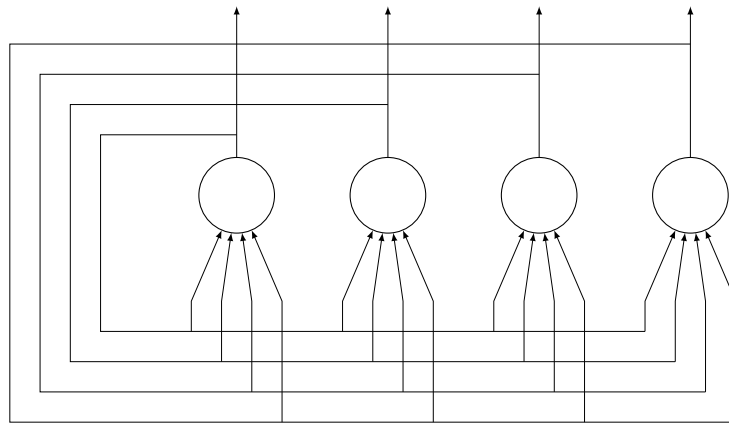


Figura 7.5: rete neurale di Hopfield

In questo caso viene realizzato un grafo completamente connesso, in cui è facile notare che tutte le uscite sono riportate in ingresso a tutti i neuroni realizzando un **feed-back** completo fra i neuroni.

Le capacità di approssimazione di una rete di questo tipo sono molto più ampie ma se da un lato l'introduzione dei loop algebrici aumenta la potenza della rete, dall'altro si complicano notevolmente le problematiche connesse ad essa: gli algoritmi di apprendimento sono notevolmente più complicati di quelli impiegati per le MLP e può capitare di incorrere in fenomeni di instabilità e comportamenti caotici.

7.2.3 Rappresentazione di una rete neurale artificiale

Gli schemi e i simboli di seguito riportati indicano la notazione delle reti neurali impiegata in ambiente MATLAB® (*Neural Network Toolbox*) questa notazione permette di chiarire con maggior precisione il ruolo di tutte le variabili coinvolte in una rete neurale, nonché tutti i legami, logici e matematici, che ne caratterizzano il funzionamento.

Il neurone artificiale è illustrato in figura 7.6 nelle due varianti che differiscono per la quantità di ingressi del neurone.

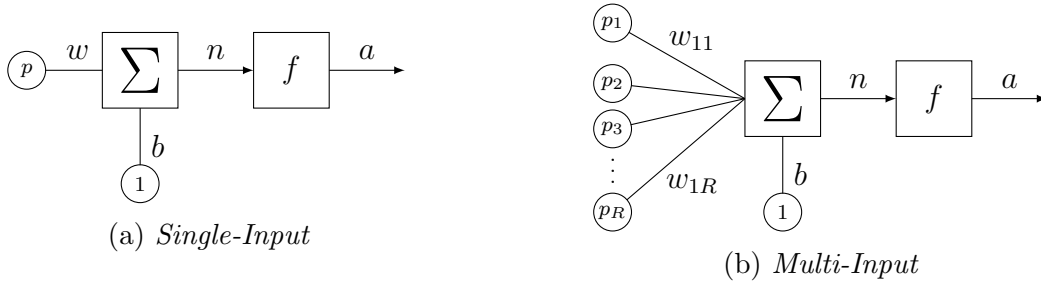


Figura 7.6: Neurone artificiale

La funzione matematica f implementata nel neurone è inserita in un apposito blocco. La variabile in ingresso n alla funzione f non è un segnale elementare ma una variabile ottenuta secondo la seguente relazione:

$$n = w \cdot p + b \quad (7.7)$$

dove b è il bias, p l'ingresso vero e proprio, w il peso sinaptico associato all'ingresso p . L'uscita a è data da:

$$a = f(n) = f(wp + b) \quad (7.8)$$

Nel caso più generale *Multi-Input*, vi è un vettore di R ingressi (e quindi con R pesi sinaptici associati) e l'ingresso della funzione f viene calcolato come combinazione lineare degli ingressi più il bias:

$$n = \left(\sum_{i=1}^R p_i \cdot w_{1i} \right) + b \quad (7.9)$$

L'uscita scalare a è data da:

$$a = f(n) = f(\mathbf{W}\mathbf{p} + b) \quad (7.10)$$

dove, in questo caso, $\mathbf{W}$ è una **matrice dei pesi**, $\mathbf{p}$ è il **vettore degli ingressi** e b è il **bias**.

La funzione che implementa il neurone è detta **funzione di attivazione** ed è scelta in base all'applicazione a cui è destinata la rete neuronale: se ad esempio si vuole impiegare una ANN per applicazioni di classificazione o pattern recognition, conviene impiegare funzioni discontinue, le cui uscite appartengono a dominio discreto, numerabile. Se invece si vuole progettare una ANN per approssimazione di funzioni continue, conviene impiegare funzioni di attivazione continue e, generalmente, di classe C^∞ (ovvero continue e derivabili infinite volte). Una funzione di questo tipo è la **sigmoide**, una funzione con andamento simile a quello di un gradino ma con una transizione morbida. L'espressione di una sigmoide è del tipo:

$$y = \frac{1}{1 + e^{-n}} \quad (7.11)$$

Le funzioni impiegate, nella pratica comune sono il gradino, la sigmoide e la funzione lineare:

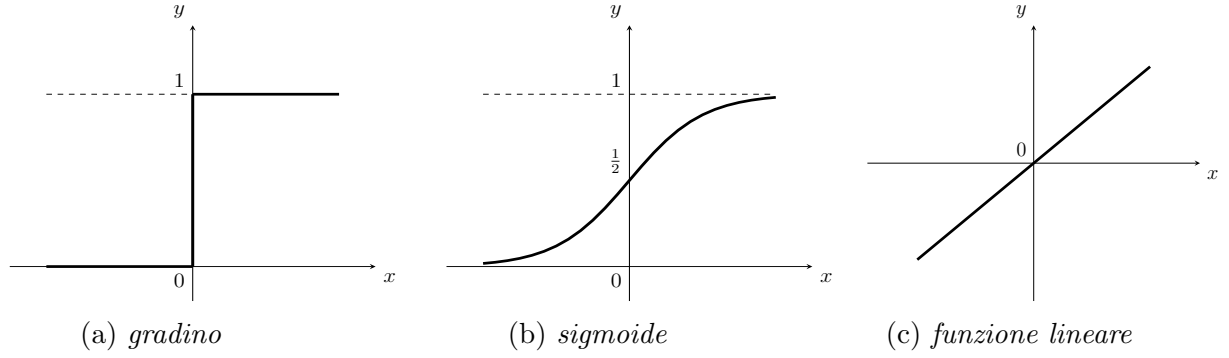


Figura 7.7: Funzioni di attivazione

Un'espressione matematica del gradino secondo McCulloch e Pitts è il **gradino a soglia** θ :

$$y = \text{sign} \left(\sum_{i=1}^n w_i x_i - \theta \right) \quad (7.12)$$

questa funzione si basa sul segno della differenza fra una somma pesata di ingressi e una soglia θ . Quando si supera tale soglia la differenza cambia di segno e l'uscita y può valere -1 o 1 (ma con opportune operazioni di traslazione e scaling dell'uscita si può riportare quest'ultima ai valori 0 e 1).

Lo strato di una rete di neuroni è ottenuto aggregando più neuroni in parallelo (figura 7.8).

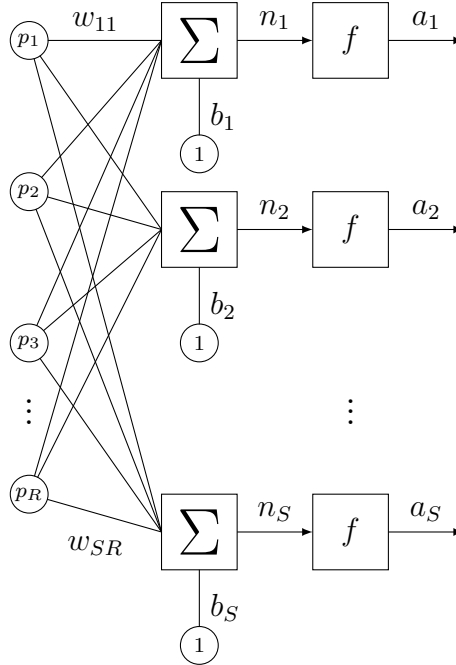


Figura 7.8: Strato di una rete neurale

La relazione ingresso-uscita dello strato è, in questo caso, totalmente priva di variabili scalari:

$$\mathbf{a} = \mathbf{f}(\mathbf{W}\mathbf{p} + \mathbf{b}) \quad (7.13)$$

Il vettore $\mathbf{p}$ è il vettore degli R ingressi e $\mathbf{a}$ è il vettore delle S uscite. $\mathbf{f}$ è un vettore avente S funzioni (in genere dello stesso tipo), W è la **matrice dei pesi** e $\mathbf{b}$ il **vettore dei bias**. Una più compatta e conveniente rappresentazione (usata, d'ora in avanti) della stessa topologia è illustrata in figura 7.9:

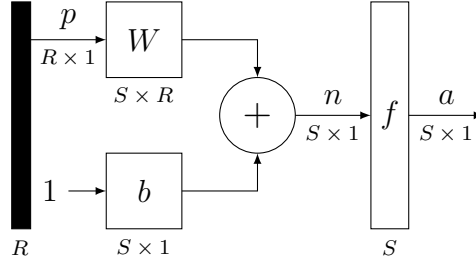


Figura 7.9: Strato di una rete neurale

Aggregando, in serie, più strati si ottiene la rete neurale con architettura MLP:

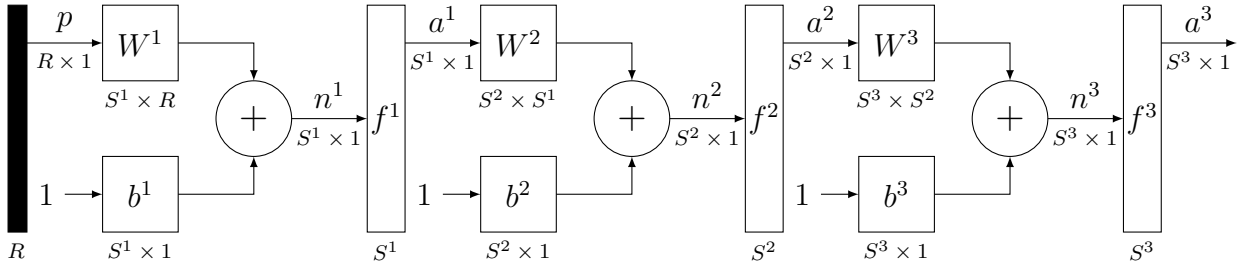


Figura 7.10: Rete neurale con architettura MLP

la cui equazione è:

$$\mathbf{a}^3 = \mathbf{f}^3(\mathbf{W}^3 \mathbf{f}^2(\mathbf{W}^2 \mathbf{f}^1(\mathbf{W}^1 \mathbf{p} + \mathbf{b}^1) + \mathbf{b}^2) + \mathbf{b}^3) \quad (7.14)$$

ottenuta componendo le equazioni per ciascuno strato:

$$\mathbf{a}^1 = \mathbf{f}^1(\mathbf{W}^1 \mathbf{p} + \mathbf{b}^1) \quad (7.15)$$

$$\mathbf{a}^2 = \mathbf{f}^2(\mathbf{W}^2 \mathbf{a}^1 + \mathbf{b}^2) \quad (7.16)$$

$$\mathbf{a}^3 = \mathbf{f}^3(\mathbf{W}^3 \mathbf{a}^2 + \mathbf{b}^3) \quad (7.17)$$

Il seguente esempio, illustrato da Hagan, mostra gli effetti sull'uscita delle variazioni dei parametri di una rete neurale.

La rete neurale in questione è composta da un solo strato nascosto avente due neuroni con funzioni di attivazione sigmoidali, mentre lo strato di uscita prevede un solo neurone con funzione di attivazione lineare, la quale combina linearmente le uscite sigmoidali dello strato nascosto (figura 7.11).

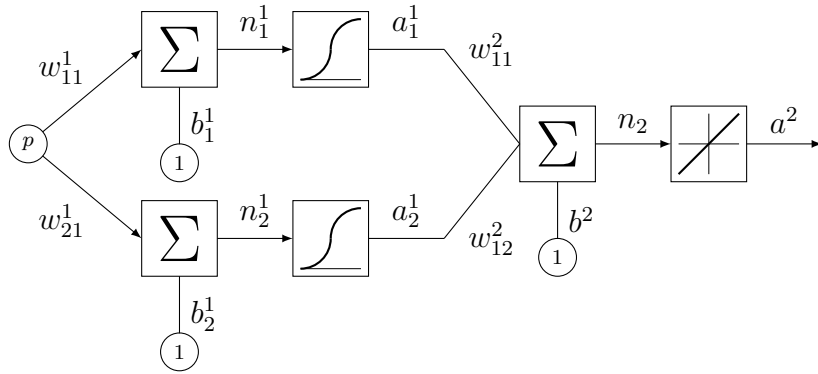


Figura 7.11: Esempio di Hagan

Le funzioni di attivazione sigmoidali dello strato nascosto hanno espressione:

$$f^1(n) = \frac{1}{1 + e^{-n}} \quad (7.18)$$

e la funzione di attivazione dello strato di uscita ha espressione:

$$f^2(n) = n \quad (7.19)$$

L'andamento dell'uscita dovuto ai due contributi sigmoidali combinati linearmente è rappresentato nella figura seguente:

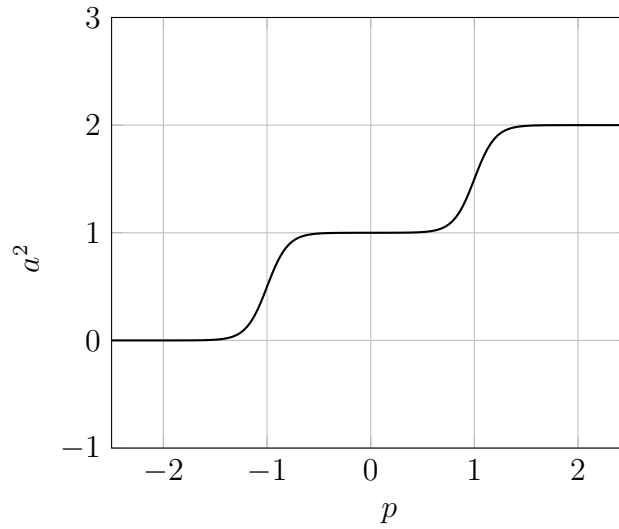


Figura 7.12: Andamento dell'uscita in condizioni normali

La figura 7.13 mostra come una variazione del **bias** di una delle due sigmoids, come ad esempio b_1^1 , comporti una **traslazione** corrispondente (naturalmente, la traslazione riguarderà entrambe le sigmoids se variano entrambi i bias).

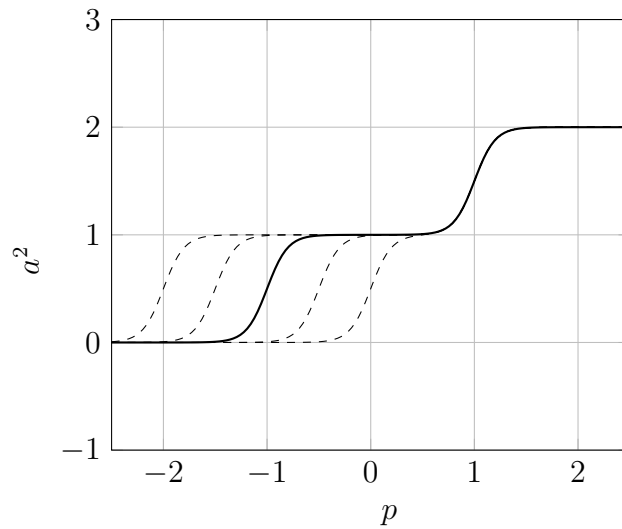


Figura 7.13: Effetto della variazione di uno dei bias dei neuroni dello strato nascosto

Nel caso in cui a variare siano i **pesi degli ingressi** dello strato nascosto, ad esempio w_{21}^1 (figura 7.14) si osserva una **variazione dell'ampiezza** della sigmoide corrispondente.

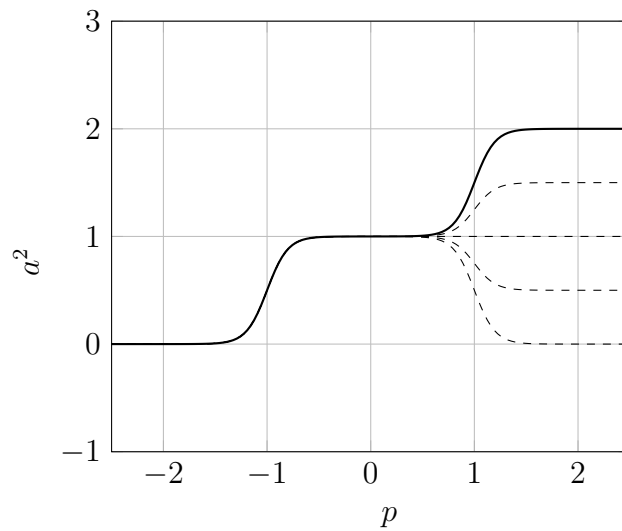


Figura 7.14: Effetto della variazione di uno dei pesi degli ingressi dello strato nascosto

Se invece si modificano i pesi delle uscite dello strato nascosto (ovvero i pesi degli ingressi dello strato di uscita) come ad esempio w_{11}^2 il comportamento dell'uscita sarà del tipo seguente:

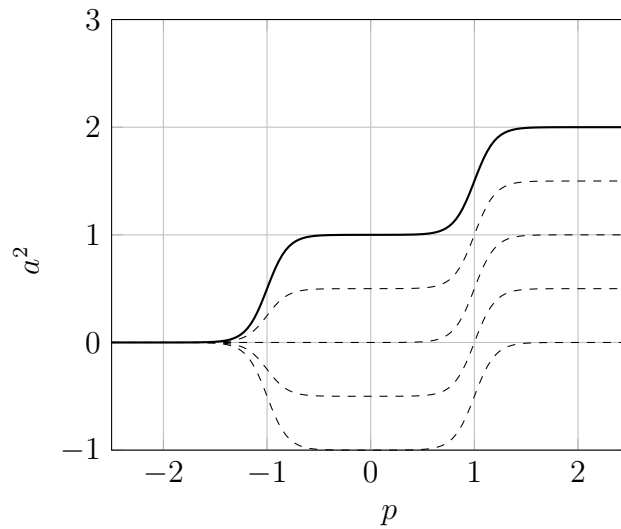


Figura 7.15: Effetto della variazione di uno dei pesi degli ingressi dello strato di uscita

La figura 7.16 mostra infine la possibilità di spostare verticalmente le sigmoidi modificando il valore del bias b^2 del neurone di uscita (la cui funzione di attivazione è lineare):

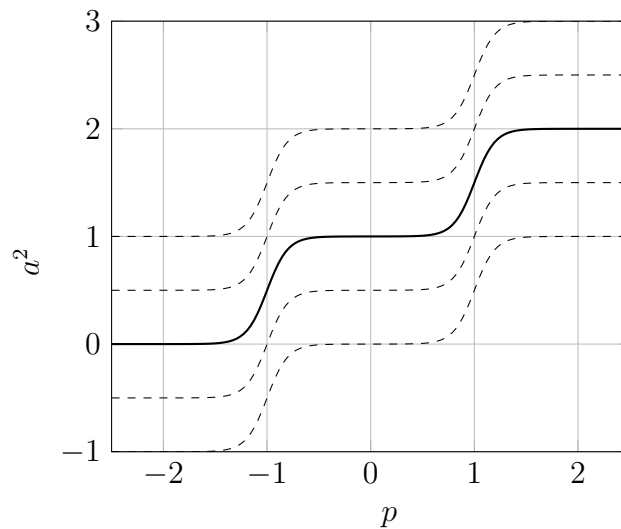


Figura 7.16: Effetto della variazione del bias del neurone dello strato di uscita

7.2.4 Retropropagazione

Quando un organismo biologico apprende, memorizza e impara a classificare, il processo avviene biologicamente tramite l'ispessimento delle sinapsi: vi è un aumento di un canale di comunicazione piuttosto che di un altro, favorendo determinati percorsi di segnale, in luogo di altri.

Alcune sinapsi diventano più attive di altre, permettendo una più rapida esecuzione del percorso elettrochimico del segnale.

I meccanismi con cui avvengono le variazioni dei pesi sinaptici permettono il **training** della rete neurale.

Addestrare una rete neurale significa dunque trovare tutti i pesi per tutte le interconnessioni neuronali, inclusi i bias. Questi parametri possono essere successivamente migliorati e raffinati operando successive iterazioni fino al soddisfacimento di un certo criterio di arresto.

Le reti neurali artificiali devono dunque essere addestrate secondo specifiche metodologie e criteri: uno tra i più comuni per le reti MLP è l'**algoritmo di retropropagazione** (*Backpropagation*) dell'errore, una tecnica di ottimizzazione del primo ordine che impiega il concetto della discesa del gradiente.

Si considera la generica rete neurale con architettura MLP, con n ingressi, uno strato nascosto composto da m neuroni aventi funzioni di attivazione sigmoidali e uno strato di uscita composto da l neuroni aventi funzioni di attivazione lineari.

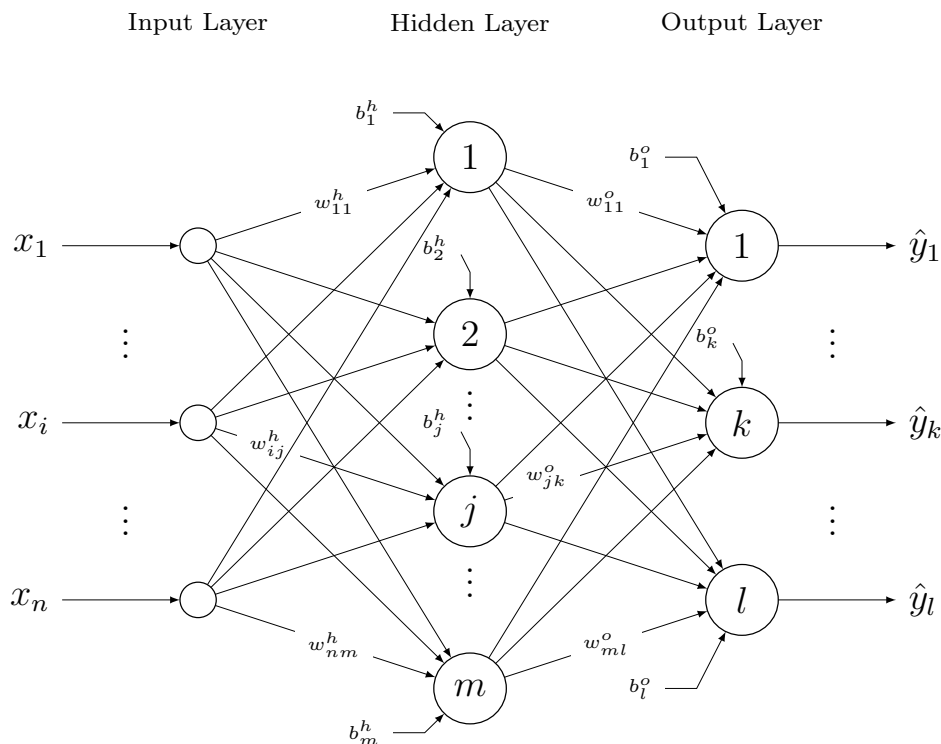


Figura 7.17: Rete neurale con architettura MLP

Il termine w_{ij}^h indica il peso sinaptico relativo alla connessione fra l' i -esimo ingresso e il j -esimo neurone. L'apice indica lo strato di destinazione (h indica *hidden*, o indica *output*). Ad esempio, il termine w_{12}^o indica il valore del peso sinaptico fra il primo neurone (primo pedice, 1) dello strato nascosto e il secondo neurone (secondo pedice, 2) dello strato di uscita (apice o).

L'uscita dei neuroni nel j-esimo strato nascosto è:

$$h_j = \sigma \left(\sum_{i=1}^n w_{ij}^h x_i + b_j^h \right) = \sigma \left(\sum_{i=0}^n w_{ij}^h x_i \right) = \tanh \left(\sum_{i=0}^n w_{ij}^h x_i \right) \quad (7.20)$$

dove l'operatore σ indica il tipo di funzione sigmoidale impiegata come funzione di attivazione per ciascuno dei neuroni dello strato nascosto; in questo caso si è scelta la tangente iperbolica, la cui espressione è:

$$\tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

Retropropagazione sullo strato di uscita

L'uscita dei neuroni nello strato di uscita (figura 7.18) è:

$$\hat{y}_k = \ell \left(\sum_{j=1}^m w_{jk}^o h_j + b_j^o \right) = \ell \left(\sum_{j=0}^m w_{jk}^o h_j \right) = \sum_{j=0}^m w_{jk}^o h_j \quad (7.21)$$

dove l'operatore ℓ indica il tipo di funzione lineare impiegata come funzione di attivazione per ciascuno dei neuroni dello strato di uscita; in questo caso si è scelta la bisettrice $y = x$.

L'algoritmo di retropropagazione segue il classico approccio orientato alla minimizzazione dell'errore di predizione (sulle l uscite):

$$\epsilon_k = y_k - \hat{y}_k \quad k = 1, \dots, l \quad (7.22)$$

In questo caso l'errore di predizione è calcolato come doppia sommatoria, ossia come la somma su tutte le l uscite delle somme dei quadrati degli errori di predizione su tutti gli N dati sperimentali:

$$\mathcal{J}(w) = \frac{1}{2} \sum_{k=1}^l \sum_{q=1}^N \epsilon_{kq}^2 \quad (7.23)$$

dove w è il vettore di parametri che minimizza $\mathcal{J}$.

L'apprendimento consiste nell'aggiornare i pesi seguendo un approccio di minimizzazione dell'errore lungo più epoche.

Un'epoca, indicata con la lettera p , è l'insieme di iterazioni necessarie ad effettuare un solo aggiornamento per tutti i pesi della rete neurale.

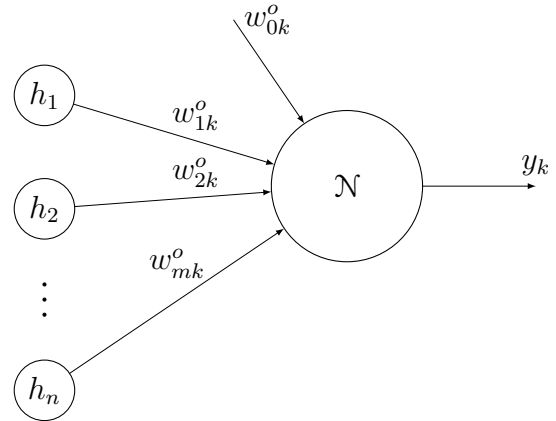


Figura 7.18: Neurone di uscita

Si parte dunque considerando l'errore sull'uscita e quindi i pesi sinaptici più prossimi all'uscita. Per il generico peso associato al k-esimo neurone dello strato di uscita si ha che:

$$w_{jk}^o(p+1) = w_{jk}^o(p) - \alpha [\nabla \mathcal{J}(w_{jk}^o)]_j \quad (7.24)$$

dove $\nabla \mathcal{J}(w_{jk}^o)$ è il gradiente di $\mathcal{J}$ rispetto al peso, calcolato come segue:

$$\nabla \mathcal{J}(w_{jk}^o) = \left(\frac{\partial \mathcal{J}}{\partial w_{1k}^o}, \frac{\partial \mathcal{J}}{\partial w_{2k}^o}, \dots, \frac{\partial \mathcal{J}}{\partial w_{mk}^o} \right)^T \quad (7.25)$$

L'equazione 7.24 è un'implementazione del metodo a discesa del gradiente: essa impone che il peso sinaptico che collega il j-esimo neurone dello strato nascosto al k-esimo neurone dello strato di uscita all'iterazione $(p+1)$ -esima venga calcolato come la differenza fra lo stesso peso all'iterazione p-esima precedente e il gradiente di $\mathcal{J}$ rispetto al peso stesso pesata per un opportuno learning rate α . In termini pratici, il peso viene aggiornato aggiungendo al valore che aveva precedentemente il gradiente cambiato di segno scalato per il learning rate.

Applicando la regola della catena si ha:

$$\frac{\partial \mathcal{J}}{\partial w_{jk}^o} = \frac{\partial \mathcal{J}}{\partial \epsilon_k} \cdot \frac{\partial \epsilon_k}{\partial \hat{y}_k} \cdot \frac{\partial \hat{y}_k}{\partial w_{jk}^o} \quad (7.26)$$

Poiché:

$$\mathcal{J}(w_{jk}^o, w_{ij}^h) = \frac{1}{2} \sum_{k=1}^l \epsilon_k^2 \quad , \quad \epsilon_k = y_k - \hat{y}_k \quad , \quad \hat{y}_k = \sum_{j=0}^m w_{jk}^o h_j \quad (7.27)$$

le derivate valgono:

$$\frac{\partial \mathcal{J}}{\partial \epsilon_k} = \epsilon_k \quad , \quad \frac{\partial \epsilon_k}{\partial \hat{y}_k} = -1 \quad , \quad \frac{\partial \hat{y}_k}{\partial w_{jk}^o} = h_j \quad (7.28)$$

sostituendo in 7.26 si ha che:

$$\frac{\partial \mathcal{J}}{\partial w_{jk}^o} = -h_j \epsilon_k \quad (7.29)$$

La derivata è pari al prodotto tra il valore dell'**uscita del j-esimo neurone dello strato nascosto** e l'**errore di predizione della k-esima uscita**. Si può dunque riscrivere la 7.24 come segue:

$$w_{jk}^o(p+1) = w_{jk}^o(p) + \alpha h_j \epsilon_k \quad (7.30)$$

Questo è un risultato importante in quanto vale per tutti i neuroni dello strato di uscita (la cui struttura è mostrata in figura 7.18) e permette di calcolare la derivata senza di fatto effettuare operazioni di derivazione (equazione 7.30).

I parametri da cui dipende il gradiente richiesto sono infatti noti dall'iterazione precedente e quindi sono già presenti in memoria.

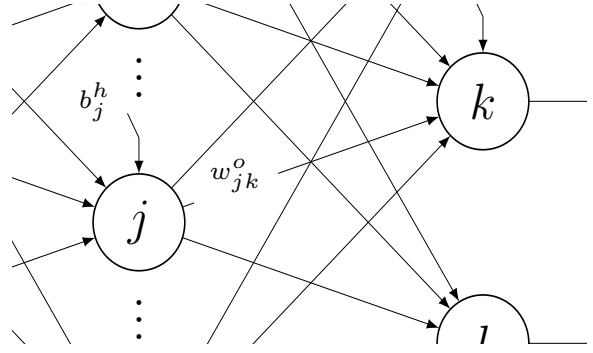


Figura 7.19: Apprendimento mediante retro-propagazione: strato di uscita

Retropropagazione sullo strato nascosto

Per quanto riguarda i neuroni dello strato nascosto vale un ragionamento del tutto analogo a quello fatto per lo strato di uscita.

L'uscita del generico neurone dello strato nascosto è:

$$h_j = \tanh(net_j) \quad (7.31)$$

dove net_j è l'argomento della funzione di attivazione (che in questo caso è una tangente iperbolica) del neurone nascosto ovvero una variabile intermedia ottenuta come somma di tutti gli ingressi pesati e il bias.

$$net_j = \sum_{i=0}^n w_{ij}^h x_i \quad (7.32)$$

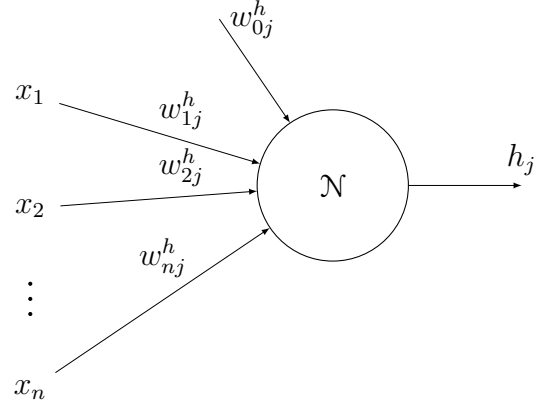


Figura 7.20: Neurone dello strato nascosto

si applica ancora l'equazione:

$$w_{ij}^h(p+1) = w_{ij}^h(p) - \alpha \nabla \mathcal{J}(w_{ij}^h) \quad (7.33)$$

e per la regola della catena si ha:

$$\frac{\partial \mathcal{J}}{\partial w_{ij}^h} = \frac{\partial \mathcal{J}}{\partial h_j} \cdot \frac{\partial h_j}{\partial net_j} \cdot \frac{\partial net_j}{\partial w_{ij}^h} \quad (7.34)$$

le derivate valgono:

$$\begin{aligned} \frac{\partial \mathcal{J}}{\partial h_j} &= \frac{\partial}{\partial h_j} \left[\frac{1}{2} \sum_{k=1}^l \epsilon_k^2 \right] = \frac{\partial}{\partial h_j} \left[\frac{1}{2} \sum_{k=1}^l (y_k - \hat{y}_k)^2 \right] = \frac{\partial}{\partial h_j} \left[\frac{1}{2} \sum_{k=1}^l \left(y_k - \sum_{j=0}^m w_{jk}^o h_j \right)^2 \right] = \\ &= \frac{1}{2} \sum_{k=1}^l -2w_{jk}^o \left(y_k - \sum_{j=0}^m w_{jk}^o h_j \right) = \sum_{k=1}^l (-\epsilon_k w_{jk}^o) \end{aligned} \quad (7.35)$$

$$\frac{\partial h_j}{\partial net_j} = \sigma'_j(net_j) \quad , \quad \frac{\partial net_j}{\partial w_{ij}^h} = x_i \quad (7.36)$$

e quindi si ottiene:

$$\frac{\partial \mathcal{J}}{\partial w_{ij}^h} = x_i \cdot \sigma'_j(net_j) \cdot \sum_{k=1}^l (-\epsilon_k w_{jk}^o) \quad (7.37)$$

Quindi anche in questo caso l'operazione di derivazione richiesta per la discesa del gradiente non comporta calcoli differenziali significativi, fatta eccezione per il calcolo della derivata della tangente iperbolica (e ciò accade perché le funzioni di attivazione trattate non sono lineari ma sigmoidi).

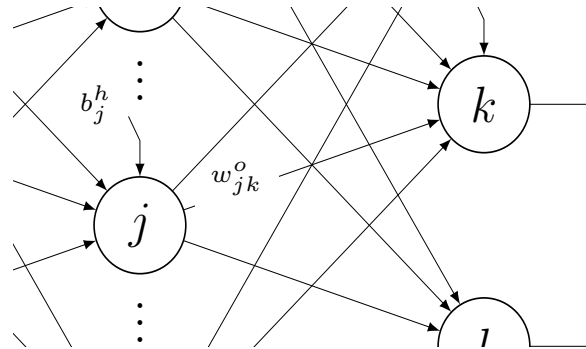
Quanto detto vale anche in presenza di più strati nascosti: la retropropagazione si applica in maniera del tutto analoga, con un numero maggiore di iterazioni da compiere per risalire a ritroso tutti gli strati, fino a quello di ingresso.

Significato pratico della retropropagazione

L'equazione 7.35 racchiude e ben sintetizza il significato del concetto di retropropagazione. Si riconsideri la figura 7.19, riportata in basso per immediatezza:

Se c'è un errore non nullo² ϵ_k sul neurone k -esimo dello strato di uscita, l'algoritmo deve aggiornare i pesi. Il concetto di retropropagazione si fonda sull'attribuire una maggiore "responsabilità", circa l'errore commesso, ai neuroni con il maggior peso sinaptico sul calcolo della grandezza in uscita dal neurone.

In termini ancor più intuitivi, per l'errore su un dato neurone l'algoritmo dà una colpa maggiore al neurone con il peso maggiore e quindi l'intervento correttivo su quel neurone sarà più marcato.



Strategie di apprendimento

Vi sono due diverse strategie con cui addestrare una rete neurale, a seconda di come sono gestiti gli N dati in ingresso.

- **Apprendimento *Batch*:** tutti gli N campioni sono considerati prima di effettuare l'aggiornamento dei pesi. Viene calcolato il valor medio del gradiente per ciascuno degli N dati prima di applicare la variazione dei pesi. Questo approccio è il più efficace nel caso in cui si dispone a priori dell'intero set di dati, viceversa è inapplicabile.
- **Apprendimento incrementale:** in contesti in cui non si disponga dell'intero set di campioni ma questi ultimi sono disponibili a una frequenza relativamente bassa si può scegliere di tarare la rete neurale ogni volta che arriva un nuovo dato in ingresso. Questo approccio è utilizzato quando l'apprendimento batch non è perseguibile o se i dati sperimentali sono forniti in tempo reale.

L'algoritmo di retropropagazione

Il training di una rete neurale mediante retropropagazione è un processo algoritmico sintetizzabile nei seguenti passi.

- **Inizializzazione:** tutti i pesi della rete vengono inizializzati con piccoli numeri casuali. La ragione per cui si scelgono numeri piccoli è dovuta al fatto che le funzioni di attivazione dei neuroni possono saturare (comportamento tipico della sigmoide),

²a un errore nullo corrisponderebbe una predizione esatta e non ci sarebbe alcun peso da aggiornare

di conseguenza la rete neurale potrebbe impiegare un ampio numero di epoche per tarare correttamente i pesi riportandoli nella zona non saturata³.

- **Discesa del gradiente**

1. Inserimento dei dati di training e calcolo delle uscite della rete neurale;
2. Discesa del gradiente sui neuroni dello strato di uscita:

$$w_{jk}^o(p+1) = w_{jk}^o(p) - \alpha(-h_j \epsilon_k) \quad (7.38)$$

In alternativa si può utilizzare una tecnica del secondo ordine⁴ e stimare la matrice Hessiana $\mathbf{H}(\mathbf{w})$ sostituendola al learning rate:

$$\mathbf{w}(p+1) = \mathbf{w}(p) - \mathbf{H}(\mathbf{w}(p)) \cdot \nabla \mathcal{J}(\mathbf{w}(p)) \quad (7.39)$$

3. Discesa del gradiente sui neuroni degli strati nascosti:

$$w_{ij}^h(p+1) = w_{ij}^h(p) - \alpha \cdot x_i \cdot \sigma'_j(\text{net}_j) \cdot \sum_{k=0}^l (-\epsilon_k w_{jk}^o) \quad (7.40)$$

- **Criterio di arresto:** le iterazioni proseguono fino al soddisfacimento di un determinato criterio di arresto. Di seguito ve ne sono alcuni.

- La maniera più semplice di scegliere un criterio di arresto è fissare un numero di epoche entro cui far addestrare la rete o alternativamente imporre un tempo limite di elaborazione. Questi metodi non sono in grado di valutare la performance della rete neurale e di solito non vengono utilizzati.
- Poiché i gradienti tendono a zero, si può stabilire l'arresto dell'algoritmo al momento in cui i gradienti raggiungono un certo intorno dello zero.
- Si può calcolare una norma dei pesi e successivamente la differenza fra le norme lungo diverse iterazioni: se la norma risultante è sufficientemente contenuta allora i gradienti saranno mediamente contenuti quindi si può arrestare l'algoritmo.
- Si può sfruttare il meccanismo dell'overtraining: quando il fitting fra la curva del MGD e quella dello stimatore aumenta fino a che l'andamento di $\mathcal{J}$ raggiunge un minimo, si verifica il fenomeno dell'ordine ottimo. Addestrare la rete oltre questo limite significa sovra-addestrare la rete stessa e, conseguentemente, perdere l'aderenza massima fra la curva del MGD e la curva dello stimatore. Si può imporre l'arresto dell'algoritmo nel momento in cui si rileva un minimo nell'andamento di $\mathcal{J}$ quindi appena prima di incorrere nell'overtraining.

³Questo effetto indesiderato è analogo al fenomeno di wind-up nei controlli automatici, dovuto alla presenza di integratori. Inizializzare i pesi della rete neurale con piccoli numeri è un modo per prevenire la saturazione delle funzioni di attivazione ed equivale a scaricare gli integratori in un equivalente sistema di controllo affetto da wind-up.

⁴una tecnica del secondo ordine molto efficace, impiegata di default in ambiente MATLAB®, è l'algoritmo di Levenberg-Marquardt.

7.2.5 Reti neurali a base radiale

Le **reti a base radiale** sono particolari reti neurali con una struttura monostrato, ovvero aventi un solo strato nascosto le cui funzioni di attivazione hanno base radiale.

Una **funzione a base radiale** (*RBF*) è una funzione di una o più variabili reali $\phi(\mathbf{x}, \mathbf{c}) = \phi(\|\mathbf{x} - \mathbf{c}\|)$ i cui valori dipendono solamente dalla distanza dall'origine o, equivalentemente da un centro c specificato. Una qualunque funzione ϕ che verifica la proprietà:

$$\phi(\mathbf{x}) = \phi(\|\mathbf{x}\|) \quad (7.41)$$

è una funzione a base radiale. La norma è usualmente definita come distanza euclidea, ma è anche possibile definire altri tipi di norma.

La **gaussiana** è un tipo di funzione a base radiale ed è impiegata come funzione di attivazione dell' i -esimo neurone per le reti neurali di questo tipo:

$$\sigma_i(x) = e^{-\frac{\|x - c_i\|^2}{s_i^2}} \quad (7.42)$$

dove x rappresenta il vettore degli ingressi, c_i è un vettore contenente i centri delle gaussiane e s_i è il vettore che contiene le varianze di ciascuna gaussiana.

La figura 7.21 illustra una rete neurale a base radiale a più uscite.

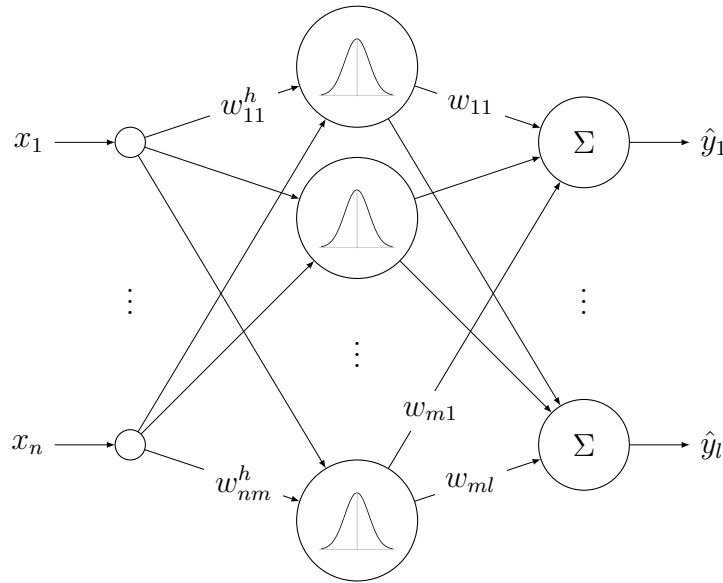


Figura 7.21: Rete neurale artificiale a base radiale con n ingressi ed l uscite.

Assumendo che la rete sia un sistema SISO, soffermandosi dunque su un singolo ingresso e una singola uscita, l'uscita della rete è data da:

$$y_j = \sum_{i=1}^n w_{ij} \sigma_i(x) = \sum_{i=1}^n w_{ij} e^{-\frac{\|x - c_i\|^2}{s_i^2}} \quad (7.43)$$

Se invece le gaussiane sono definite in $\mathbb{R}^2$ allora le curve di livello delle gaussiane sono circonferenze (o ellissi in casi particolari). In tali circostanze lo spazio degli ingressi è il

piano $\mathbb{R}^2$ e i domini delle funzioni di attivazione dei neuroni sono distribuiti secondo specifici criteri. La figura 7.22 mostra una possibile configurazione dei domini delle gaussiane impiegate come funzioni di attivazione per lo strato nascosto.

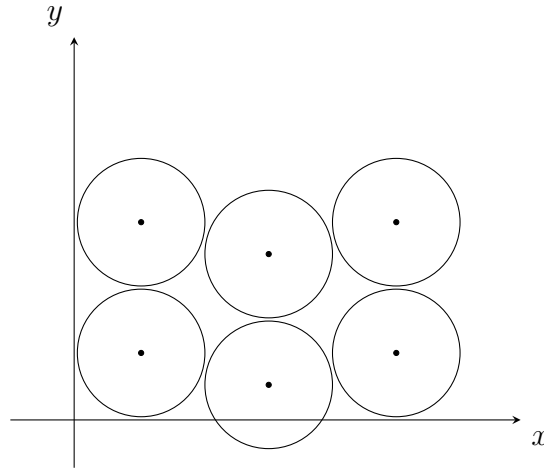


Figura 7.22: Distribuzione dei domini delle funzioni di attivazione a base radiale su uno spazio degli ingressi sottoinsieme di $\mathbb{R}^2$.

Ciascun ingresso “attraversa” una campana e il valore dell’uscita è la somma pesata delle gaussiane valutate su tutti gli ingressi: quanto più vicino al centro della gaussiana è l’ingresso, maggiormente sarà attivato il neurone corrispondente; viceversa, se l’ingresso si allontana dal centro la sua uscita è molto prossima a zero e il neurone corrispondente viene scarsamente attivato.

Il pregio principale delle reti neurali a base radiale è la **linearità nei parametri**, pertanto vale l’applicazione dell’algoritmo dei minimi quadrati, il quale converge in un solo passo.

L’uscita è una combinazione lineare di funzioni note solamente se si fissano i centri e le varianze, in caso contrario il problema di identificazione diventa non lineare.

La disposizione delle gaussiane nello spazio degli ingressi è un aspetto importante e la scelta di un criterio con cui distribuire le funzioni di attivazione di ciascun neurone dipende dalla conformazione della funzione da approssimare.

Ciascuna delle funzioni a base radiale rappresenta il centro di un modello locale della funzione da minimizzare. Se la funzione da approssimare esibisce variazioni significative solamente in una certa regione dello spazio degli ingressi è opportuno condensare più gaussiane (o più in generale, più modelli locali) laddove vi sono maggiori variazioni. In regioni di minor rilevanza si possono usare meno gaussiane, con dispersioni relativamente più ampie (figure 7.23a e 7.23b).

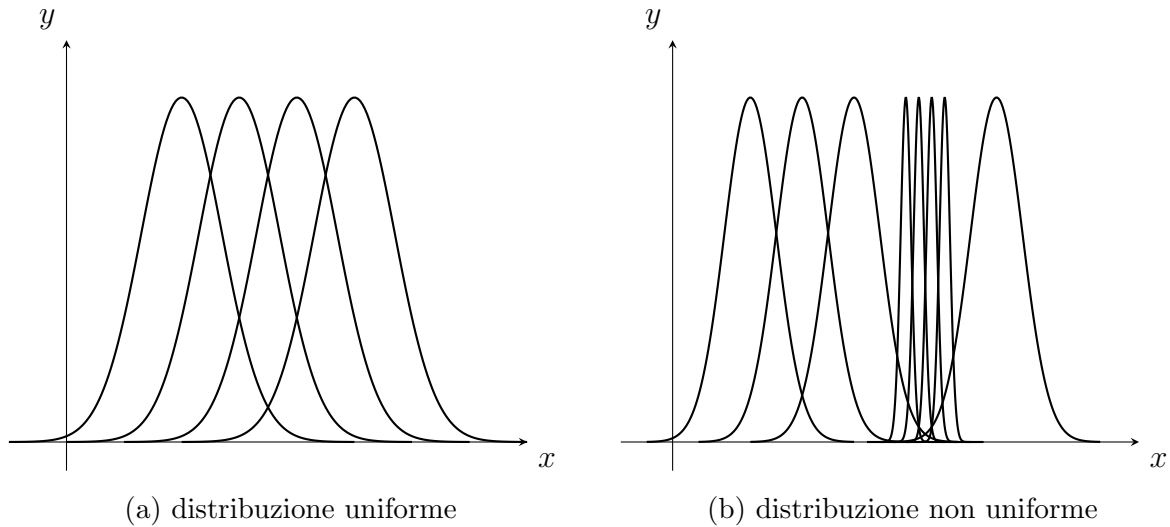


Figura 7.23: Distribuzioni di gaussiane in uno spazio degli ingressi monodimensionale.

Ciò considerato, è possibile procedere sostanzialmente in due modi diversi:

1. si scelgono centri e dispersioni per ciascuna gaussiana in modo da coprire uniformemente lo spazio degli ingressi. Il problema diventa allora LIP e ciò è un vantaggio perché il metodo dei minimi quadrati converge in un solo passo iterativo. Lo svantaggio è il fatto che può ottimizzare solo i parametri che hanno influenza lineare sull'uscita (e quindi non i centri e le dispersioni delle gaussiane).
2. si includono nella ricerca anche i centri e le dispersioni per ciascuna gaussiana: ciò comporta l'impiego di algoritmi molto più complessi per l'ottimizzazione della rete in quanto il problema di identificazione non è più lineare.

Esistono anche funzioni a base radiale **semi-gaussiane** (figura 7.24), le quali presentano un'asimmetria caratterizzata da più varianze. Nel caso di funzioni semi-gaussiane definite in $\mathbb{R}^2$ le curve di livello sono ellissi (e non più circonferenze come nel caso di gaussiane caratterizzate da una sola varianza).

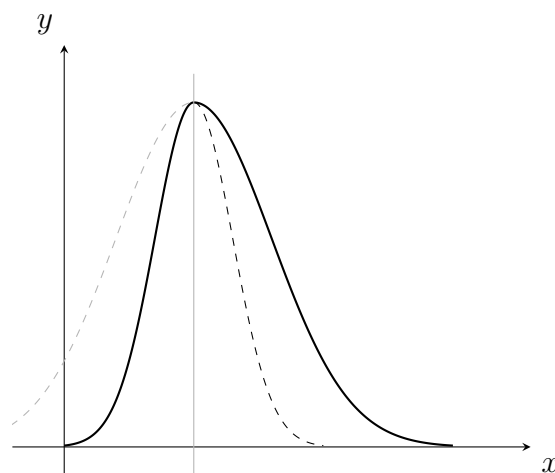


Figura 7.24: Funzione semi-gaussiana

7.2.6 Considerazioni conclusive

Tra i vantaggi principali delle reti neurali vi è l'adeguatezza a problemi che presentano alta dimensionalità ma vi è anche uno svantaggio: le reti neurali artificiali sono **approssimatori black-box**. La logica con cui si giunge al risultato è sconosciuta, in quanto l'esito dell'approssimazione è il prodotto di un processo basato sull'aggregazione di funzioni matematiche elementari il cui ampio numero rende praticamente impossibile comprendere in che modo si è giunti alla soluzione.

Per questa ragione le reti neurali sono applicate a problemi per i quali non è importante conoscere le dinamiche interne che governano la logica di funzionamento dell'intero sistema; a emergere è solamente il suo funzionamento "ai morsetti". Le reti neurali sono usate per la modellazione e la predizione di sistemi non lineari, classificazione e *pattern recognition*.

Questa categoria di approssimatori riveste, nonostante i suoi svantaggi, un'importanza centrale poiché vi sono due risultati matematici (sulle reti feed-forward) che mettono in particolare risalto l'efficacia delle reti neurali rispetto ad altri approssimatori.

Un teorema pubblicato da **G. Cybenko** nel 1989 afferma il seguente

Teorema 1. *Una rete neurale feed-forward con almeno uno strato nascosto e un numero sufficiente di neuroni al suo interno può approssimare una qualsiasi funzione $\mathbb{R}^p \mapsto \mathbb{R}^n$ definita su un intervallo compatto.*

Pochi anni più tardi, nel 1993, **A. Barron** dimostra il

Teorema 2. *Una rete neurale feed-forward con uno strato nascosto e neuroni nascosti aventi funzioni di attivazione sigmoidali può raggiungere un errore quadratico integrale dell'ordine $J = \mathcal{O}(1/h)$, dove h è il numero di neuroni nascosti.*

Come immediato corollario dei due teoremi sopracitati, si può dire che una rete neurale opportunamente progettata è in grado di approssimare una qualunque funzione continua, indipendentemente dalla dimensione p dello spazio degli ingressi, raggiungendo una precisione il cui grado dipende **solamente** dal numero h di neuroni nascosti.

Questa è una proprietà del tutto singolare, che non si verifica con altre categorie di approssimatori, affetti dalla *Curse Of Dimensionality*.

I metodi polinomiali sono approssimatori che determinano l'uscita come combinazione lineare di funzioni di base note. Per questi approssimatori, l'ordine dell'errore di approssimazione è $J = \mathcal{O}(1/h^{2/p})$, dove h è il numero di funzioni di base e p è la dimensione dell'ingresso. Man mano che la dimensione p aumenta l'errore di approssimazione utilizzando approssimatori polinomiali aumenta, mentre quello associato alle reti neurali rimane uguale e dipende appunto unicamente dal numero di neuroni nascosti.

Apprendimento supervisionato e non supervisionato

L'approccio più classico alla ricerca dei modelli è basato su **apprendimento supervisionato** ovvero sull'addestramento di modelli (in genere lineari) confrontando l'uscita del modello con l'uscita del MGD. Queste tecniche sono dunque basate sulla valutazione dell'errore di predizione, per le quali esiste sempre un'informazione sull'uscita del MGD ad ogni istante di tempo.

Le reti neurali sono spesso applicate in contesti nei quali si adoperano tecniche di **apprendimento non supervisionato**: i parametri del modello non sono più determinati confrontando l'uscita del modello con l'uscita del MGD ma vengono modificati utilizzando altri criteri. Un esempio può essere la modellazione della dinamica inversa di sistema di controllo: si potrebbe cercare una G^{-1} ponendovi una G in cascata e valutare la vicinanza del risultato a 1; in un simile contesto non si userebbe dunque l'errore di predizione ma un'altra misura.

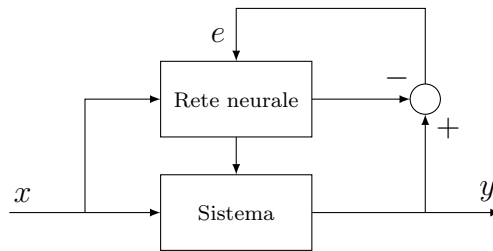


Figura 7.25: Controllo con rete neurale ad apprendimento supervisionato

I pesi possono dunque essere tarati seguendo altri criteri come ad esempio l'efficacia a cancellare la dinamica della G , pertanto l'approssimatore non sarà addestrato tanto a riprodurre un'uscita quanto a svolgere uno specifico compito. Meglio si svolge il task specificato, migliore è il modello, pur essendo questo eventualmente poco efficiente nel riprodurre il MGD.

Appendice

Appendice A

Teoria della probabilità

A.1 Correlazione e covarianza

Siano $x(\cdot)$ ¹ e $y(\cdot)$ due segnali (o processi) aleatori.

La **funzione di correlazione incrociata** (o **cross-correlazione**) tra $x(\cdot)$ e $y(\cdot)$ è definita nel seguente modo:

$$R_{xy}(\tau) = \mathbb{E}[x(t) \cdot y(t + \tau)] \quad (\text{A.1})$$

La **funzione di autocorrelazione** di un segnale $x(\cdot)$ è definita come la cross-correlazione fra $x(\cdot)$ e sé stesso:

$$R_{xx}(\tau) = \mathbb{E}[x(t) \cdot x(t + \tau)] \quad (\text{A.2})$$

Per $\tau = N > 0$, $N \in \mathbb{Z}$, si ottiene la **funzione di autocorrelazione in N passi**:

$$R_{xx}(N) = \mathbb{E}[x(\bar{t}) \cdot x(\bar{t} + N)] \quad (\text{A.3})$$

dove $\bar{t}$ è un istante di tempo fissato, rispetto al quale si correlano i segnali, traslati di N passi l'uno rispetto all'altro (se il segnale è stazionario allora la definizione è valida per ogni valore di $\bar{t} \in \mathbb{R}$).

Se un segnale è **stazionario**, la funzione di autocorrelazione dipende solo da τ : un segnale stazionario preserva le sue caratteristiche rispetto a operazioni di traslazione nel tempo.

La **varianza** di un segnale $x(\cdot)$ è, per definizione, il valor medio del suo quadrato e può essere definita a partire dalla funzione di autocorrelazione, ponendo $\tau = 0$:

$$\text{var}[x] = R_{xx}(0) = \mathbb{E}[x(t) \cdot x(t)] = \mathbb{E}[x^2(t)] \quad (\text{A.4})$$

Analogamente, la **covarianza** tra due segnali $x(\cdot)$ e $y(\cdot)$ è definita come segue:

$$\text{cov}[xy] = R_{xy}(0) = \mathbb{E}[x(t) \cdot y(t)] \quad (\text{A.5})$$

La completa caratterizzazione della funzione di autocorrelazione (o di cross-correlazione) di un segnale $x(\cdot)$ è un risultato utilizzabile solo teoricamente.

¹con la notazione $x(\cdot)$ ci si vuol riferire a un segnale inteso come elemento di uno spazio funzionale. La notazione $x(t)$ indica invece il **valore** assunto da uno specifico segnale $x(\cdot)$ all'istante di tempo t .

Nella realtà, il numero N di campioni è sempre finito, perciò la funzione di autocorrelazione è indicata tramite una sua **stima campionaria** (con N campioni):

$$R_{xx}(\tau) = \frac{1}{N} \sum_{t=1}^N x(t) \cdot x(t + \tau) \quad (\text{A.6})$$

Appendice B

Algebra lineare

B.1 Test di Sylvester

Un metodo che consente di determinare se una matrice è definita positiva nell'origine oppure semidefinita positiva nell'origine è il **test di Sylvester**, il quale afferma quanto segue.

- Condizione necessaria e sufficiente affinché una matrice A sia definita positiva nell'origine è che tutti i minori principali dominanti siano positivi.
Come corollario di ciò, una matrice A è definita negativa nell'origine se tutti i minori principali dominanti di $-A$ sono positivi.
- Condizione necessaria e sufficiente affinché una matrice A sia semidefinita positiva nell'origine è che tutti i minori principali siano non negativi.
Analogamente, una matrice A è semidefinita negativa nell'origine se tutti i minori principali di $-A$ sono non negativi.

Se A risulta definita positiva allora è anche semidefinita positiva.

Esempio 5. *test di Sylvester*

$$V(x) = 2x_1^2 - 2x_1x_2 + x_2^2 \quad \text{in } \mathbb{R}^2 \quad \Rightarrow \quad A = \begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix}$$

i minori principali dominanti sono:

$$\Delta_1 = 2 \quad , \quad \Delta_2 = |A| = 1$$

tutti i minori principali dominanti sono positivi quindi A è definita positiva (perciò è anche semidefinita positiva). $\square$

Esempio 6. *test di Sylvester*

$$V(x) = 2x_1^2 - 2x_1x_2 + x_2^2 \quad \text{in } \mathbb{R}^3 \quad \Rightarrow \quad A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

i minori principali dominanti sono:

$$\Delta_1 = 2 \quad , \quad \Delta_2 = |A| = 1 \quad , \quad \Delta_3 = 0$$

A non è definita positiva (perché il minore Δ_3 è nullo).

Calcoliamo i minori principali.

I minori di ordine 1 sono:

$$\Delta_{11} = 2 \quad , \quad \Delta_{12} = 1 \quad , \quad \Delta_{13} = 0$$

I minori di ordine 2 sono:

$$\Delta_{21} = \begin{vmatrix} 2 & -1 \\ -1 & 1 \end{vmatrix} = 1 \quad , \quad \Delta_{22} = \begin{vmatrix} 2 & 0 \\ 0 & 0 \end{vmatrix} = 0 \quad , \quad \Delta_{23} = \begin{vmatrix} 1 & 0 \\ 0 & 0 \end{vmatrix} = 0$$

Il minore di ordine 3 è la matrice stessa che ha determinante pari a 0.

Tutti i minori principali sono non negativi pertanto A è semidefinita positiva. $\square$

L'unica forma quadratica sia semidefinita positiva che semidefinita negativa nell'origine è quella associata alla matrice identità.

Vi sono forme quadratiche che non risultano né semidefinite positive, né semidefinite negative.